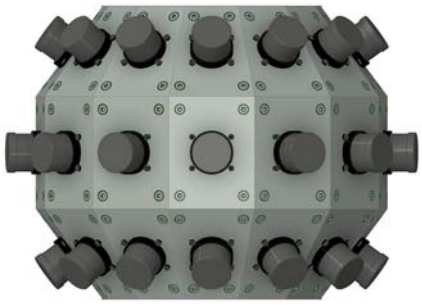
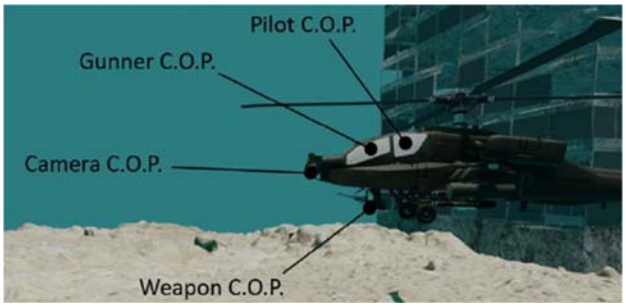
Geometric Calibration

This document describes geometric calibrations for the Apache strap-down pilotage contract, including evaluation of potential approaches.





There are four separate calibrations to be performed, as follows:

- **Camera intrinsic:** Individual camera fields of view and lens distortion.
- **Camera extrinsic:** Poses of cameras mounted on a common frame (left image above).
- **Frame extrinsic:** Frame w.r.t. aircraft/pilot (center image above).
- **Head-mounted display (HMD):** HMD w.r.t. either replay stream or live stream (right image above).

Summary

Intrinsic+Extrinsic: Four potential approaches are presented along with details on the selected approach (appendix A describes the optimization approach and its validation). An ***iOptron CEM40 or CEM70 stage with one (for Phase 1) or two (for Phase 2) fixed-location sources is recommended*** for data collection based on the following:

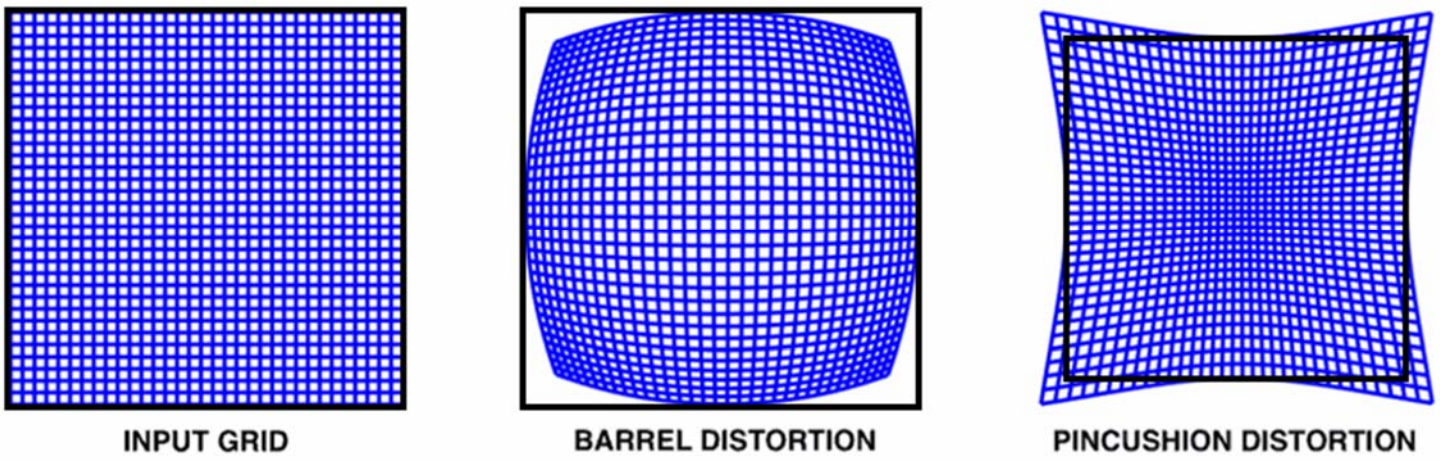
- Use of standard point-source IR calibration targets.
- Rotational accuracy to better than 1/3-pixel.

Frame extrinsic: Four potential approaches are presented. ***By construction is recommended, along with an online manual reset fallback approach***.

HMD: Three potential approaches are presented. ***By construction is recommended, along with an online manual reset fallback approach***.

Camera intrinsic calibrations

Radial distortion: Camera lenses usually produce radially symmetric distortions around their center of projection.

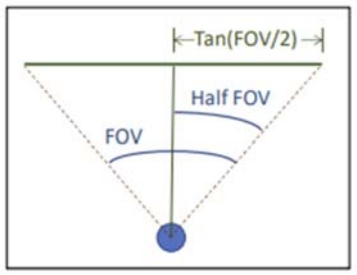


As seen above in this augmented Figure 1 from LearnOpenCV[1], these include *barrel* distortions (where the corners of the original image are pulled in towards the center and additional regions are visible in the image) and *pincushion* distortions (where the corners of the original image are pushed outward and portions of it are cropped).

Calibration of this distortion usually involves determining a polynomial with terms of even degree that specifies a new radius for each pixel given its original distance from the center of projection in the image. This maps pixels from the image into their location in an **ideal camera** that has no distortion. The choice of the field of view for the ideal camera involves a ***trade-off***: because the correction of a barrel distortion pushes pixels at the corners outward, the situation looks like the pincushion image on the right above – in this case the black square is the largest ideal camera whose pixels are all from the original image, but some of the pixels in the original image are pushed outside of the field of view of this ideal camera. Selecting a rectangular ideal camera that includes all pixels from the original image will also include regions that have no pixels in the original image.

Center of projection: Slight misalignment between the sensor and lens in a camera causes the center of projection for the lens to be located somewhere other than the center of the sensor. Because radial distortion pushes pixels away from this center non-linearly, the center must be calibrated along with the distortion correction to successfully remove it.

Fields of view (FOVs): The horizontal and vertical fields of view of a camera determine the region of space that its view frustum can observe. As described above, this is not uniquely defined for a camera with distortion correction. Here, this will be taken as the FOV of the ideal camera to which the radial distortion function maps pixels.



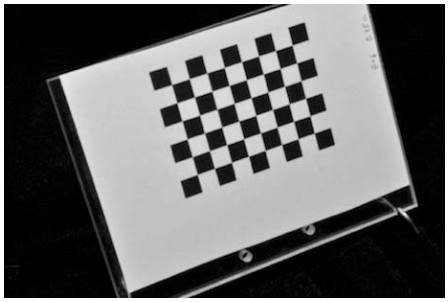
Requirements

Determining the radial distortion function requires ***sampling 3D points at known directions that project across the entire range of radii from the center of projection in the image***. This requires sampling at least along a line from the center of projection to the furthest corner of the image.

Determining the fields of view requires ***sampling 3D points at known directions with wide baselines*** so that noise in projected location estimation has small impact.

[1] https://learnopencv.com/understanding-lens-distortion

Potential approaches



Checkerboard: Several approaches are available that use multiple images of 2D checkerboard patterns with known sizes, viewed at various tilt angles with respect to the camera and covering a large fraction of the display. These include the following:

- **Matlab Computer Vision Toolbox:** This commercial solution includes a Camera Calibrator application.[2]
- **OpenCV:** This open-source framework includes a tutorial on determining camera parameters.[3] Other online resources are available that describe how to perform individual camera and two-camera calibrations.[4]

Producing this type of target for use with IR cameras has traditionally been challenging, but a recent article describes the construction of a precise thermal radiation checkerboard target.[5] The checkerboard is usually placed at ten or more locations and orientations to produce a consistent calibration.

Included in extrinsic calibration: The methods proposed below for extrinsic calibration of the set of cameras with respect to the frame that they are all mounted to could also be used to calibrate the intrinsic parameters for each camera.

[2] https://www.mathworks.com/help/vision/ref/cameracalibrator-app.html
[3] https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html
[4] https://temugeb.github.io/opencv/python/2021/02/02/stereo-camera-calibration-and-triangulation.html
[5] https://www.mdpi.com/1424-8220/23/7/3479

Camera extrinsic calibrations

The cameras are rigidly mounted to the same frame, which is then attached to the helicopter. The relative **pose** (position + orientation) of each camera with respect to the frame coordinate system is required so that the pixels from multiple cameras can be reprojected to form a single unified image.

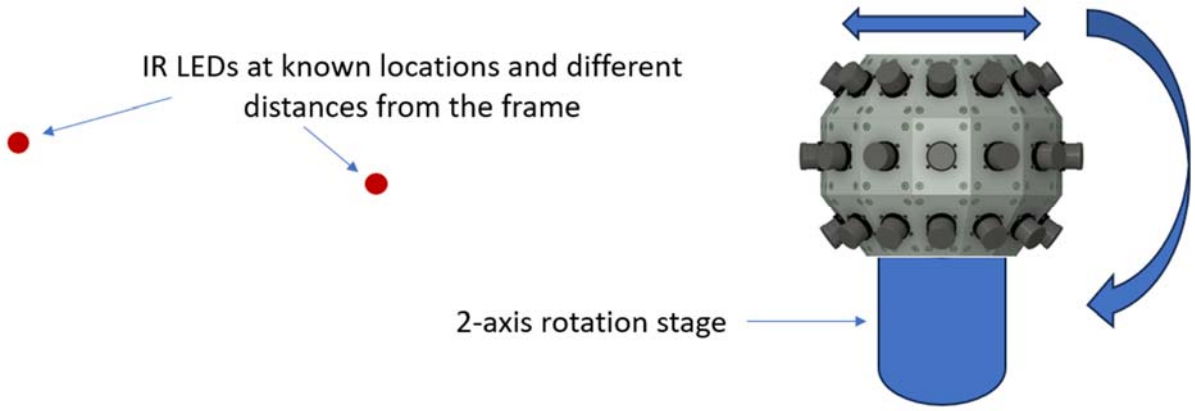
Requirements

Determining the camera relative poses requires ***sampling a set of 3D points with known relative orientation and offsets that project into sufficiently diverse regions within all cameras (or all pairs of camera overlaps)***. This can be done pairwise between cameras that can see a common target or groupwise among a set of points that span the camera views.

If intrinsic calibration is performed at the same time, its requirements must also be met for all cameras. Furthermore, the points must provide the ability to reliably discriminate between Z motion and field of view changes, requiring ***points at different distances from the cameras***.

Potential approaches

Checkerboard: The checkerboard approach described under *Camera intrinsic calibrations* can also determine the camera's pose with respect to the target. For a single camera, this enables us to determine its extrinsic parameters if we know the frame of reference of the checkerboard. When the same checkerboard can be viewed by more than one camera (as is the case for stereo cameras), it can therefore be used to determine the pose of each relative to the other. However, the small overlap between camera views in the strap-down pilotage systems means that no two cameras can see the same pattern at the same time.



Point target(s) + gimbal: Another approach is to place an IR light source at a known relative position compared to a 2-axis rotation stage on which the frame with the cameras is mounted. (If intrinsic calibration is being performed at the same time, two sources at different distances from the frame should be used to distinguish between Z translation and field of view differences.) The motorized stage can then be rotated to sample the desired subset of the entire field of regard with a desired step size; each point's location in all cameras is used to provide the mappings between the 3D locations and 2D projections.

This approach was used to calibrate both the intrinsic and extrinsic parameters of the multi-view infrared *HiBall* ceiling tracker's sensor that included six lateral-effect photodiodes embedded in an enclosure with six lenses.[6] Its rotational positioning motors were rated at 0.0055 degrees but were calibrated using a survey-grade theodolite to within ***0.0017 degrees***. Measurements in a 0.1-degree grid were repeated 100 times each to reduce noise in the captured signal.

[6] https://www.cs.unc.edu/~welch/media/pdf/VRST99_HiBall.pdf

Potential controlled-rotation platforms include the following:

- **Flir PTU-D100EX:** Maximum payload 25 lbs, pan resolution 0.0075 degrees, tilt 0.0038 degrees.[7]
- **Zaber:** Offers a range of stages and multi-axis custom solutions.
- **Newport:** Offers a range of stages and multi-axis custom solutions.[8]

Not selected: A ***Zaber X-G-RST-DE*** series stage was initially selected for this project.[9] Its specifications include 0.01 degree accuracy, 0.005 degree repeatability, 0.005 degree backlash, and 24 degrees/second maximum speed with 33.6 lbs. maximum centered load. This is approximately ***1/3 pixel accuracy*** for a camera that has 1280 pixels over a 40 degree region. It will be modified to mount the yaw rotation stage on the pitch-rotation platform so that it first tilts out of the plane and then rotates the camera around the new vertical axis, with the camera centered on the fork. The target will either be large and on a building far away or will be viewed through a collimator with a large enough exit pupil to handle the camera offsets during rotation. The 45-lb. weight of the IR camera exceeds its maximum.

Phases 1 and 2: An ***iOptron CEM70***[10] has been selected for this project. Its specifications include 70 lbs. maximum load and 0.0021 degree tracking accuracy. The CEM40 was selected and will be used for the visible-light camera, but the IR camera ball is 45 lbs. so required the CEM70.

Subpixel-accurate target localization from images requires a target that is several pixels in radius; a far subpixel target can at best be localized with single-pixel accuracy. On the other hand, a larger target cannot be localized near the edge of an image because it will not be entirely contained within the image; the smaller the target, the better. Two spherical targets with ***2-3 pixel radius*** have been selected for this project; each will include a ***constant lower-intensity circular surround***. The two targets will be different sizes so that their projected sizes match even though their distances differ.

The targets and gimbal will be mounted using bolts to the same stable, vibration-isolated substrate: a ground-floor concrete floor or optical bench or other stable platform ***without vibrations of more than 0.5mm***. The mounting platforms for the gimbal and targets will be rigid and free of vibration larger than 0.5mm. The targets shall be placed at distances of 3 meters and 15 meters separated by an angle sufficient that the spacing between backgrounds is larger than the diameter of the larger background, facing the gimbal center of rotation. Their positions shall be known ***within 5cm laterally and 1cm axially*** compared to the center of rotation and 0-degree aiming axis for the gimbal and the plane of the floor.

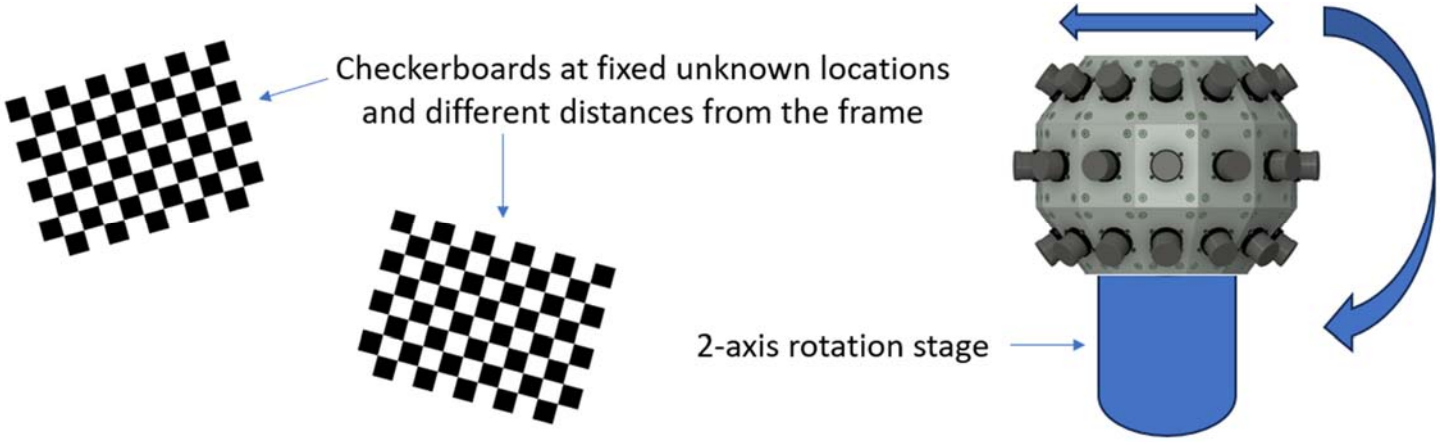
The camera mount shall meet the following criteria:

- ***(Phase 1) Total mass less than 50 lbs. (including any required balancing mass), with mass centered*** on the center of rotation.
- Camera and mount must not interfere with the gimbal or table/floor when ***rotated up or down by 60 degrees***.
- Going to the home ***limit-switch orientation must not crash the camera***.
- Rigidly mounted to gimbal at an orientation that is ***within 2 degrees*** of its coordinate system, with its principal ray pointing towards the vertical-tilt axis of the gimbal.

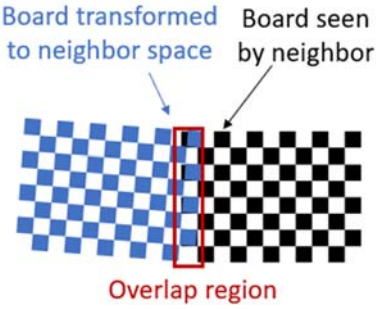
[7] https://www.flir.com/products/ptu-d100e/
[8] https://www.newport.com/s/assemblies-xy-stages-xyz-stages-gimbals
[9] https://www.zaber.com/products/gimbal-stages/X-G-RST-DE/specs?part=X-G-RST500-DE50SR10
[10] https://www.ioptron.com/product-p/c401a1.htm

A custom C++ program will be run on the Storage Server using the ASDP Render Module to control the gimbal stage (using a USB-to-RS232 adapter if required) and synchronously collect images from cameras that are expected to be able to see the targets based on as-built specifications. These images will be stored on the 4TB data drive along with a CSV file describing the gimbal orientation when each was taken. Based on as-built specifications with margins of error, each target will be moved to angles that should project to evenly spaced points along lines near each edge and along both diagonals within each camera's field of view.

The optimization approach shall read the CSV file and images along with an as-designed JSON configuration and produce an as-built JSON configuration that includes distortion correction and camera pose updates that ***produce pixel reprojection errors of less than half a pixel*** for pixels along the edges of overlapping images. The approach for each phase is described in Appendix A.

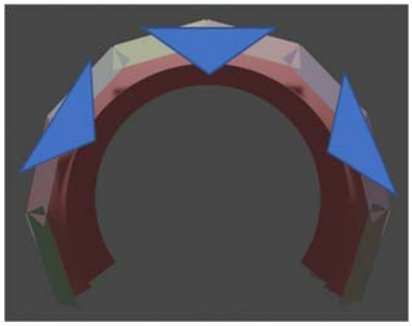


(Not selected) Checkerboard + gimbal: Combining the two approaches above, two fixed-location checkerboard targets can be placed at different depths and with different tilt angles w.r.t. the stage. By rotating the stage, the relative pixel locations of the targets are adjusted so that multiple views of each are available within each camera; this provides the views necessary for ***intrinsic*** camera calibrations.



For ***extrinsic*** camera calibration, sufficient overlap between cameras must occur for there to be sufficient features seen by both cameras to distinguish both vertical and horizontal shifts. (This may require the ***addition of microtexture*** within the checkerboard to provide sufficient features.) A ***custom optimization*** can be performed pairwise (and then across groups of cameras) to adjust the relative poses of each camera relative to the others by reprojecting the image from one camera through the forward distortion, forwards pose, inverse neighbor pose, and inverse neighbor distortion to produce an image from the first camera in the neighbor's point of view. The difference between the two images in the region of expected overlap can be used as an error metric to solve for the optimal relative pose between the pair of cameras.

This approach has much less stringent gimbal angle accuracy requirements than the point-target approach. It is only necessary to align each checkerboard within coarse regions inside each camera's viewpoint and then to align them so that they are visible along the edges of each pair of cameras. This may require a larger target distance from the frame to enable sufficient overlap at the near distance. It may require ***two different-sized checkerboard*** targets to enable the far one to be sufficiently large. A checkerboard target large enough to cover about 1/3 of the camera (10 degrees) when displayed at 20 feet would be about ***3.5 feet across***.



Checkerboard + gimbal + wFOV VIS: Augmenting the 2-axis rotation stage in the checkboard + gimbal approach with a set of 3 high-resolution wide-field-of-view ***visible-light cameras*** (whose zoom and focus are fixed) that have overlapping fields of view with each other and with all cameras on the frame would enable standard checkerboard-based intrinsic + extrinsic camera calibration to be used among all cameras in the system.

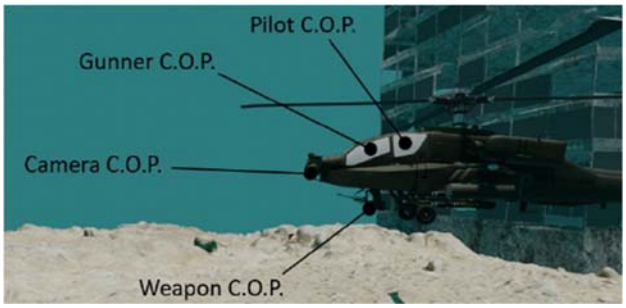
The three wFOV cameras would first be aligned to one another. This may require the mounting of two more larger-sized checkerboard targets at different distances to provide sufficient baseline for intrinsic calibration. (These boards could be printed on paper and mounted to plywood or thick foam-core boards, so may be less expensive to produce than the laser-etched IR targets. Alternatively, the intrinsic calibration for the wFOV cameras could be performed in a separate step using hand-held closer placement of the IR targets before they are mounted in the environment.)

Each narrow-field camera would then be aligned to one of the wFOV cameras by determining the pose of each w.r.t. a checkerboard target that is seen by both. The average over multiple stage orientations could be used to reduce measurement noise. The wFOV cameras' higher resolution would enable use of the smaller targets while retaining accurate pose estimation during this step.

This approach requires neither the microtexture nor the custom optimization approaches from the checkerboard + gimbal method, but it still requires ***two different-sized checkerboards***.

Additional custom optimizations could make use of the fact that the targets remain at the same fixed locations throughout the scan and could make use of images that can view both targets.

Frame extrinsic calibration



When the frame holding the cameras is mounted, its ***orientation relative to the aircraft*** must be determined so that pixels from a given camera can be projected in the correct direction.

Potential approaches

Image-based: An image of a ***checkerboard target*** that is manually placed in front of the helicopter, using a bubble level to ensure verticality and by-eye estimation of centering and perpendicularity. This ***relies on manual placement*** and so is not expected to be precise.

IMU: When flying live, the relative orientation of the camera frame and the HMD is what is relevant. There will be "inertial" measurement units (IMUs) on both the frame and the HMD, so the difference between their orientations can be used. However, this ***assumes that the north vector computed by the two magnetometers matches***, and this is unlikely to be the case. This approach ***does not work for replay*** because it does not provide a way during recording to know the orientation relative to the helicopter.

By construction: Because the frame will have known mounting points and angles, and because a mounting bracket is expected to be constructed, the geometry can be fixed during construction. If needed, shims along with a level could be used for fine adjustment.

Online manual reset: The live (or replay) system can provide a control for the pilot to use to recenter the orientation based on their current viewing direction, where they orient their head to be looking at the images that match straight ahead and level. This could be provided as a fallback approach for both live and replay modes.

Head-mounted display (HMD) calibration

Its ***orientation relative to the aircraft*** must be determined so that pixels from a given camera can be projected in the correct direction.

When using the advanced rendering technique of ego-centric reprojection for the pilot, the ***camera frame location relative to the pilot's resting head location*** must also be known.

The HMD is presumed to have a built-in absolute tracking system that reports its position and orientation relative to a base station that is rigidly attached to the helicopter. The calibration is from the pose of this base station to the frame of reference of the helicopter.

Potential approaches

Image-based: A profile image of the helicopter can be taken from a camera whose lens is vertical and whose X axis is horizontal (along the ground plane that the helicopter is resting on), allowing measurements to be made in plane. The assumption is made that both the camera and pilot's head are in the centerline of the helicopter. The assumption is made that the dimensions of the helicopter are known. In the image, the pilot will be wearing the HMD and will be looking straight ahead and level. The dimension of each pixel in the image will be estimated by counting pixels between two locations on the helicopter with known separations. Then the count of pixels in X and Y from the origin of the camera frame to the center of the pilot's head will be scaled by the inverse of this factor to determine the distances in X and Y between the two. The HMD location will be recorded and its source-to-head transform used to determine the offset from the pilot's head to the HMD base, which will be added to the calculation to produce the frame-to-HMD-base offset.

By construction: Because the base will have known mounting points and angles, and because a mounting bracket is expected to be constructed, the geometry can be fixed during construction. If needed, shims along with a level could be used for fine adjustment.

Online manual reset: The live (or replay) system can provide a control for the pilot to use to recenter the orientation based on their current viewing direction, where they look straight ahead and level. This could be provided as a fallback approach for both live and replay modes.

Appendix A: Optimization Approach

Helicopter space is defined to have its origin at the center of rotation of the ***gimbal*** (with its +Y axis facing such that moving the vertically tilting axis tilts it vertically) in the plane of the floor when the gimbal is at its (0,0) orientation. The +Z axis will be away from the floor and the +X axis perpendicular to the Y axis. This should be the space that the individual microcameras are defined in. Any mounting of the camera unit at a different location or orientation will be calibrated out as part of the optimization. See README.md in the ASDP_Render_Module repository as well.

The gimbal shall not be reset during the entire multi-step image collection process described below, so that the coordinate system for all stages remains the same without variation due to the magnetic limit switches triggering at different orientations for different runs.

For Phase 1, the following approach will be used to optimize stitching at a single depth based on a single target:

- Step 0: Initial target and camera orientation estimation:
  - Capture a set of (3-10) images with the center of each camera pointing at a target.
  - For each image set:
    - Average the set into a double-precision mean image.
    - Determine the center of the target using a symmetry-optimizing kernel.[11]
    - Compute the ray from the camera center through the image-space target center.
    - Intersect that ray with the plane through the target center perpendicular to the line from the gimbal center through the target location.
  - Iterate the estimation of the lateral target location and the camera pan and tilt angles:
    - Using the most recent camera pose estimation, find the mean location in the plane of all ray intersections. This is the estimated target location (whose distance from the gimbal origin is largely unchanged).
    - Using the most recent target location estimation, adjust the up/down and side-to-side tilts on the camera orientations so that they point directly at the estimated location.
  - Record the Estimated target location and the new estimated camera poses, producing new target location and camera configuration files for use in the following.
- Step 1: Collect images at regular steps along lines that are a specified margin (one target diameter past any clipping) from all edges of each camera, ignoring distortion and relying only on FOV estimates, from all cameras that can see the point. For each camera, avoid collecting any point that is within 2 degrees of a point requested by another camera (to avoid conflicting mappings). Also select points at a coarser (X,Y) step size within each camera (for that camera only) away from the edges. For each orientation:
  - Find the ray from the camera's center through the desired pixel for each point.
  - Determine the gimbal orientation that points that ray at the target within a specified tolerance.
  - Collect a set of (3-10) images from appropriate cameras.
  - Average the set into a double-precision mean image.
  - Determine the target center location using a symmetry-optimizing kernel.
  - Reject any points that are too close to the edge of the image (within 10 pixels) to avoid incorrect optimization locations.
  - Record the target ID, microcamera ID, gimbal orientation, and target center image location.
- Step 2: For each camera:
  - Use the as-designed camera position, orientation, and fields of view as its camera parameters.

[11] https://github.com/CISMM/video/blob/master/spot_tracker.h#L289

- Construct a “bag of mappings” distortion correction that takes as its first point the projected as-seen location of a point within the image sensor and as its second the expected projected target position from each entry based on the target 3D location and gimbal orientation. This will ensure that pixels which should be coincident between cameras based on the model are coincident (to within pixel-location measurement error) because they will have both seen each edge target. It will also correctly adjust the internal points to match the visible lens distortions at the specified depth. It does this by moving each as-seen target to lie along its real-world direction from the camera.

For Phase 2, the following approach will be used to estimate all intrinsic and extrinsic camera parameters:

- For each target:
  - Step 1: from the Phase-1 algorithm, augmented either with a prompt that asks the technician to turn on a specific target and press return or with a distance-based threshold that prevents us from selecting other targets when the one we are looking for is outside of the camera’s viewpoint because of an incorrect initial guess. Waiting for the specific target is the safer option.
- Step 2: OpenCV
  - The OpenCV calibrateCamera() and related functions will be used to optimize the intrinsic, extrinsic, and distortion parameters for each camera. All points that are visible will be brought into the local camera coordinate system. OpenCV will optimize the parameters. The offsets to the local coordinate system will be transferred back into global Helicopter space. Distortions and center-of-projection offsets will be mapped to a DistortionBagOfMappings using the OpenCV initUndistortRectifyMap() function to produce pixel coordinate mappings between a camera with a centered center of projection and the real camera.
- Step 3: Embed step 2 inside an additional optimization loop:
  - Optimize the target lateral locations w.r.t. the gimbal’s helicopter coordinate system. Use the sum of the RMS reprojection errors across all cameras as the error measurement. Use the DownhillSolver to adjust the offsets to minimize the sum of errors across all cameras.

The resulting optimized estimates for all parameters will be written to new JSON configuration files for the cameras and for the targets.

Validation of Phase 1

The optimization was first tested using a simulated rendering environment that produces images from any camera at any selected rotation based on a Rendering Module JSON camera configuration file and a target-describing JSON configuration file. This includes individual camera translation, rotation, FOV, and distortions. This simulation reads in a list of gimbal orientations and sets of cameras and will produce a set of images at each pose including specified random jitter in gimbal orientation and specified per-pixel noise in each image.

The JSON configuration files provided to the optimizer have known error offsets applied to the individual camera FOVs, positions and orientations along with different distortion models (no distortion). They also have known error offsets to the target positions. This tests whether the optimizer can measure and correct these errors.

The optimizer reads the JSON input files along with the camera orientation and image files and produces optimized JSON files for the camera configuration and for the target locations.

The optimized JSON file for the camera is compared with the one fed to the simulator by a *Compare_Configurations* validation program in the Render Module that reports the mean and max 3D spatial distance in meters and projected difference in pixels between pixel locations along the edges of each camera, and then the average mean and overall maximum across all cameras. The projected pixel differences indicate how much misalignment is expected to be visible to the pilot.

The optimized JSON file for the targets is compared with the one fed to the simulator using a spreadsheet if required to aid in debugging problems with the estimation.

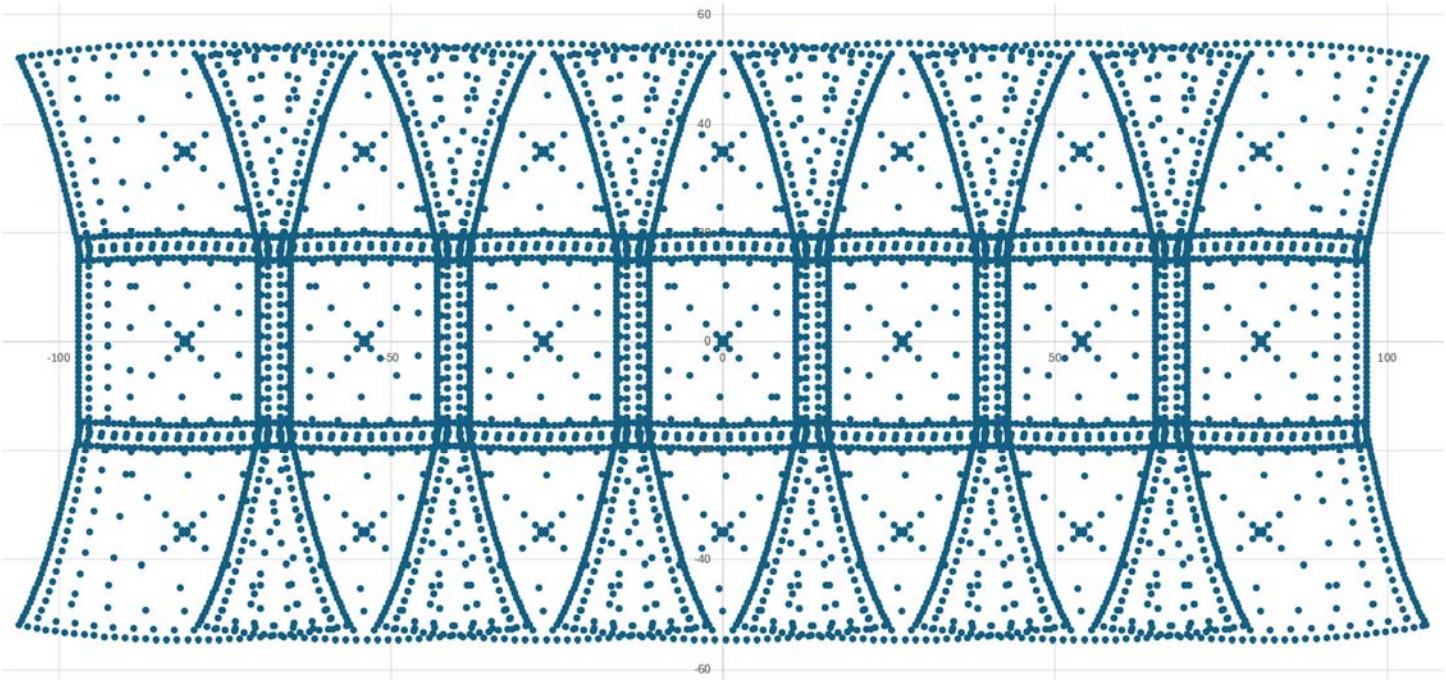
To test for correction of an incorrect alignment of the camera body mounted on the gimbal, the positions and orientations of all microcameras are rotated around the center by a specified rotation in the configuration file fed to the optimization procedure.

Results

Lateral target estimation

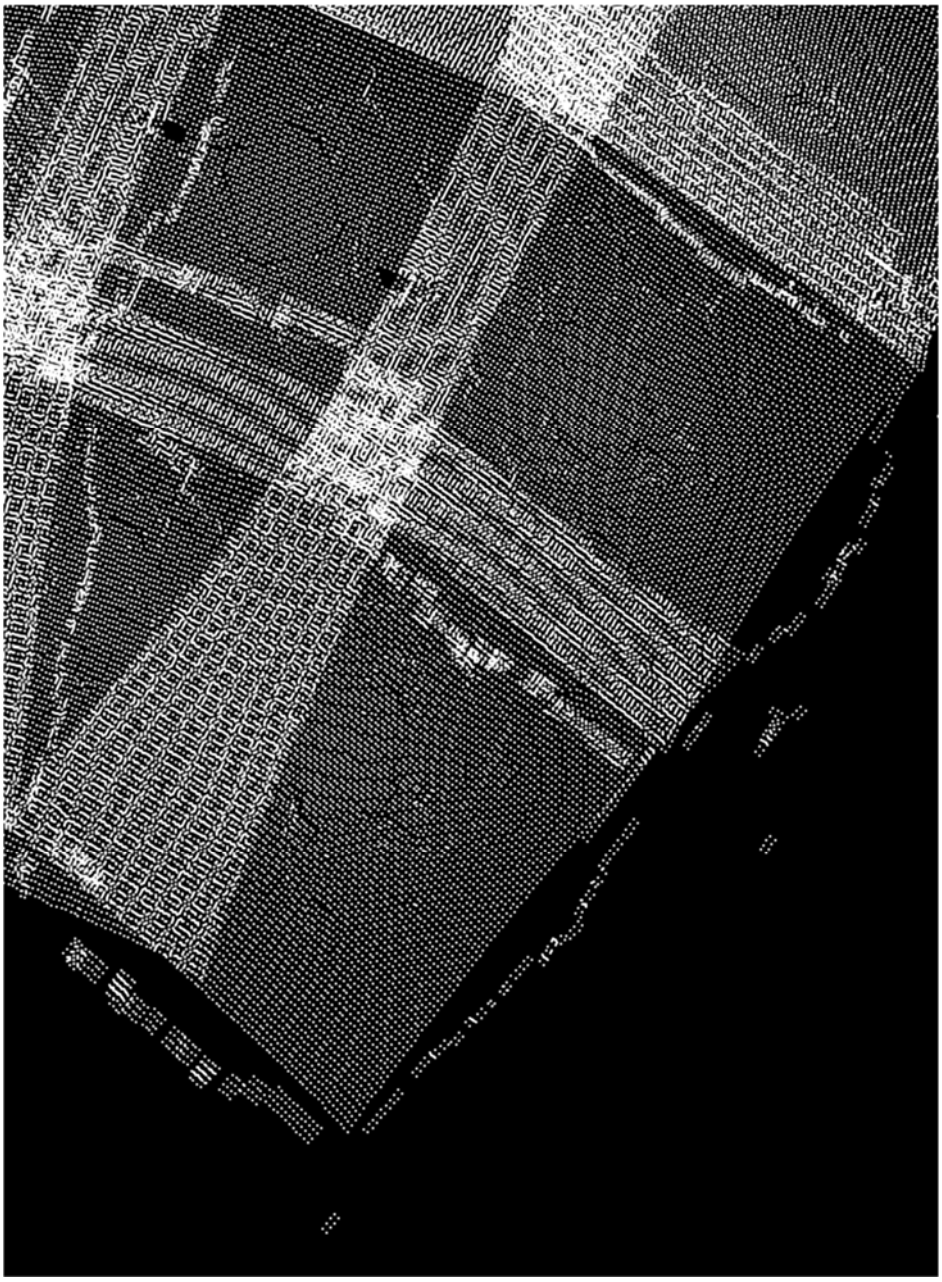
(Both phases) Using a 21-camera simulated input file with the camera center moved 20cm in front of the gimbal rotation and 0.01 degree precision for the gimbal rotations along with targets at 3M and 15M, the target lateral optimization was accurate to < 0.2mm in both directions at 3M < 0.31mm in both directions at 15M (***less than 1/4 pixel reprojection error*** in both cases).

Mesh estimation (phase 1)



The sample locations are dense around the edges and sparser towards the middle of each field of view, shown above in angle space. As they approach the center, there are only samples along the diagonals. Points that are seen by more than one camera are imaged in all cameras that can see them. The minimum step size, which is taken along the edges, is 30 pixels between points, around 43 samples on the long edge and 34 along the short edge. The number of samples is increased near the edge both radially (to handle nonlinear distortion with increasing magnitude) and axially (to provide precise stitching along the edges).

Nearest 3 non-collinear: Using the three nearest points from the bag to the mesh point being mapped whose dot product magnitude is less than 0.8 produces jagged artifacts along the edges and less-frequent folding within the interior of the meshes.



The image to the right shows some of these detached edge patches and interior holes caused by shifting of patches.

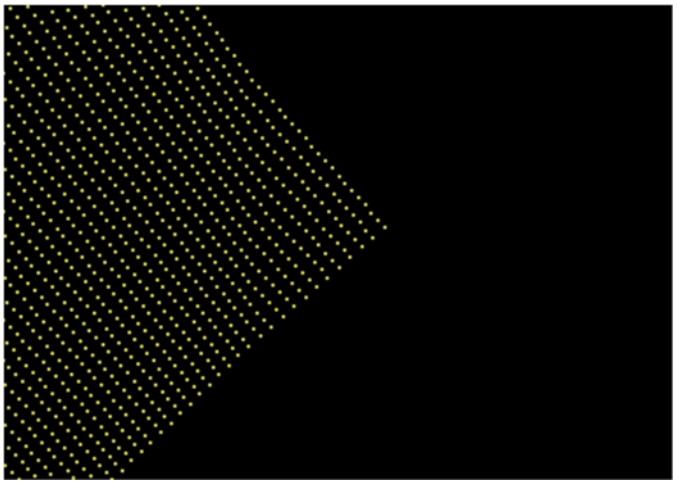
Changing to dot product threshold from 0.8 to 0.5 changes the arrangement but does not solve the issue.

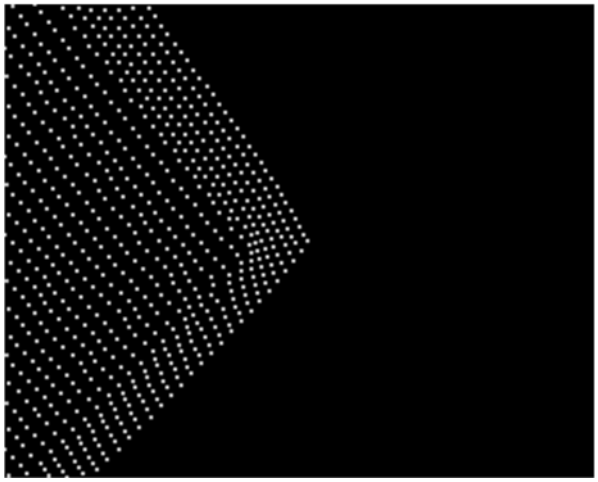
This approach was an attempt to handle highly non-uniform point sampling and enable extrapolation. It works for test cases that have very regular fields, but the point selection and short baselines between some point pairs exacerbate any error (even subpixel error) over long distances.

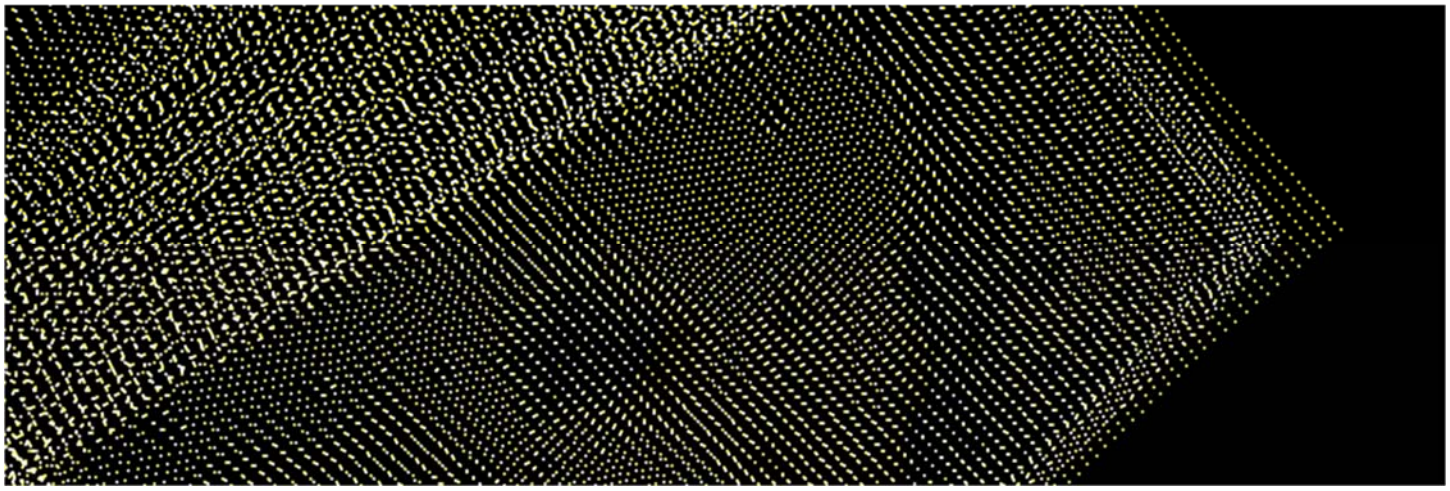
A method that ensures interpolation within the surrounding triangle and that is more robust to local point errors is called for.

All-points average offset: Switching from an interpolation based on the three nearest non-collinear points to one based on all points that estimates the offset by using the average offset at each point in the bag of points weighted by the inverse square of their distance from the mesh location being estimated produced a much smoother mesh that avoided the large outliers at the edge and most of the obvious interior folding. However, it introduced new artifacts:

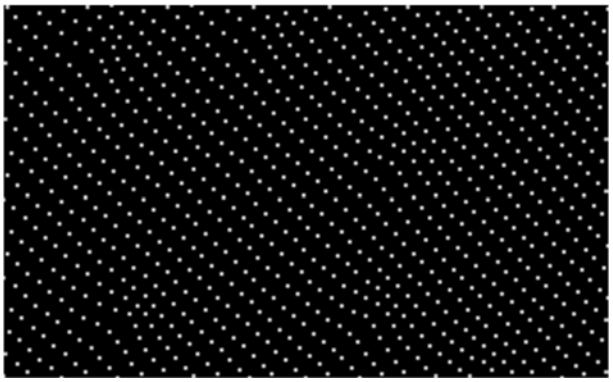
1) There is constant extrapolation past the outer sample range, producing a “squashed” region on the outside compared to the actual distortion field. The points were quite close along the sampled edge, so this may not be fatal to the approach. The two images below show the actual distortion field in yellow on the left and the resampled one in white on the right. The two are shown superimposed over a wider range in the image below. Within the interior, the points are usually very close.



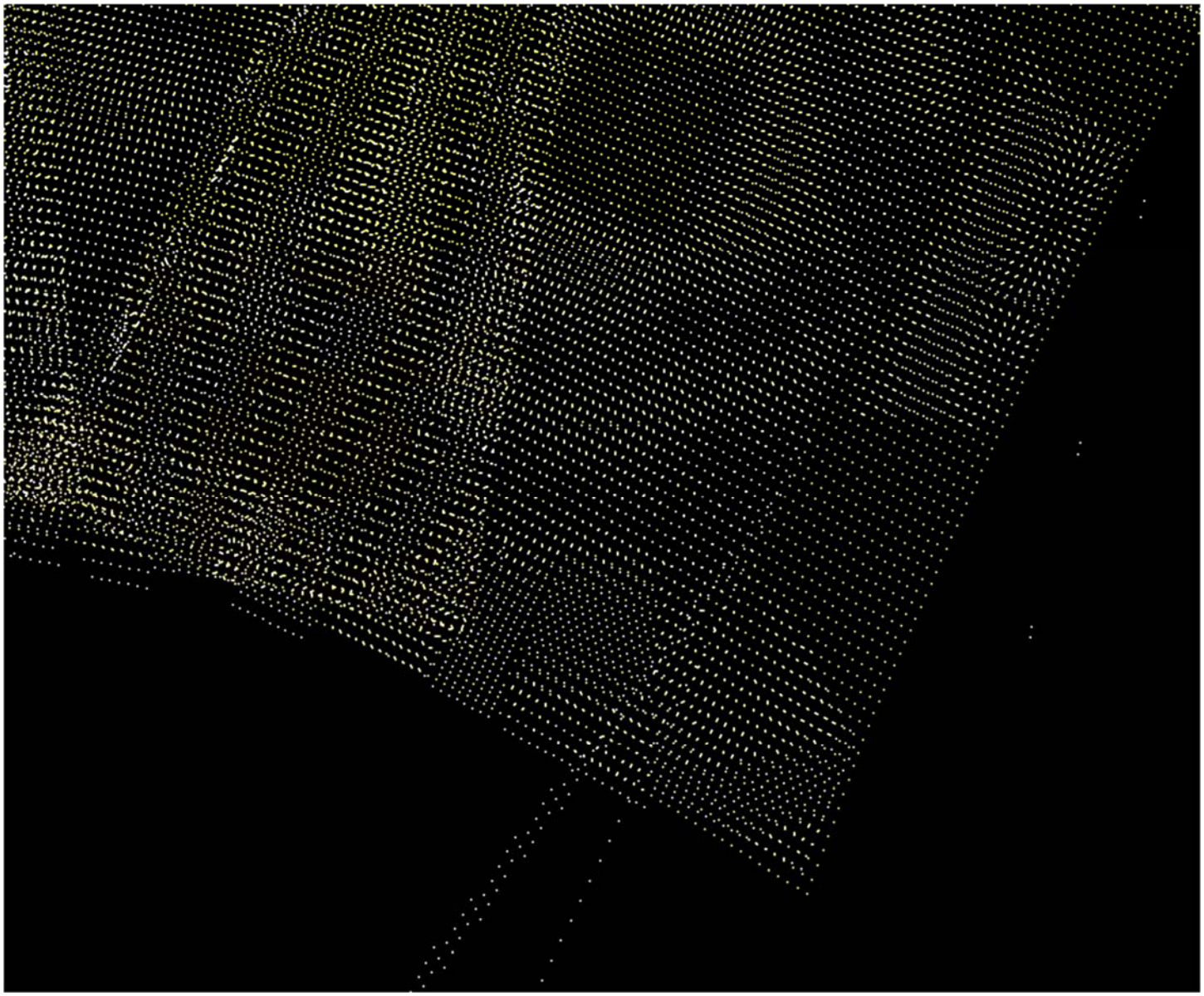




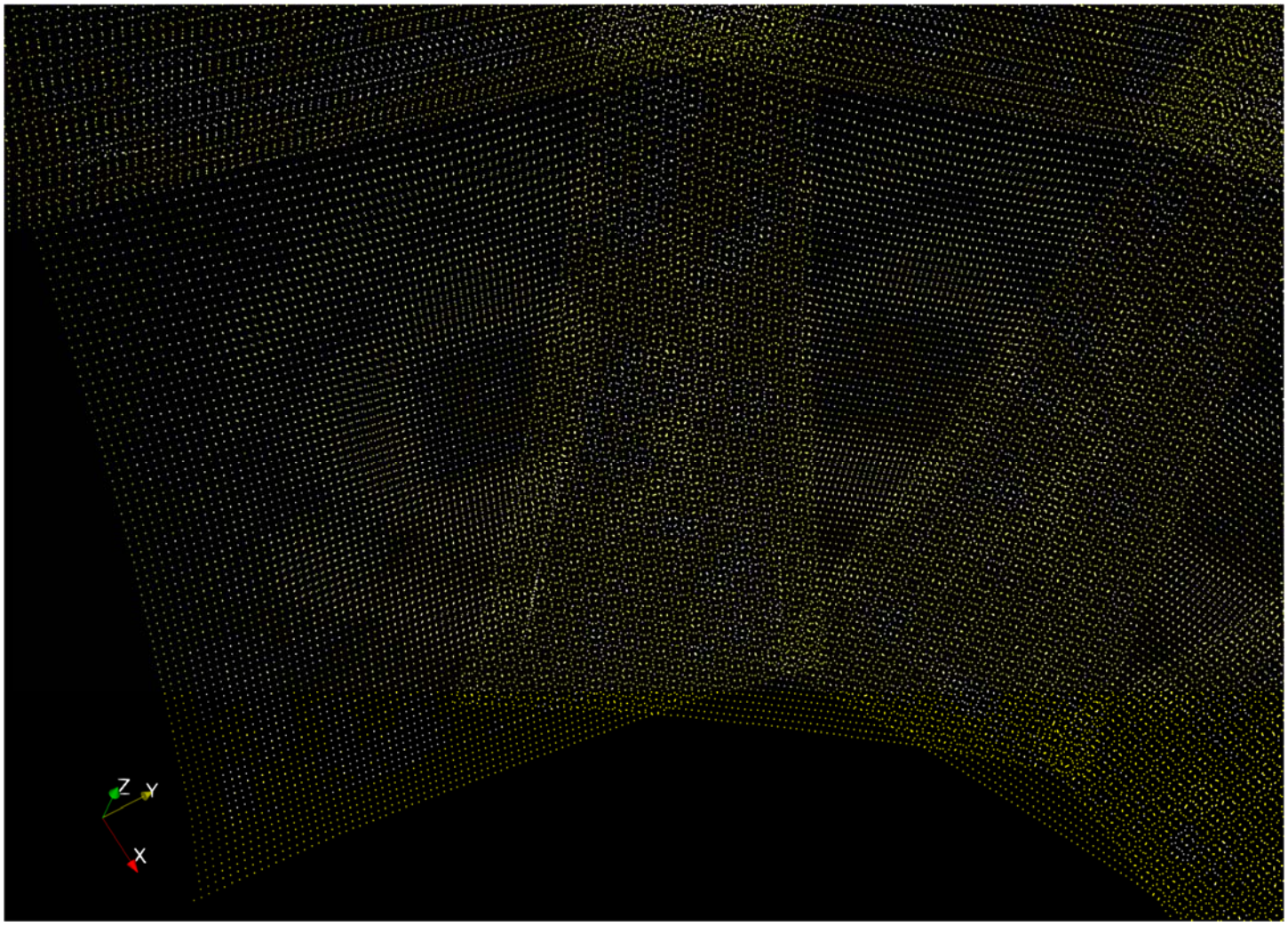
2) Within the sparsely-sampled regions of the interior, there are not enough local points to stabilize the interpolation, resulting in straight lines becoming wavy as they are pulled towards far sets of points. This does not occur as strongly near the far corners of images that do not have multiple overlapping data points (no dense bands like on the left of the image). This can be seen in the image to the right.



Delaunay/Voronoi: Computing the Delaunay triangulation of the bag of points and then interpolating on the nearest triangle (which may be a triangle that the point is inside). This has smoother internal interpolation without the wavy lines and some of the extrapolation matches reasonably well. However, other sections of the extrapolation (presumably associated with particular triangles) go to extreme distances and sometimes in the wrong direction. The image on the right below shows the ragged edge of the long sides of the three cameras on the right and the image to the left shows a zoom in on the lower corner. There are large regions of poor extrapolation (almost the entirety of the upper image) and fewer along the short edges (which seem to have point discontinuities with gaps but not extreme direction problems).



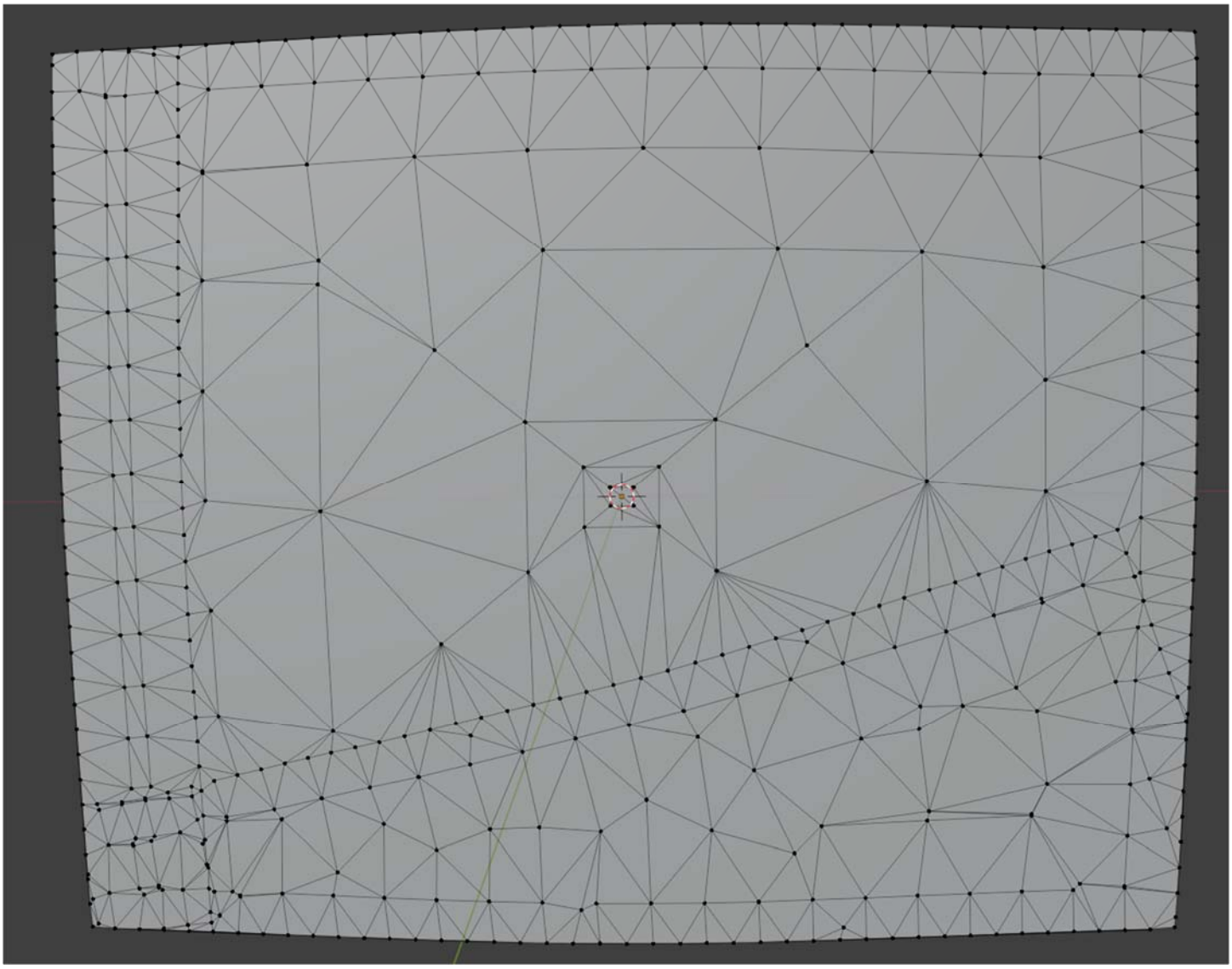
When we avoid extrapolation by setting all points outside of triangles to zero, we get a clear picture of what is happening inside and no extraneous points. The points lock down correctly for regions where there are dense samples, and there are radial differences within regions where there is coarse sampling:



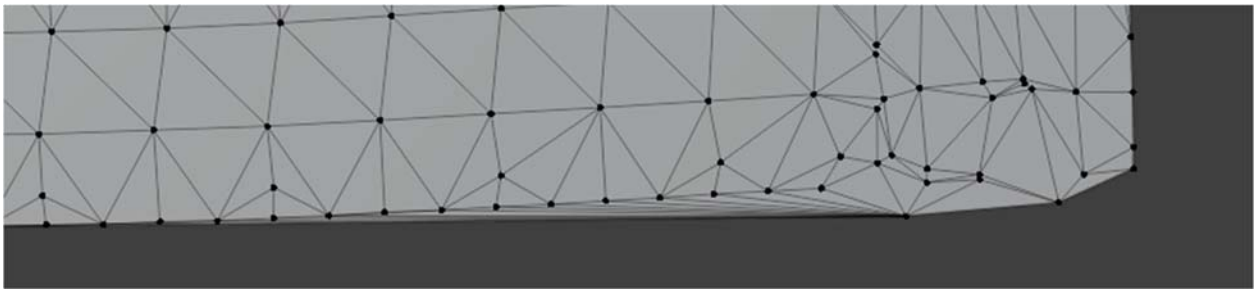
The sampled region of each image covers the overlap region between neighboring cameras, so it may be possible to simply clamp the coordinates to the edge of the image. This will trim slightly around the total field of regard but should provide sufficient stitching. Indeed, this is the case – clamping the edges to a point on the nearest triangle causes artifacts at image edges which are hidden for internal seams and only appear at the periphery.

Coarser sampling at the image edges should result in less relative error for each triangle, making them more suitable for extrapolation. This will decrease the responsiveness to curvature in the distortion near the edges but will not reduce the precision at the locations that are shared between cameras. However, ***the extrapolation is very good for some border regions and extremely bad for neighboring ones, which seems inconsistent with normally distributed triangle-point errors***. It seems like many of the edges are using incorrect triangles or points for extrapolation in a way that seems largely random. Indeed, the Bowyer-Watson algorithm the library was using does not deal properly with collinear points, and we have lots of those.

The Delaunay triangulation for at least one of the meshes looks reasonable:

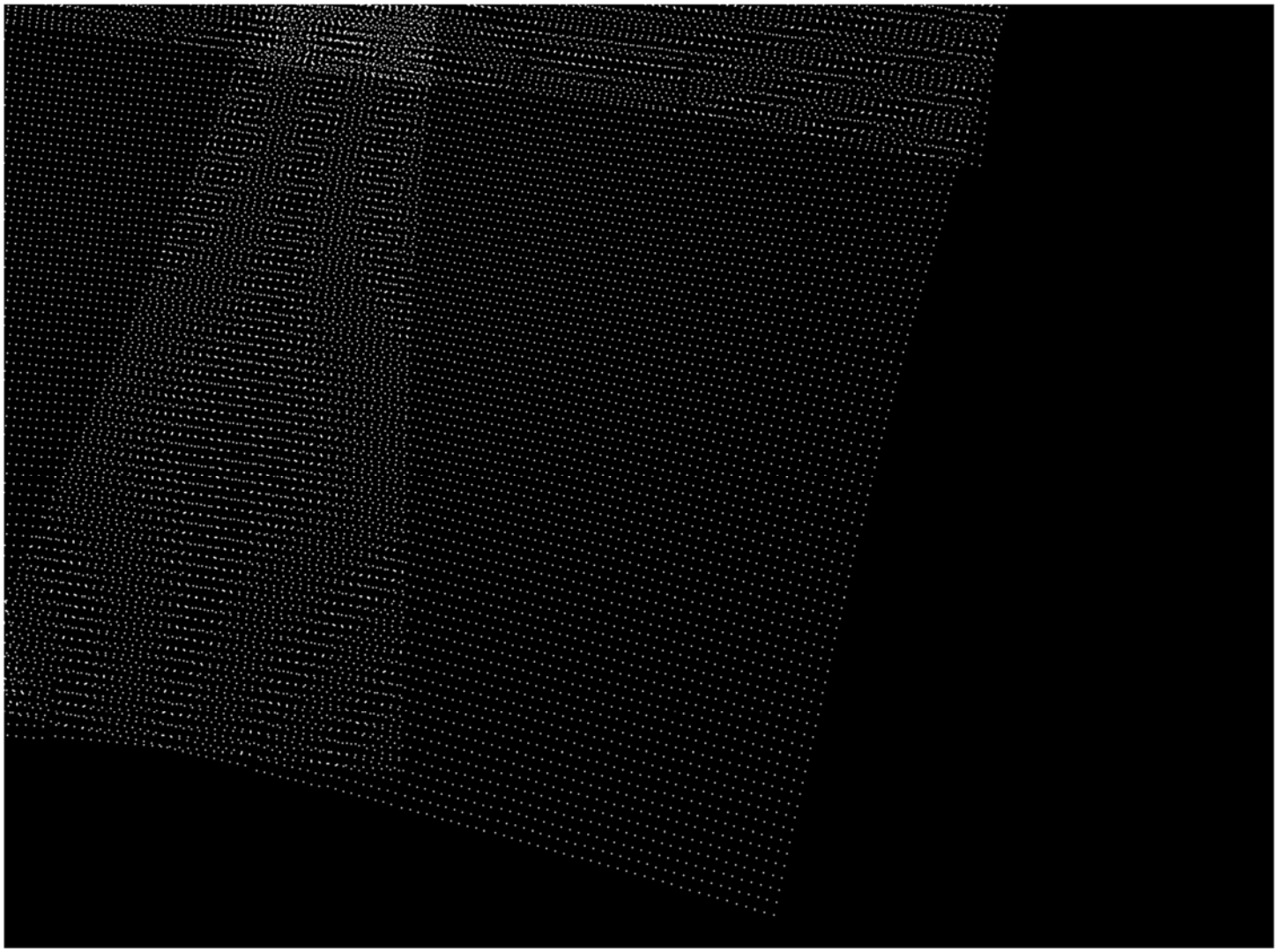


However, others have problems at the edges and have a thin sliver of triangles fanning towards the edge:



Most importantly, there seem to be some degenerate triangles at some of the edges, where there are overlapping triangles that have been improperly split. When searching for the nearest triangle to use for extrapolation, we should ignore any whose areas are very small.

By ignoring triangles whose areas are very small, we remove the irregularities in the extrapolated regions and produce meshes that are smooth all the way to the edge:



These have much smaller average (1.51) and maximum (24.6) pixel errors across all points in all meshes. The errors are most pronounced at the edges (past the stitching overlap region) and in sparsely sampled interior regions. When a static scene is stitched together, the borders of the entire field of regard are mostly smooth and the seams between cameras disappear at distances that match the stitching distance. Along the (finely-sampled) seam regions, there is very little difference between the two meshes (no visible difference when overplotting the mesh points).

Camera + mesh estimation (phase 2)

In January 2026, a 9-camera simulation was run of ideal cameras with no distortion and accurate poses and fields of view using two targets that were both within the range of visibility for the camera, one at (0, 20, 0) and the other at (2, 7, 0). The orientation accuracy for configuration file was set at 0.01 degrees for all axes. The standard per-pixel noise was added to each image, producing a set of 3 images per camera per view.

The desired target was visible in each simulated image, and the second target was sometimes visible at a distance from the first. The measured points in the images were all within 0.4 pixels of the expected locations, with their errors spread fairly evenly over the interval for both X and Y.

The OpenCV termination criteria included both a maximum count of 3000 iterations and a step threshold of 1e-7 (near the range of single-precision floating point). The meshes from the resulting calibrated camera configurations were compared to the ideal mesh that was used to drive the simulation. The mean and maximum errors in pixel counts across all 9 cameras depended on the flags from the run. The optimization was initialized with parameters that were all correct.

In all cases, the camera position and orientation were allowed to vary. The pixel measurement errors below were made a 1000 meters and measurements as close as 15 meters showed similar errors.

Mean/max Error (pixels):	0.91,3.19	0.93,3.19	0.66,1.67	0.24,1.66	0.88,3.05	0.22,0.57
Fixed aspect ratio (square pixels)		o	o	o	o	o
Fixed center of projection			o			
Fixed tangential distortion				o		o
Fixed radial distortion					o	o

Other than for the aspect ratio (which did not have a large affect), the ***fewer degrees of freedom available the lower the mean and maximum errors***. Not shown in the table is that ***fixing all four quantities produces mean 0.017 and max 0.053 pixel errors***. Fixing the ***radial distortion had little impact*** on its own and little impact on the mean error in combination with tangential distortion. The ***center of projection*** and ***tangential distortion*** had about equivalent impacts on maximum error, with tangential distortion more important for mean error.

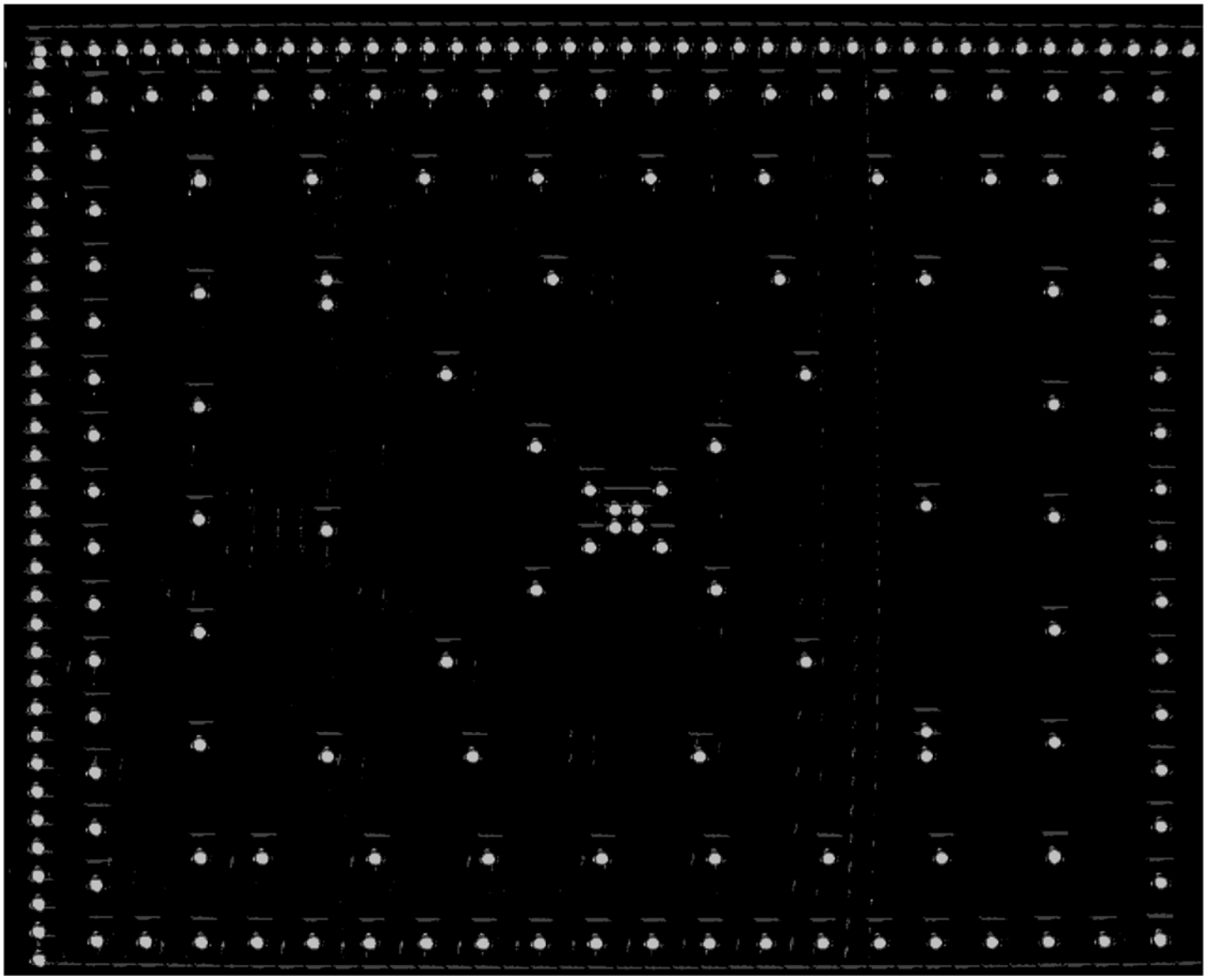
The largest single-camera mean error was 1.7 pixels in the case with only aspect ratio fixed.

Running a protocol of ***solving first for the center of projection, then for radial distortion, then for tangential distortion produces mean error 0.25 pixels and max error 1.7, with largest per-camera mean error of 0.45***. This is the protocol to be used for further testing.

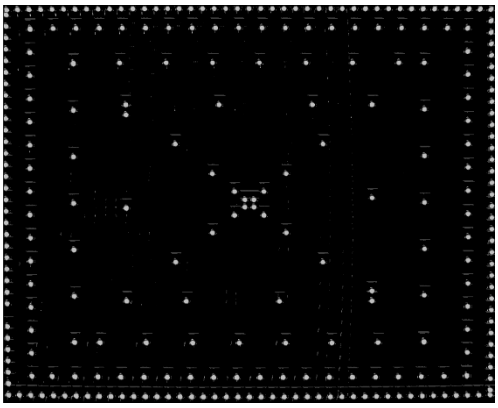
Application to real camera ball

In early July 2025, an initial Phase-1 procedure (which lacked the step that estimates the camera orientations) was applied to camera 2, which is a visible-light camera, using a single target located 45 feet from the gimbal center of rotation. An 8-pixel-diameter shielded light source was used as the target.

The camera angles in this unit were arranged such that some horizontally neighboring cameras had twice as much overlap as others, causing the target to be inset further that desired along some edges of the cameras and to be completely outside (and therefore not seen) on others.



The image above shows a maximum-intensity projection over the images for camera 11, which is the front-facing camera. It shows that the image is shifted in +X, +Y, away from the upper-left corner of the image. That corner of the image faces up and to the right, with the long along the right and the short edge at the top of the view. This offset is consistent with the visible subset of the sensor being at the upper-left corner of the total sensor area rather than being centered in the image. It is also consistent with a mis-estimation of the target location due to a preponderance of cameras at one rotation over cameras at the other rotation. Discriminating these cases will require the longer depth baseline and joint optimization of off-axis projection and rotation from phase 2. For phase 1, we will correct this using target and camera angle adjustment and leave the projection on-center.



Change 1: This necessitated the addition of the camera orientation estimation step between the target location estimation and camera stitching scan steps to ensure that the target would be visible along each edge of the cameras. When it is run with a single camera, this is essentially what is being done, and it does (barely) place all points inside the camera, as seen in the image to the right.

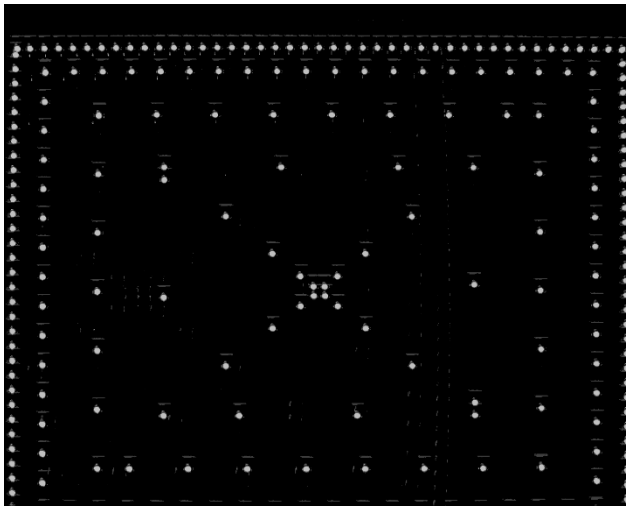
Change 2: The existence of some overlap between each pair of neighboring cameras was verified by scanning the array and observing the neighboring images in reduced-sized fields of view in cylindrical projection. The closeness to the edge of the outermost ring of target locations may require an increase in the distance from the edge to ensure that the points lie entirely within the image. However, because some edges overlap only slightly between cameras, this distance must not be so great as to move it away from the neighboring camera's region.

Change 3: The presence of other bright lights in the scene caused false alignment with non-existent targets when the main target left the scene. For later runs, other lights in the environment were turned off so that the missing target would remain missing because no pixels would exceed the brightness threshold.

Testing changes, round 1

Running a second pass target scan with an updated target estimate had locations scattered near the middle of the image with a cluster of them to the lower right and a few to the upper left of center (see image to the right). These were more tightly clustered than the original scans (seen to the left). This indicates that the position estimate is improving.

Ran a calibration with changes 1 and 3 described above. After calibrating the target and all cameras, ran a scan with just camera 11 using its adjusted orientation to ensure that the target landed inside its image region. It appears to have shifted properly in one direction but not in the other, as seen to the right.



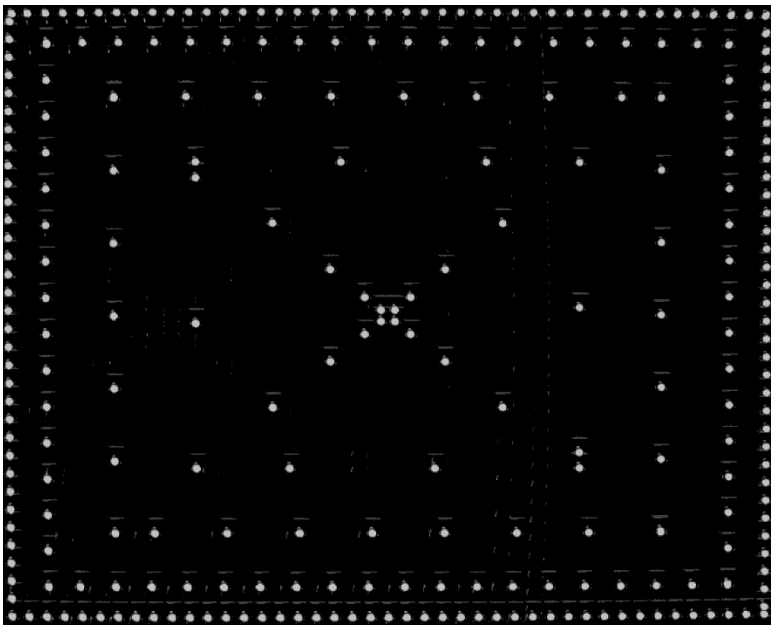
The short edge is vertical in the physical camera assembly, so the camera appears to have been reasonably well aimed in the vertical direction but be off by more than the original amount (possibly having moved in the wrong direction) on the long axis.

The initial camera alignment algorithm determined the rotation required to point the center of the camera at the expected target location, but it should have been finding the rotation to rotate the found target location to the expected actual target location (where we expect to have seen the target).

The algorithm was modified to work as expected.

Testing changes, round 2

Version 2.2.0 of the lateral estimation code produced a scan that kept all points within the image for camera 11 (see image to the right).

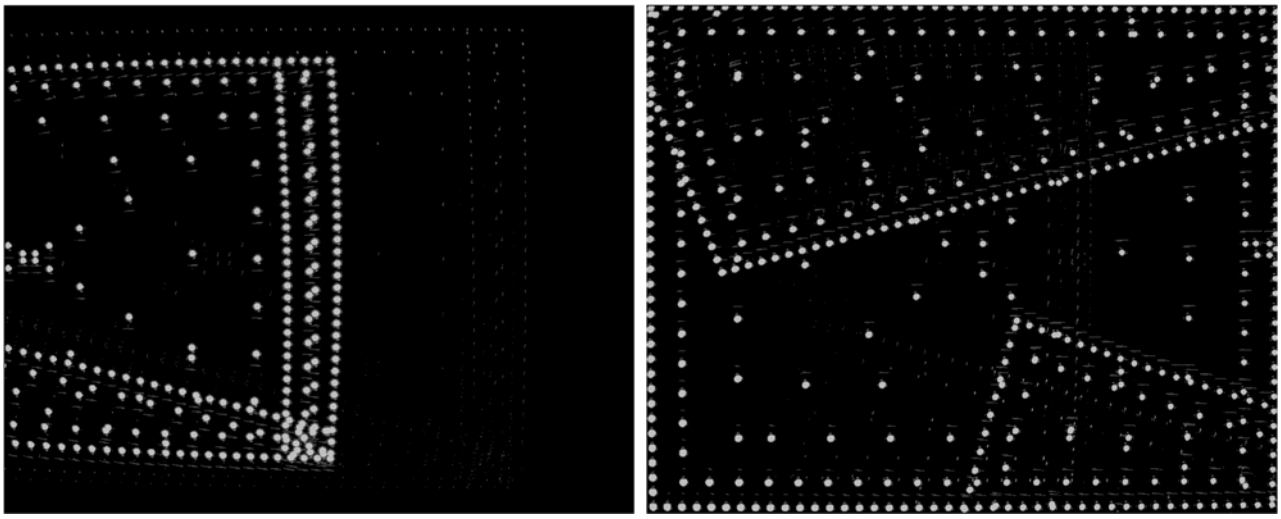


With this success, the entire set of 21 cameras was used to generate a poses.csv file that was run to collect data for calibration.

The resulting set of files all had some overlapping values from neighboring cameras, but there were issues with the data:

- ***Only some of them had their own internal dots: 4, 7, 10, 13, 16, 19.***
- ***The frames with their points shown had their points overlapping the edges, not completely within the frame: 4 (top), 7 (top, left), 10 (top), 13 (top), 16 (top), 19 (right, bottom).***

The images below show frames 3 and 4:



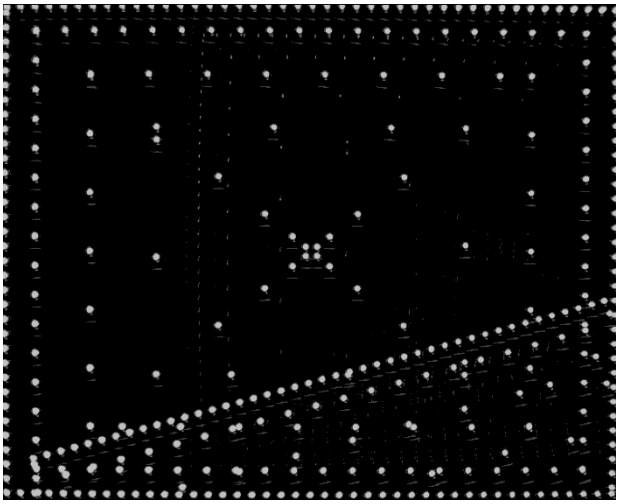
Looking at the poses.csv file shows that it contains a contiguous batch of entries for camera 1 that are only seen by camera 1; this indicates that the expected entries seem to be present in the file.

The fact that the scan for camera 11 alone produced its own entries, but that it did not produce them for that camera in the full scan, is surprising. There are 595 entries in poses.csv for camera 11, with 280 expected from its own entries (there are a string of camera-11-only values that seem to indicate that they should be present). There are 1785 images for camera 11 in the data directory, 3x as many as entries in the poses.csv file, which is consistent with all requested images being collected. ***Looking at the first-of-3 images for camera 11 shows that many of them have no spot visible in the image.*** This indicates that the images are being taken at a rotation that is not correct.

The home position for the gimbal was offset after the scan was completed, perhaps indicating that a mechanical offset happened during the run. The order of cameras in the file is 19, 16, 13, 10, 7, 4, 1, 20, … which is consistent with there being a problem with the scanning for camera 1, perhaps hitting a stop during the scan for that camera that shifted the angles for it and later cameras.

The *Collect_Calibration_Data* program verifies that all gimbal rotations are within range before running to avoid driving the motors into a limit. These limits are set to -90..90 in pitch and -175..175 in yaw in the gimbal configuration file being used. Moving the axes with the motors disengaged confirmed that the unit is free to rotate within these bounds.

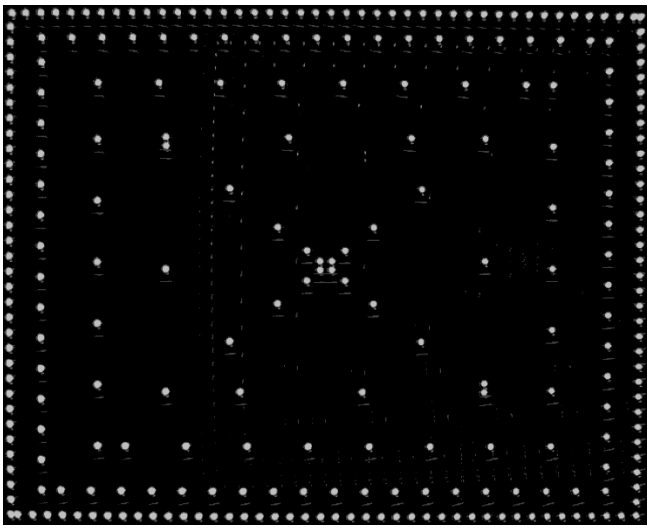
Generating and running a poses.csv for only camera 1 resulted in no slippage, with the zero position remaining the same before and after. Running cameras 4 and 1 (in case the problem was with the transition between them) also had no obvious issues and no change in the zero position. The image to the right shows the camera-1 scan from the run that did camera 4 followed by camera 1.



It appears that the offset problem may have been a glitch (perhaps caused by a thunderstorm).

Because the points are overlapping the edges, the pixel margins (left, right, top bottom) will be changed from their default value of 10 to 18 to attempt to provide clear margin without shifting the pixels so far that they will not be visible in neighboring cameras.

An initial test of camera 1 using these margins (which naturally produced slightly fewer data points, 274 vs. 280), did indeed pull the points all within the boundaries (see image to the right).

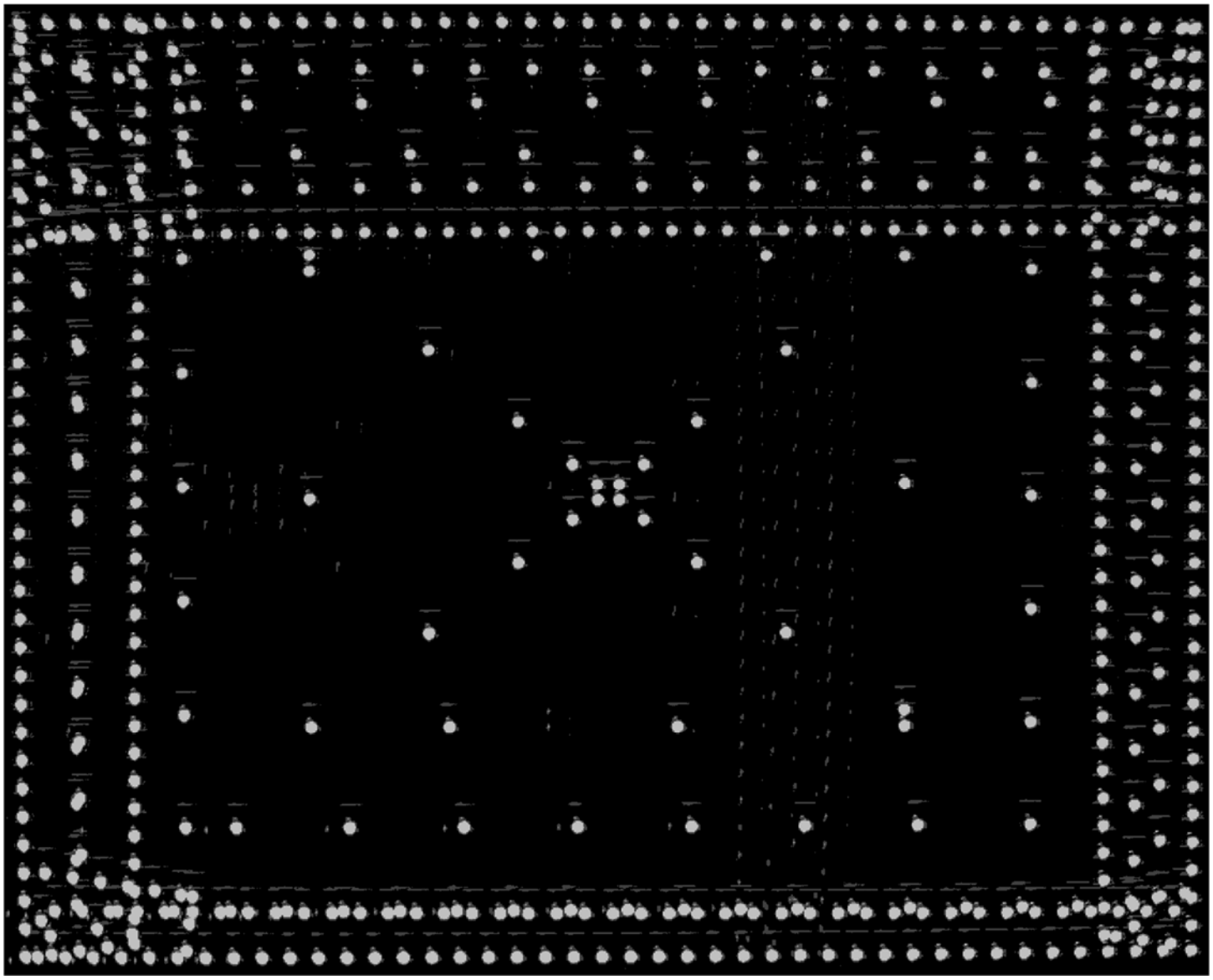


Testing changes, round 3

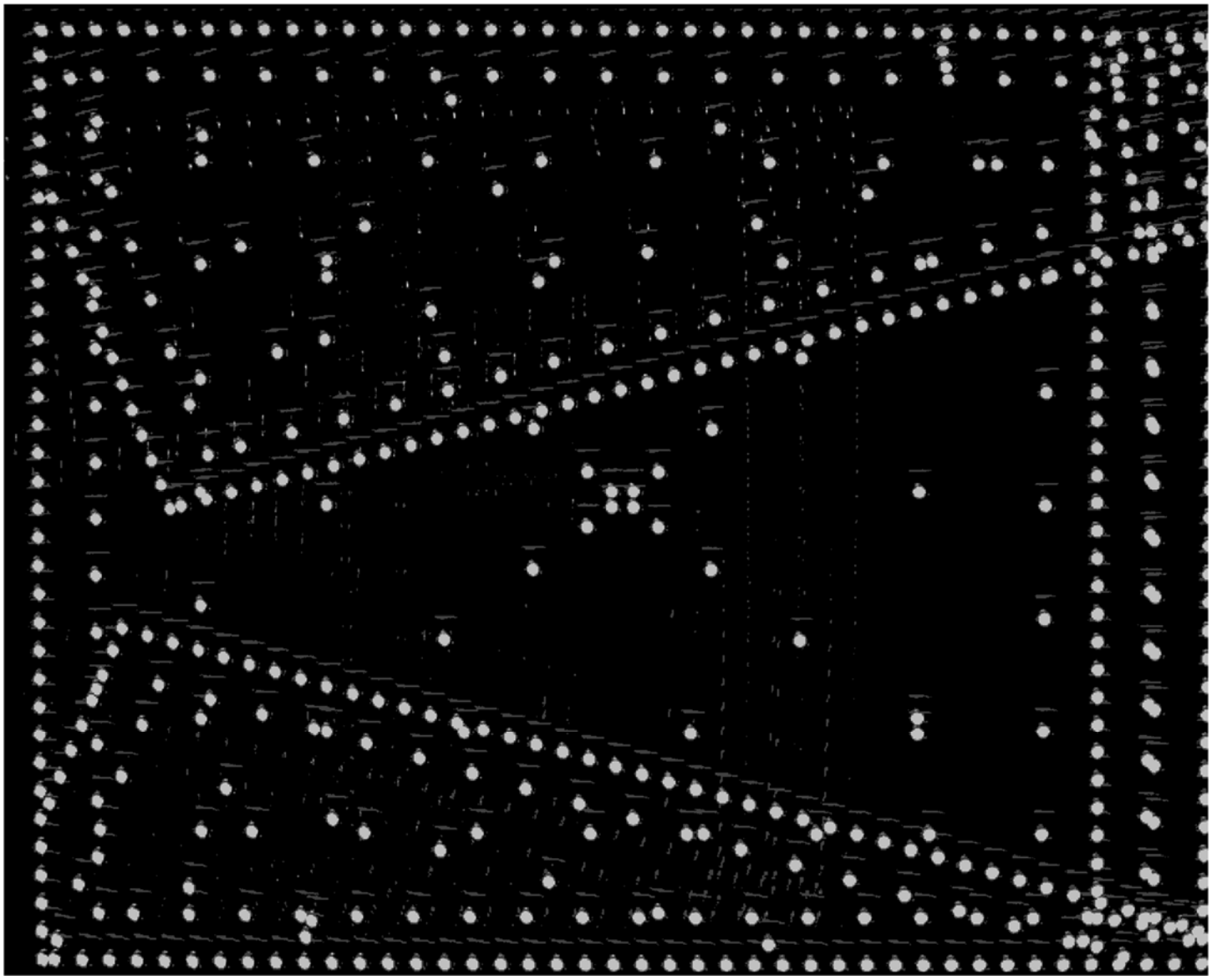
Another run using the margin size increased from 10 to 18 pixels was done, still using version 2.2.0.

The image below of camera 5 is emblematic of the behavior. This camera's points all lie within the boundaries. The neighbor on the top of the image overlapped a lot with this camera. The neighbor on the bottom overlapped much less, but still sufficiently to get a complete row of its points within the boundary. The left and right overlaps were more similar and both sufficient. ***There is definitely room for pulling further away from the boundary while still having sufficient overlap with a neighbor's high-resolution line of points.***

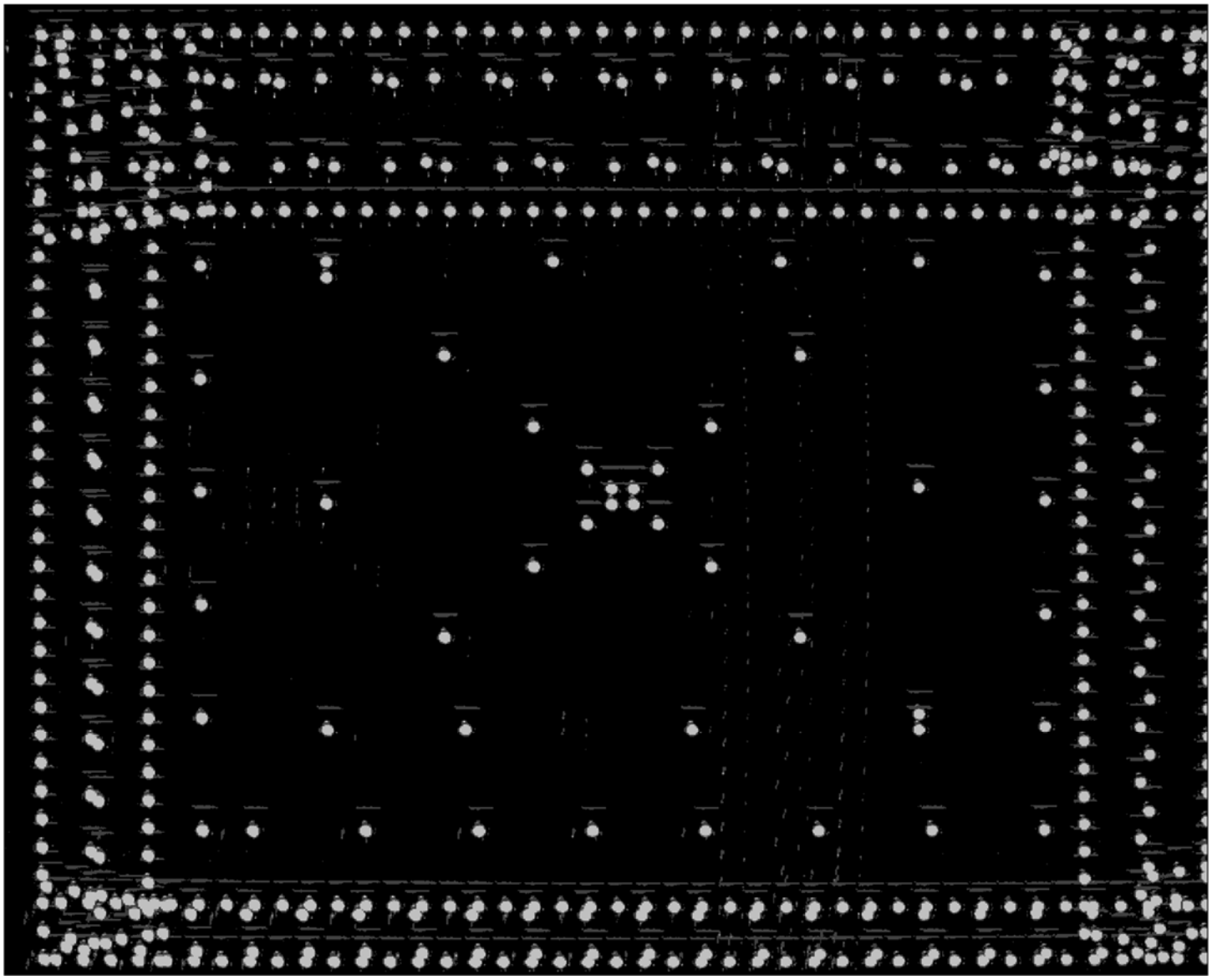
The large overlap on the top may cause poor registration at the boundaries because the high-resolution points for both images lie outside of the overlap region.



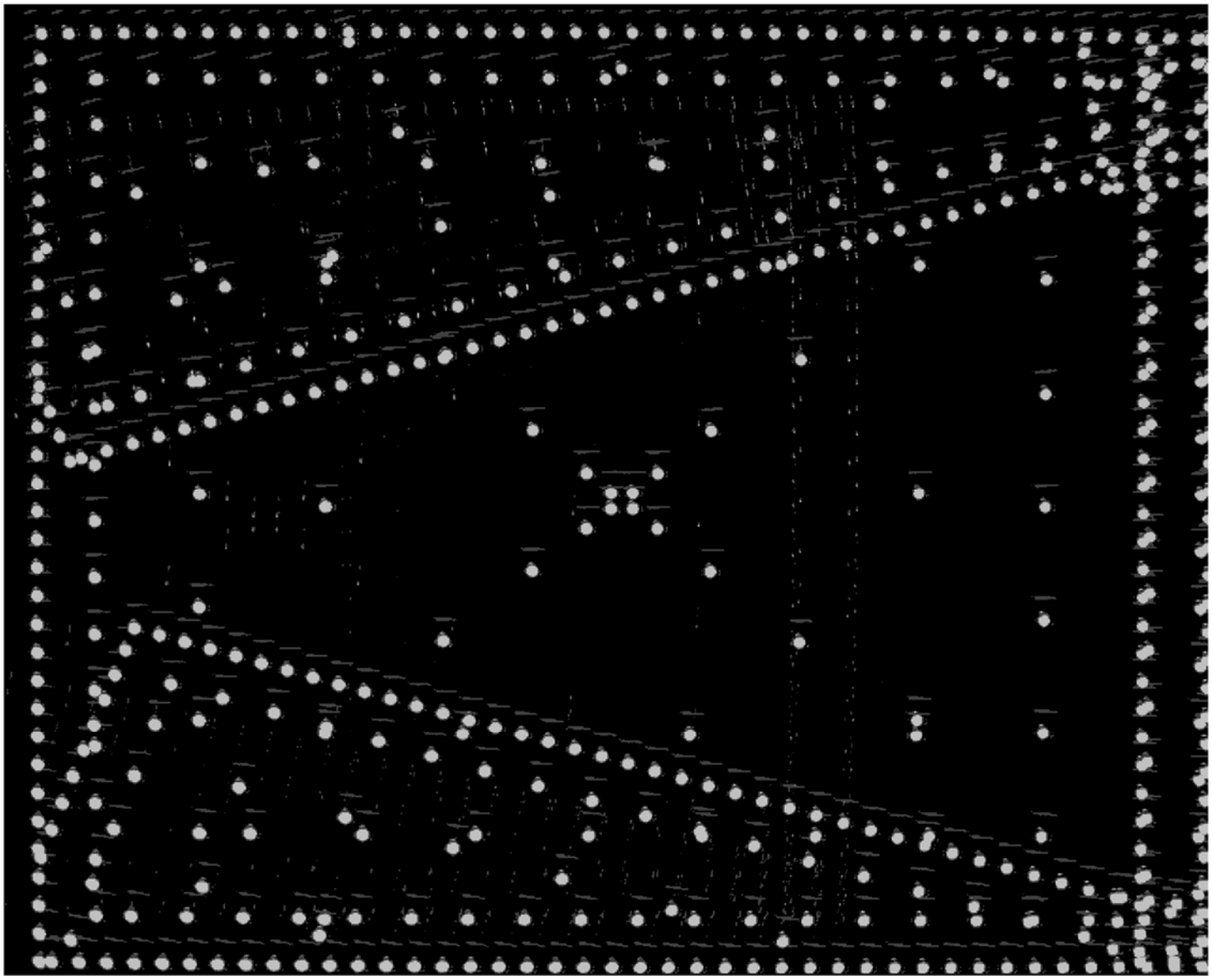
Camera 9 had some points just touching the left boundary in the upper half of the image, but this was not a shared edge with other images. Camera 10 had its whole rightmost row partially or mostly outside of the image, but its next row of points fell mostly on top of its neighbors along that edge (see below).



Camera 11 had similar behavior with a more staggered overlap with its neighbor:



Camera 16 had its entire first row and most of its second past the right edge:



Camera 19 had some of its points along the right just touching the edge.

In the replay, the following issues appeared:

- Camera 16 had a very distorted lower edge and middle.
- Camera 1 had a distorted and squashed lower edge.
- The stitching between cameras on a person standing just in front of the calibration target was poor – when the edge passes over, the feature in the new camera is closer to the just-passed edge, so part of the image appears to disappear into the crack between cameras.
    - Note: This depends on the depth assumed for rendering.
        - ***Setting 13.7 as the default depth in ComputePlanarCameraMeshInfo made them line up well!***

Testing changes, round 4

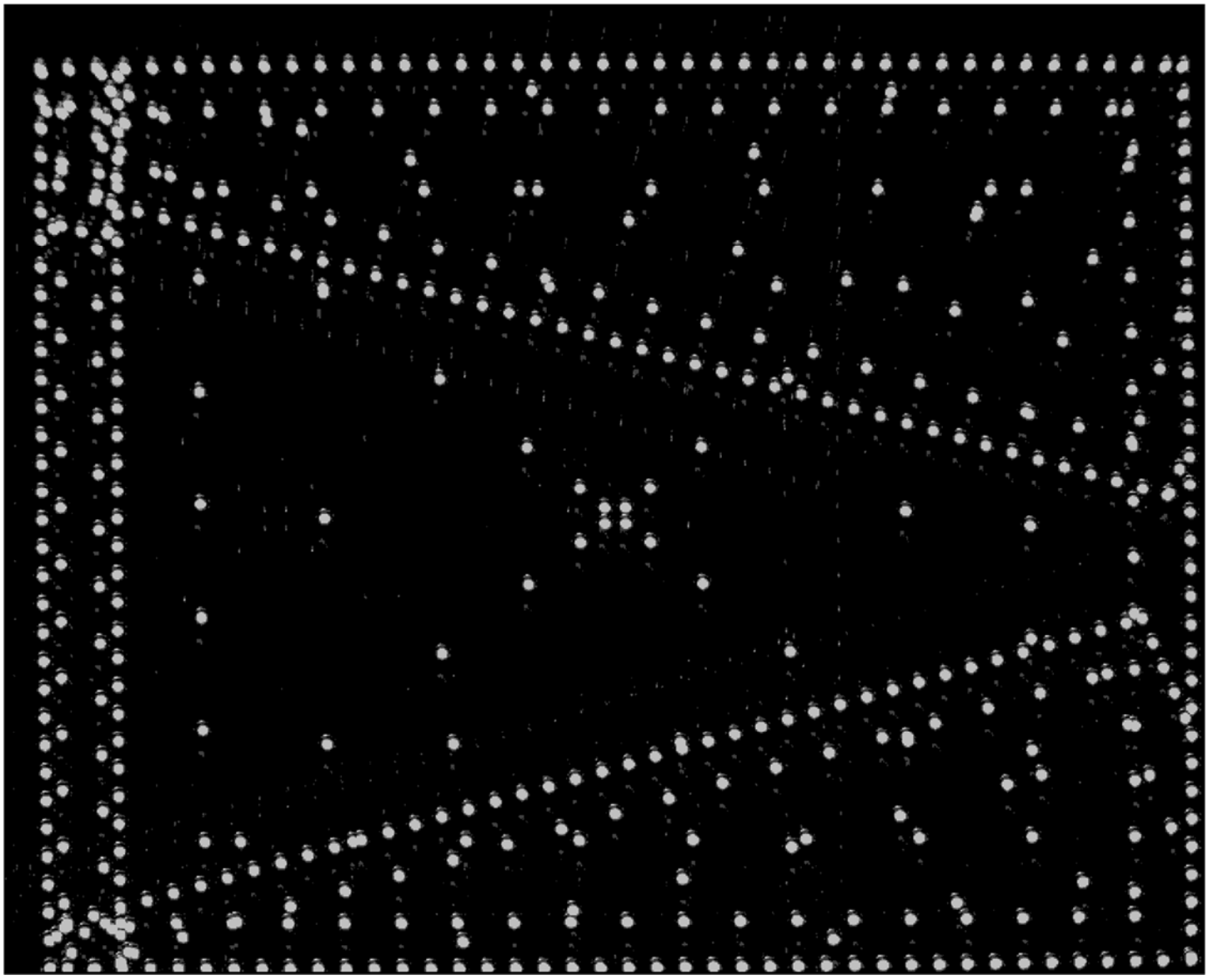
On 7/23/2025, multiple (at least 8) passes were made on the target localization with new data points being taken between each pass. This resulted in most cameras converging to a well-centered point, others converging to a second point down and to the right (4, 5, 6, 10, 11) and one slightly off to the upper left (21).

When rerunning the scan with margins again set to 18, we saw points on the edge for cameras 4, 5, 10, 11, and 21 and points past the edge for camera 6.

Testing changes, round 5

On 7/24/2025, the same lateral and camera orientation calibrations from round 4 were rerun with a margin of 30 around each edge (beyond that, it looks like neighboring points will cross over and head towards the edge on the cameras with little overlap). The goal is to bring all points within the edges even though there are biased errors in camera pose estimation.

On camera 6, some of the points touch the edge (see below) but in all others they are completely within the boundaries. It is expected that this will be sufficient to get good results on all cameras where they overlap.



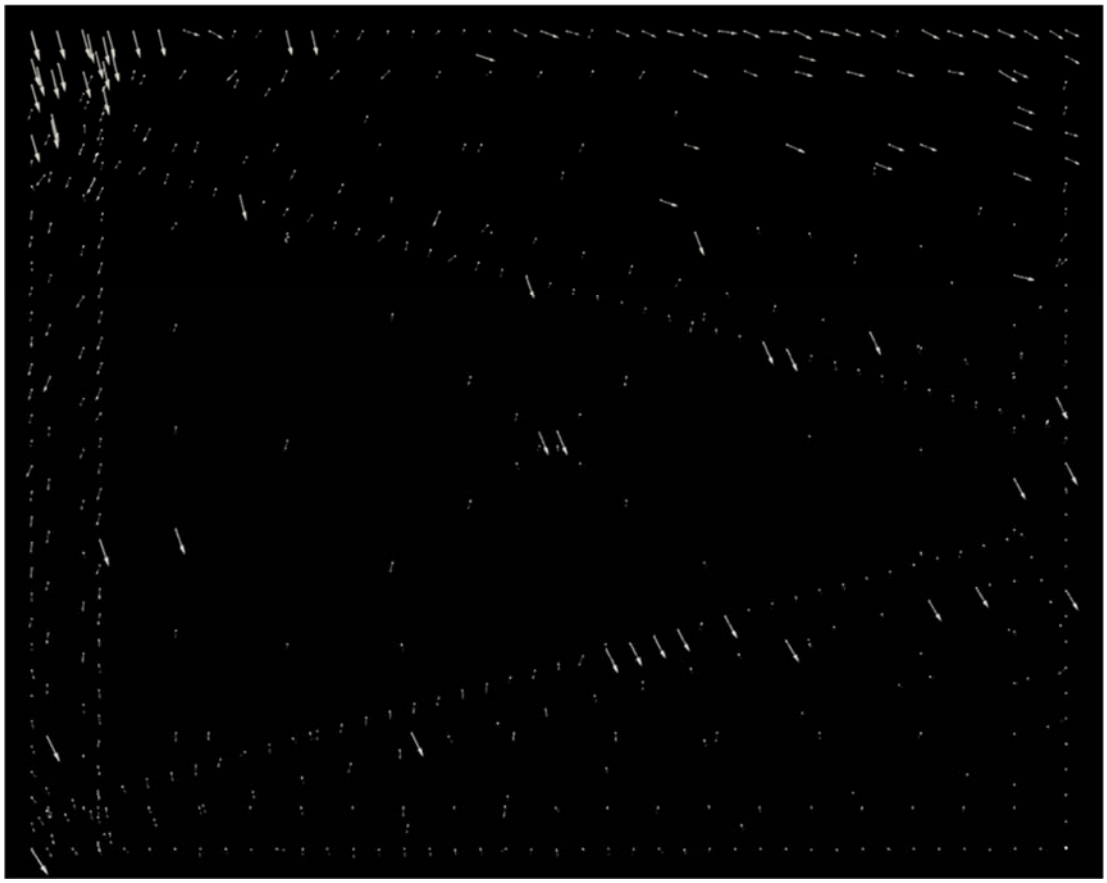
There were many issues with this stitch:

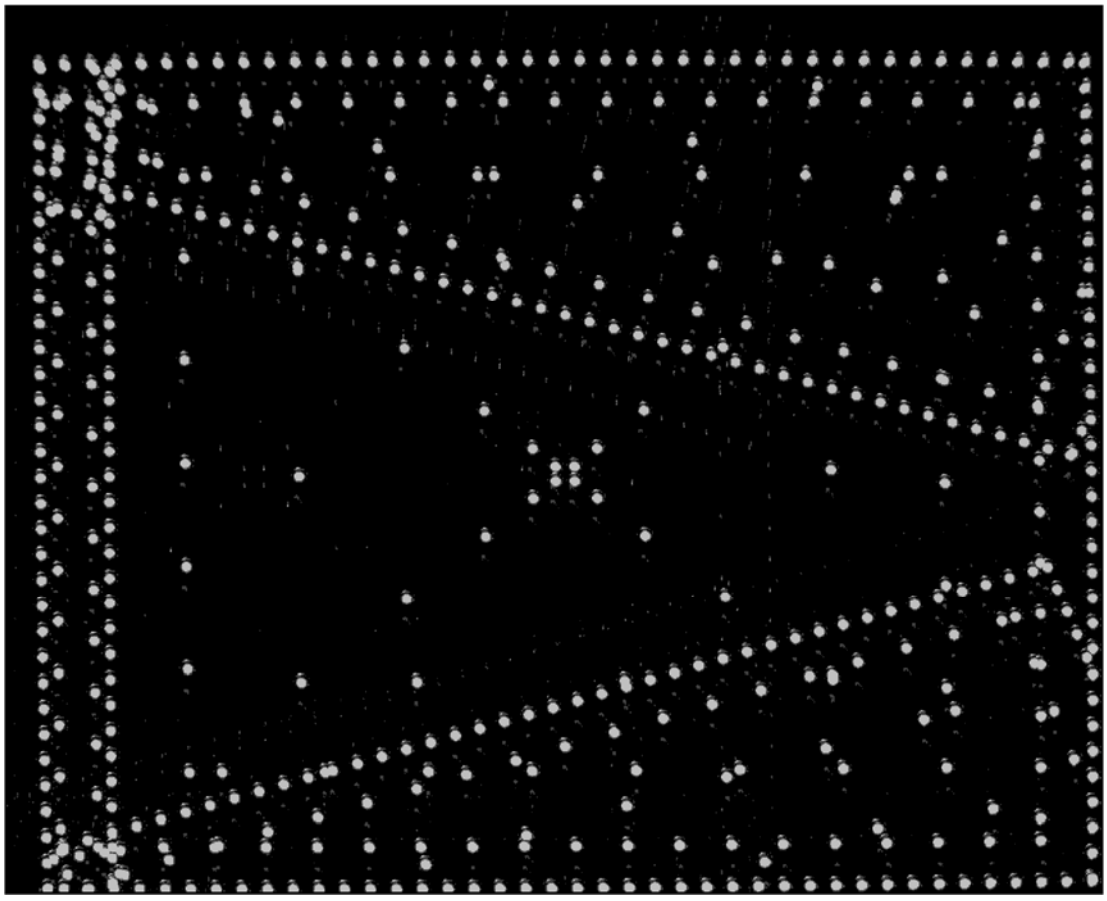
- There is strong distortion on the right side of the central camera (camera 11)
- There is misalignment on the left side of the central camera
- There are also offsets and perhaps distortion on the two cameras to the left of it on the center row

- There is distortion and gaps on the left two cameras on the bottom row (the next two look fine)
- There is a gap on the left side of the lower-right camera
- There is distortion along the bottom side of the camera in the center of the top row, and on the next three to its left

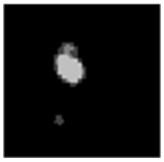
Looking at the mapping for camera 11 shows that there were several points that had long-distance offsets, some going outside of the image (expected locations in white at tails of vectors, discovered locations in pink at heads of arrows):

Looking at the results on the next-to-the-leftmost camera on the bottom row (camera 6) to see what is causing the large distortion in the center and bottom of the image (the bottom of the image is towards the right in the following two pictures):

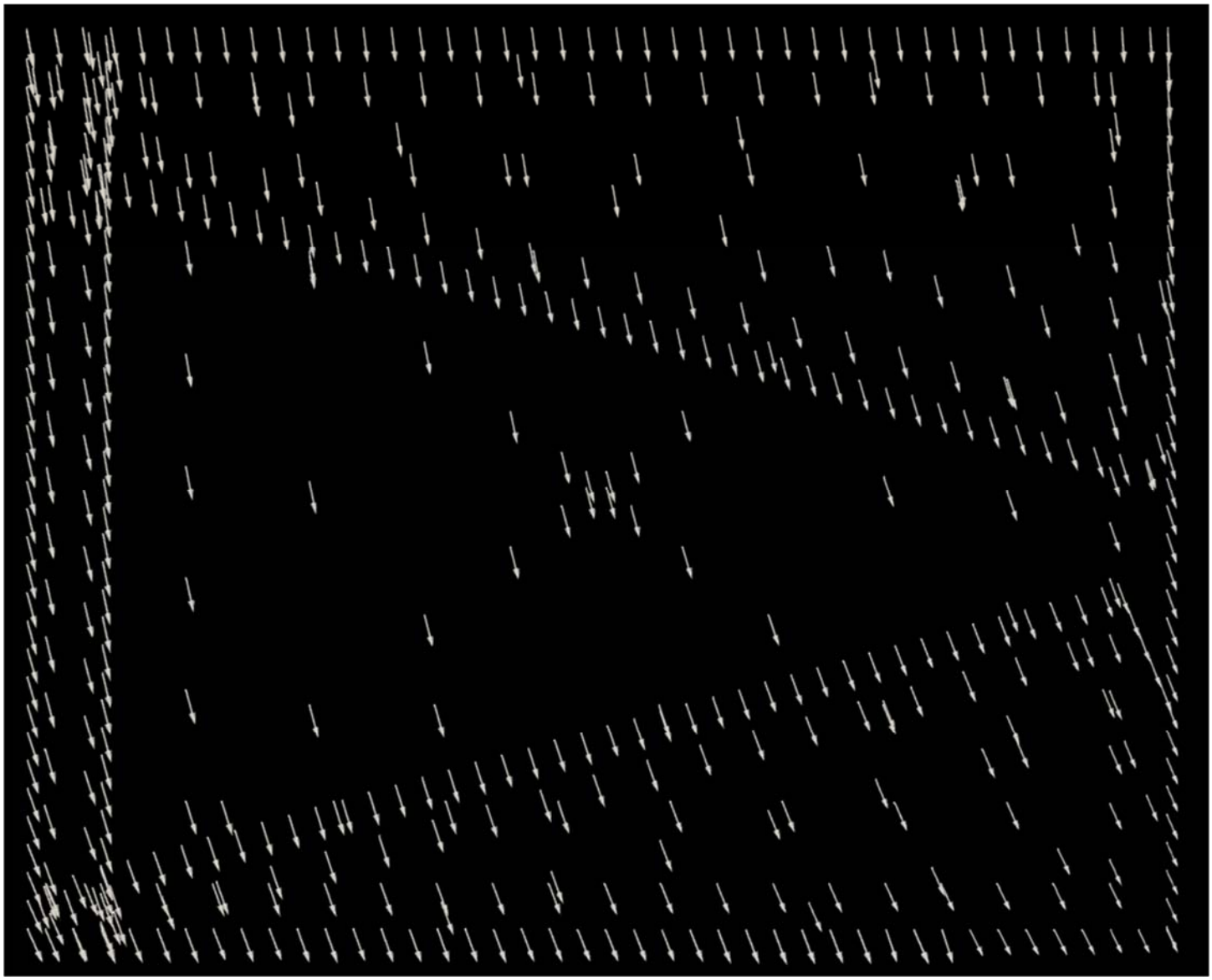




The distortions seem to be caused by the longer arrows causing too much local distortion.



The points in the upper left corner are not particularly round and are about 14 pixels across with a side lobe (see image to the left). A 10-pixel radius (20 pixel diameter) tracker should still cover it and be able to localize it well but might go in the wrong direction when launched at the edge of the spot. **Note:** The problem turns out to probably have been the short arrows rather than the long arrows; there should have been shift to the lower right across the entire image:



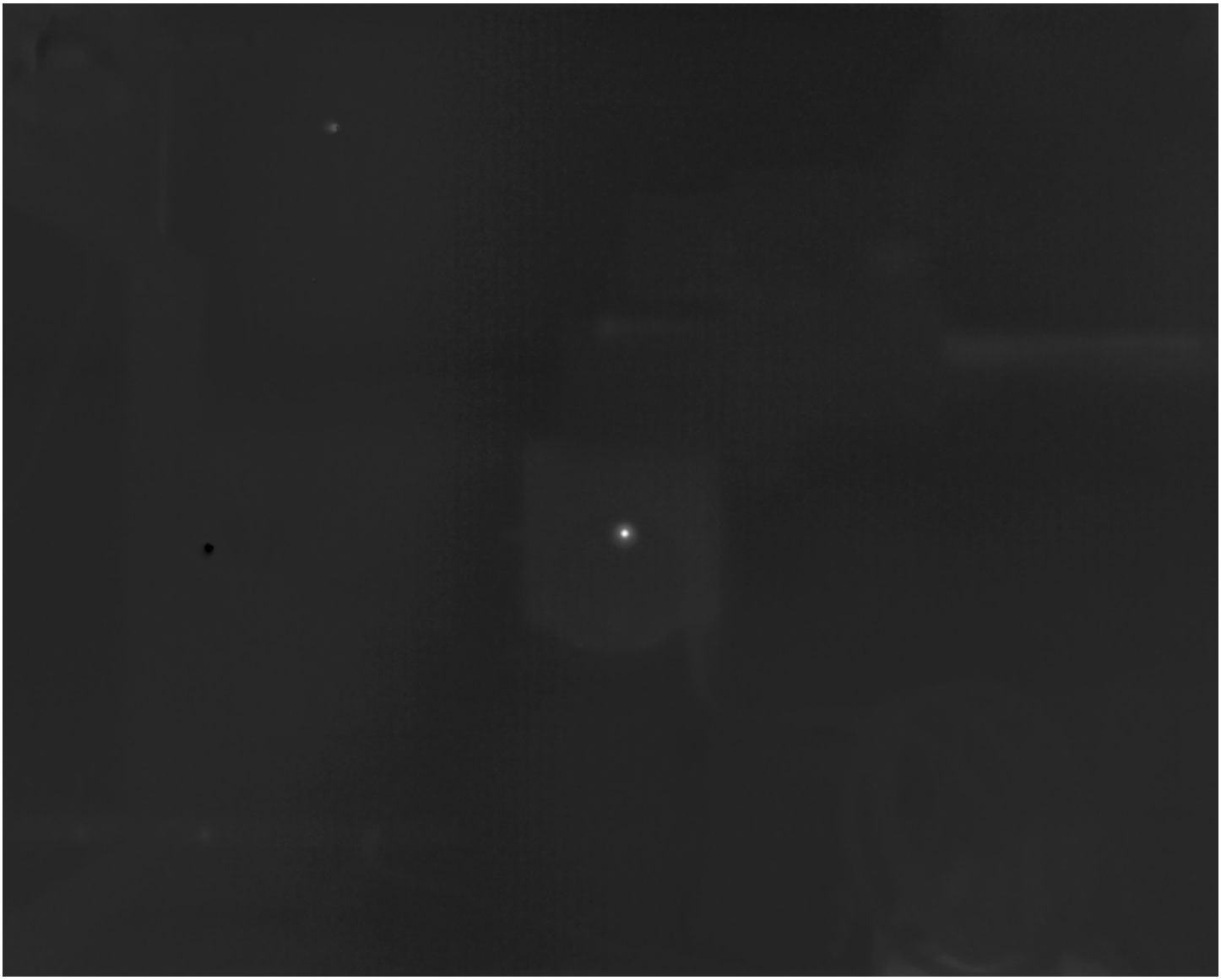
Adding a cone-based tracker to center the particles before using the symmetric tracker to precisely locate them avoided having the symmetric trackers move off a target when started near its edge.

Version 1.0.0 of *Camera_Calibration_Estimate_Distortion_Extrinsics* was able to produce a clean stitch for objects at the correct distance over the entire set of cameras.

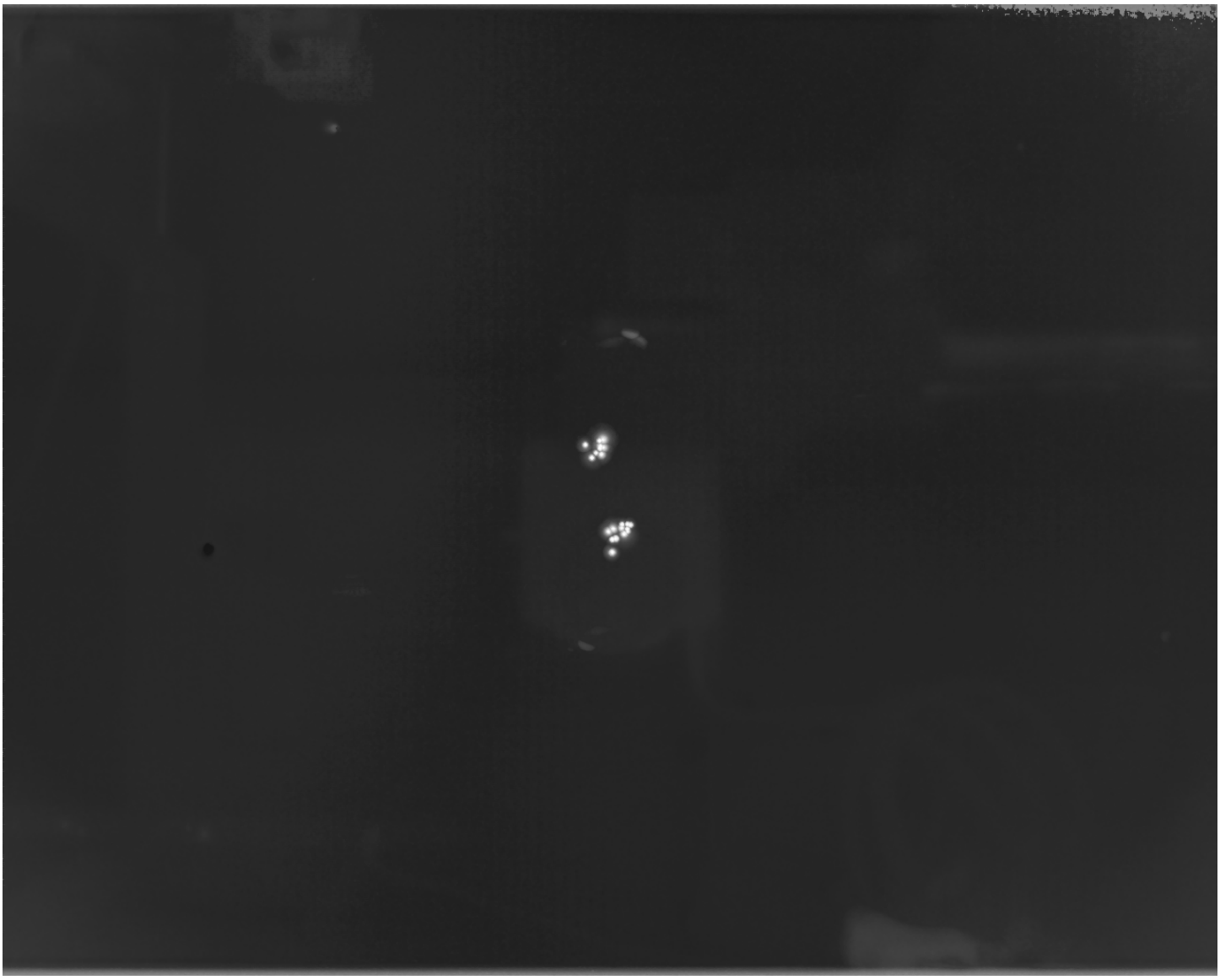
Application to LWIR camera ball

Testing in August 2025 on the long-wavelength InfraRed (LWIR) camera ball revealed the need to move beyond a simple threshold-based approach to locating the target.

An initial attempt that stretched the contrast of all pixels in the image across the entire 16-bit range (--autoScale) was thwarted by the presence of stuck pixels, both bright and dark. The removal of single pixels that are brighter or darker than all four neighboring pixels did not work in all cases, some apparently have two such neighboring pixels. The use of a median filter (--median) on the image prior to –autoScale did appear to correctly stretch all images, resulting in a pronounced bright target location compared to a dark background:

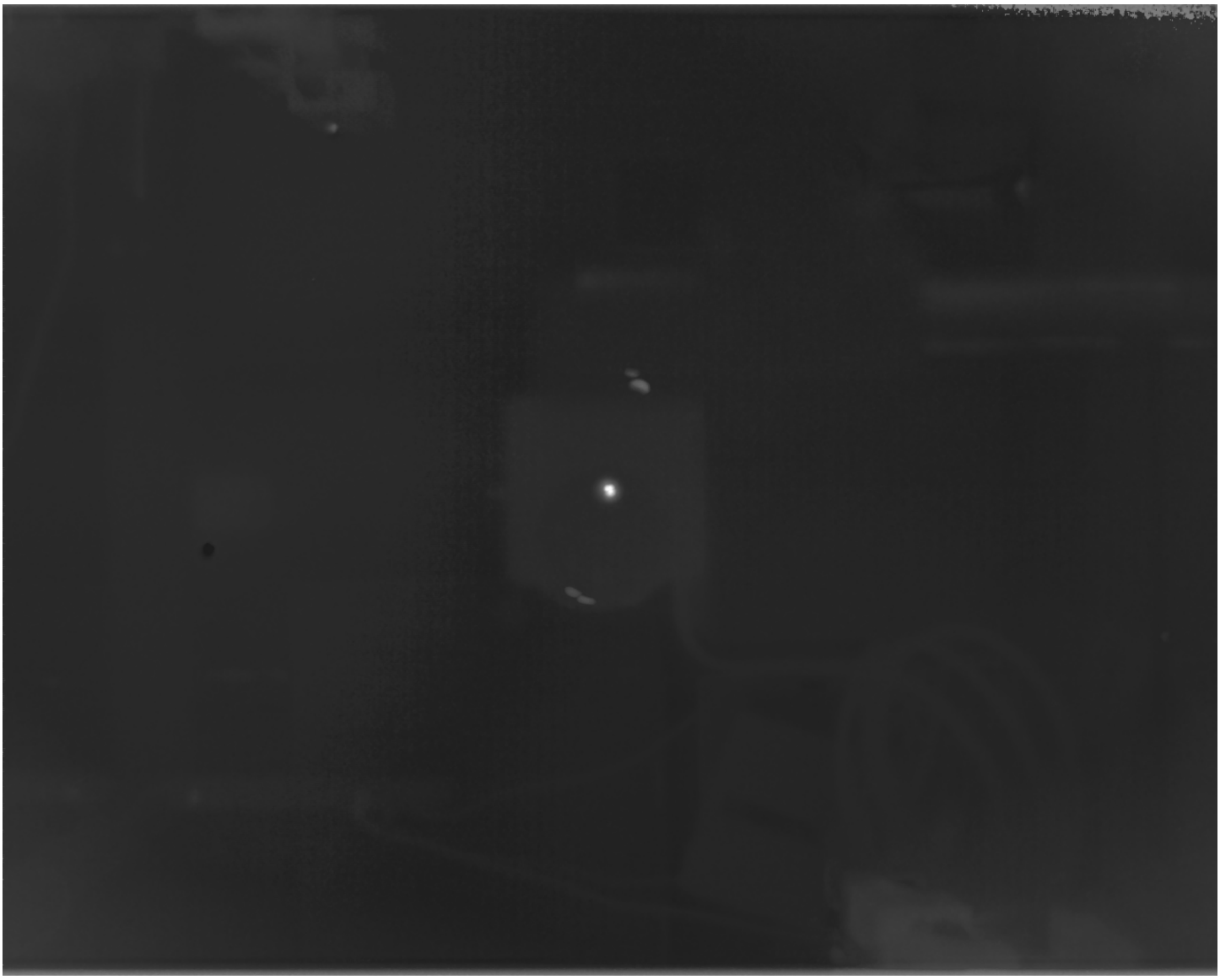


The maximum image across all 21 cameras showed a clear pair of clusters for the target location, consistent with an offset from the actual center location of the targets that is symmetric around the origin because half of the cameras are rotated 180 degrees from the others:



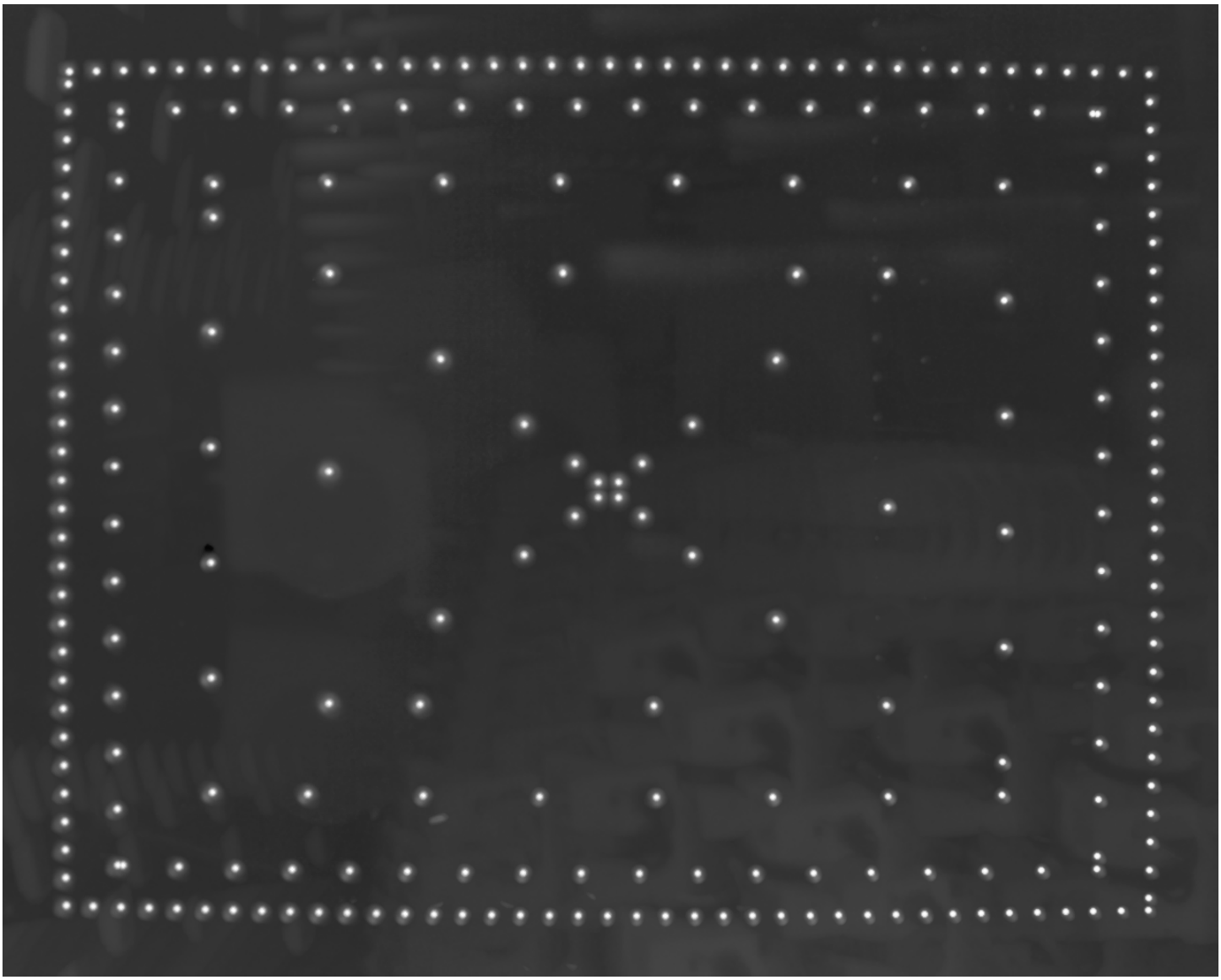
A complete calibration run was attempted using a target about 2 meters from the camera ball. The maximum overlaid images from the initial scan looked as follows:

Re-running the calibration scan with the updated target and camera configuration files should make a tight single cluster of points; it produced the following:



This looks as expected.

After running the scan across the points inside the first camera (by truncating the poses.csv file), the resulting overlaid auto-scaled images look as follows:

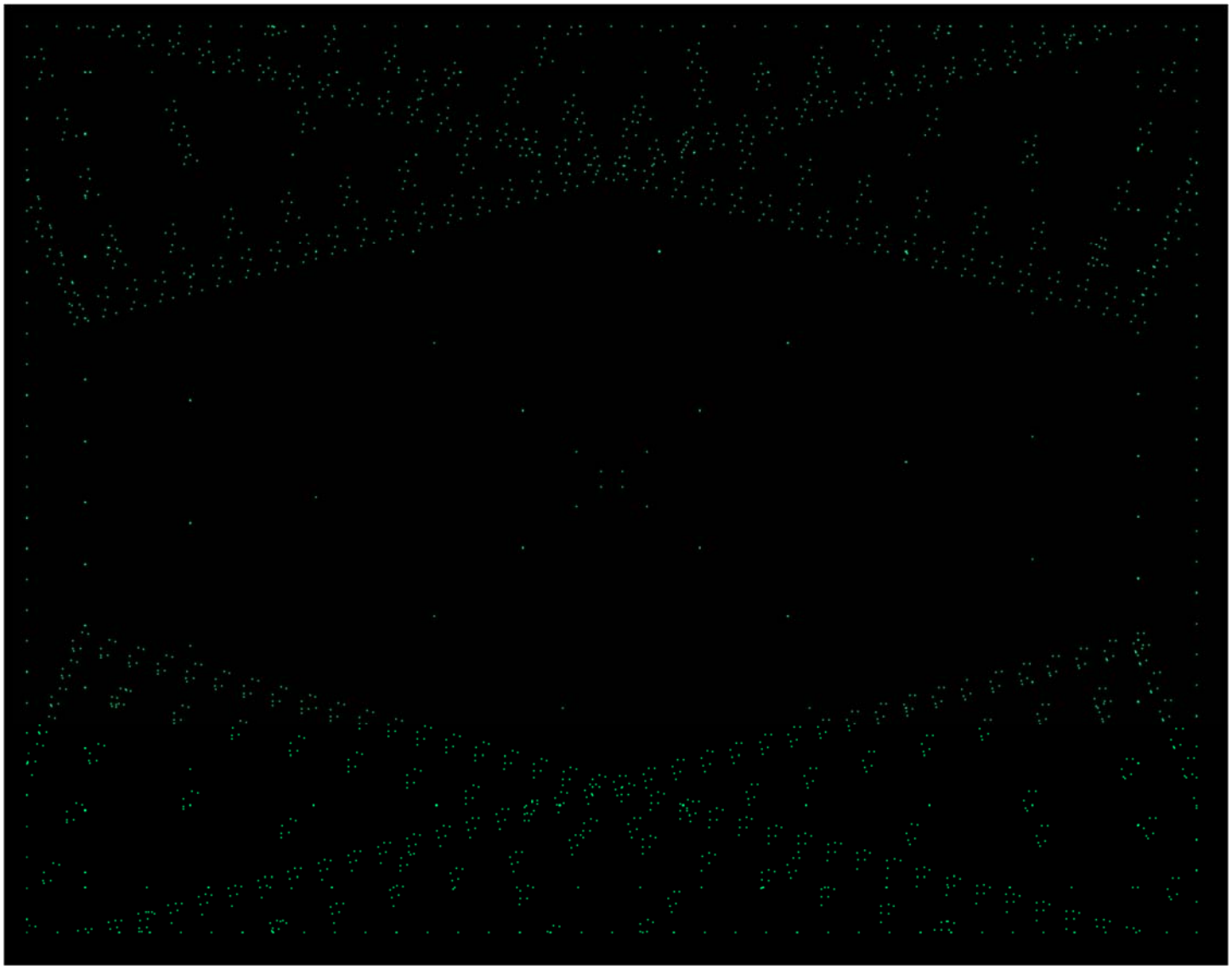


This looks like we expect: all points far from the edges, all points at or very close to maximum brightness, the points are reasonable in size, and the background is all much dimmer than the targets.

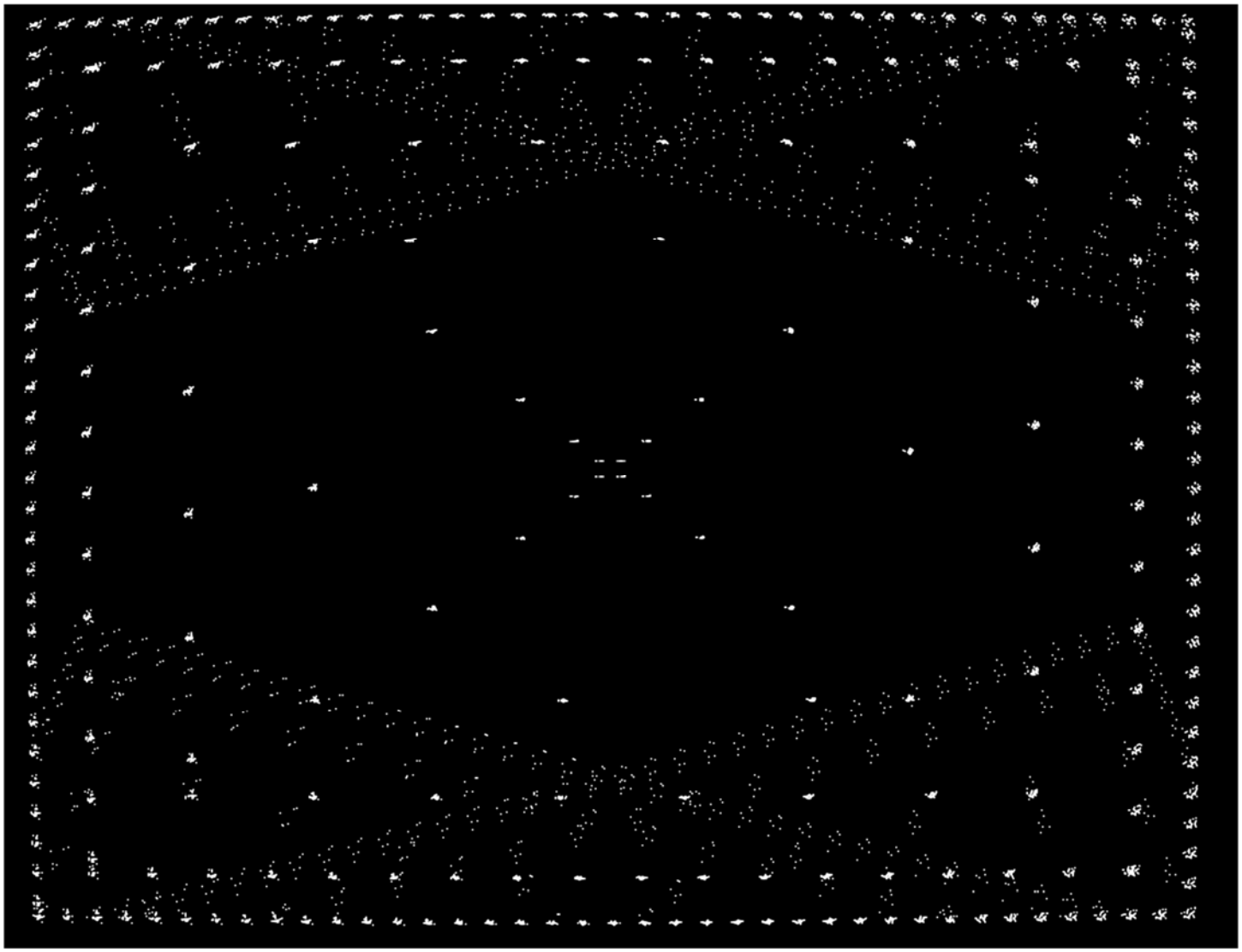
We then started a complete scan without autorange so that we could adjust the algorithm for autoranging if needed without having to retake the scans.

The resulting stitch looked much better than the original when looking at objects in the room (which are much closer than infinity) when the Render Module was run with –staticDepth 1.97167 to match the stitching depth. The target itself was very close to the same location as it transitioned between neighboring cameras from the front-facing camera. We looked at all four edges to verify that the alignment was good.

Mapping: A mapping from expected point location to measured point location is used to determine the distortion for each camera. The expected point locations are plotted below in green, overlaid for all cameras:



The actual offsets are pushed a bit outward in the middle of the edges and are plotted below in white:

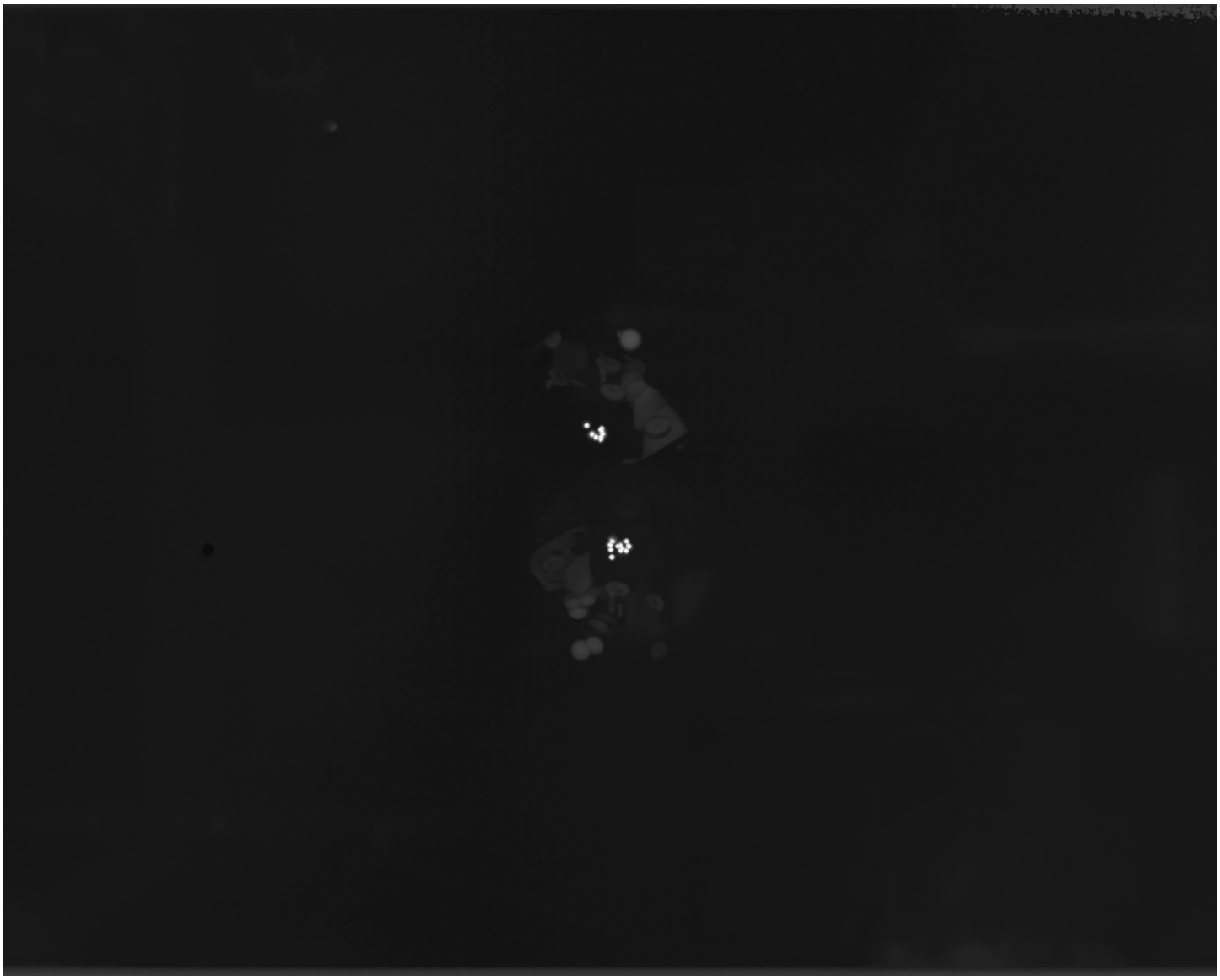


Both are plotted together below, showing the different magnitudes of offset at different points on different cameras:

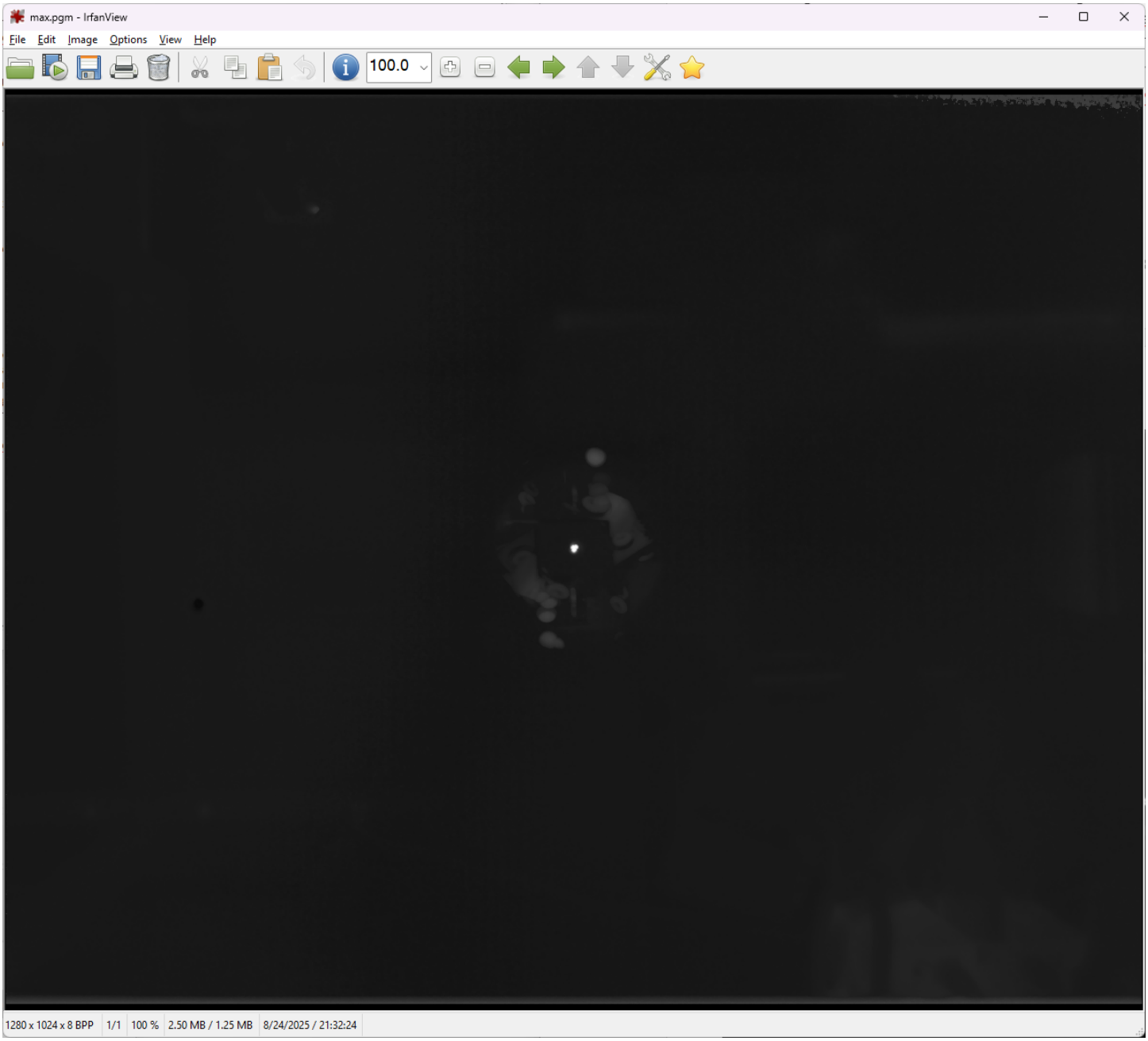
Repeating calibration using the collimator

The target was moved to be viewed through a two-mirror collimator that images the target at infinity. The calibration steps taken above were repeated with this collimator, specifying its depth to be 20000 meters.

The initial run had two clusters of points, as expected:

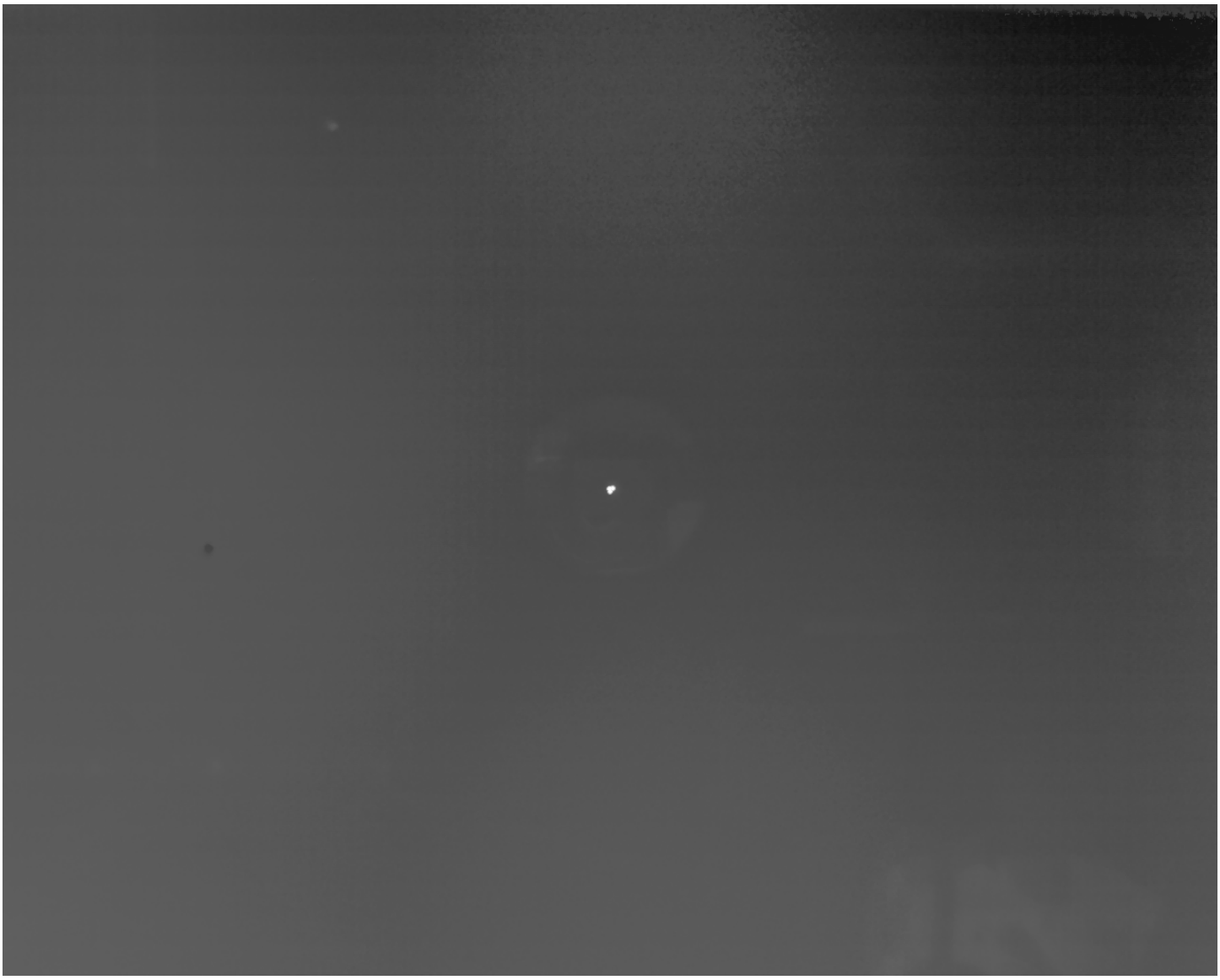


The recursive run put all points overlapping in the center of the image, as hoped:

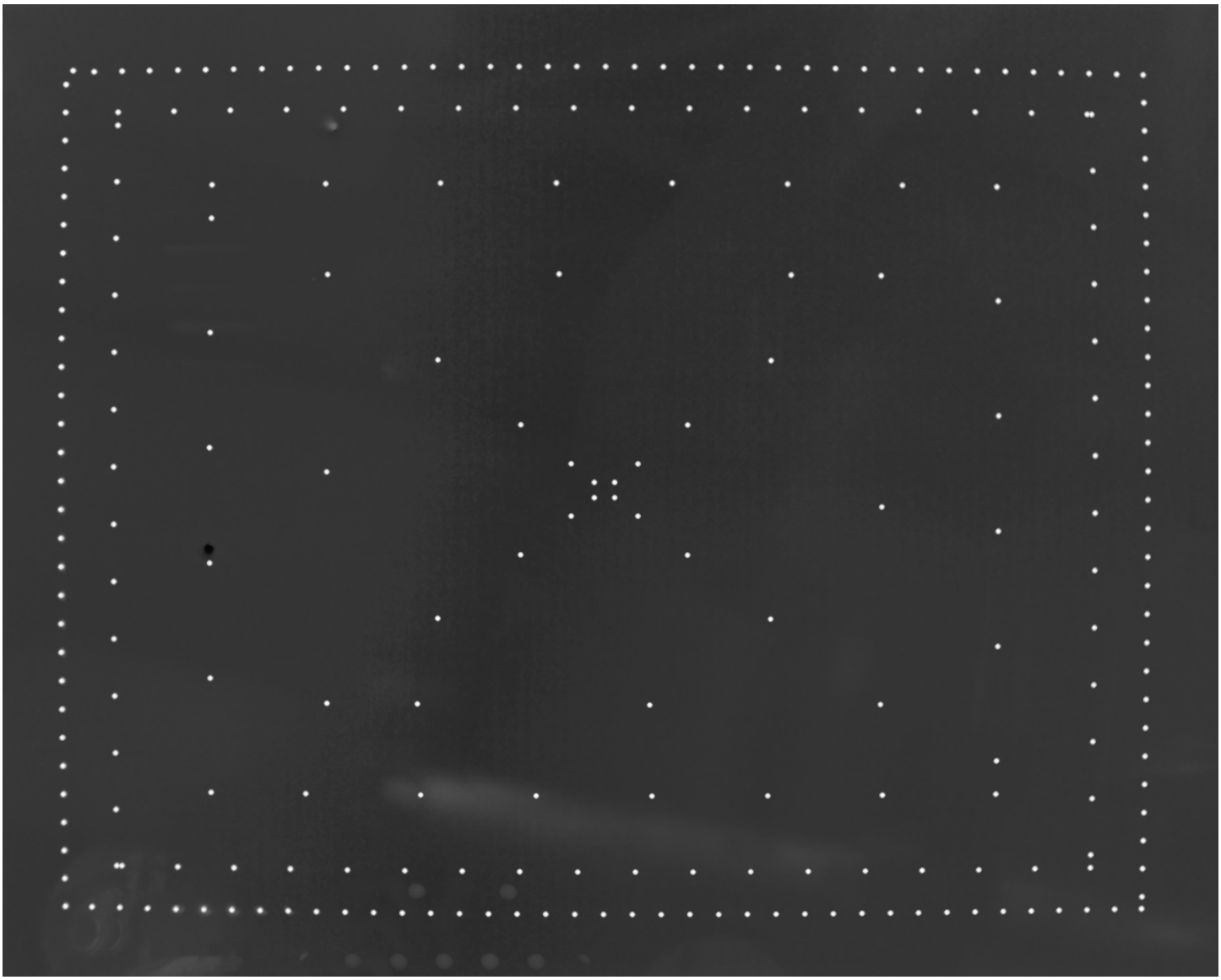


Shut down for the evening and re-ran the next day. After the second pass. One camera continues to flash from time to time even after reaching temperature. Adjusted its setpoint up slightly to try and completely avoid it.

The recursive alignment worked very well, although the autorange was not as dark for all cameras (the same one that was flashing was brighter). This camera (18 from zero index and 19 from 1 index) has a patch of dark pixels that is causing the autorange not to work as well. Re-NUCing that camera made it look significantly better.



Running the scan for the first camera to check whether the target is always visible within the aperture of the collimator. This will not guarantee that all of the cameras will have all of their locations within the aperture because there is a different offset for each, but it will tell us if this one is out of range. The overlays look as follows (with 70-pixel borders):

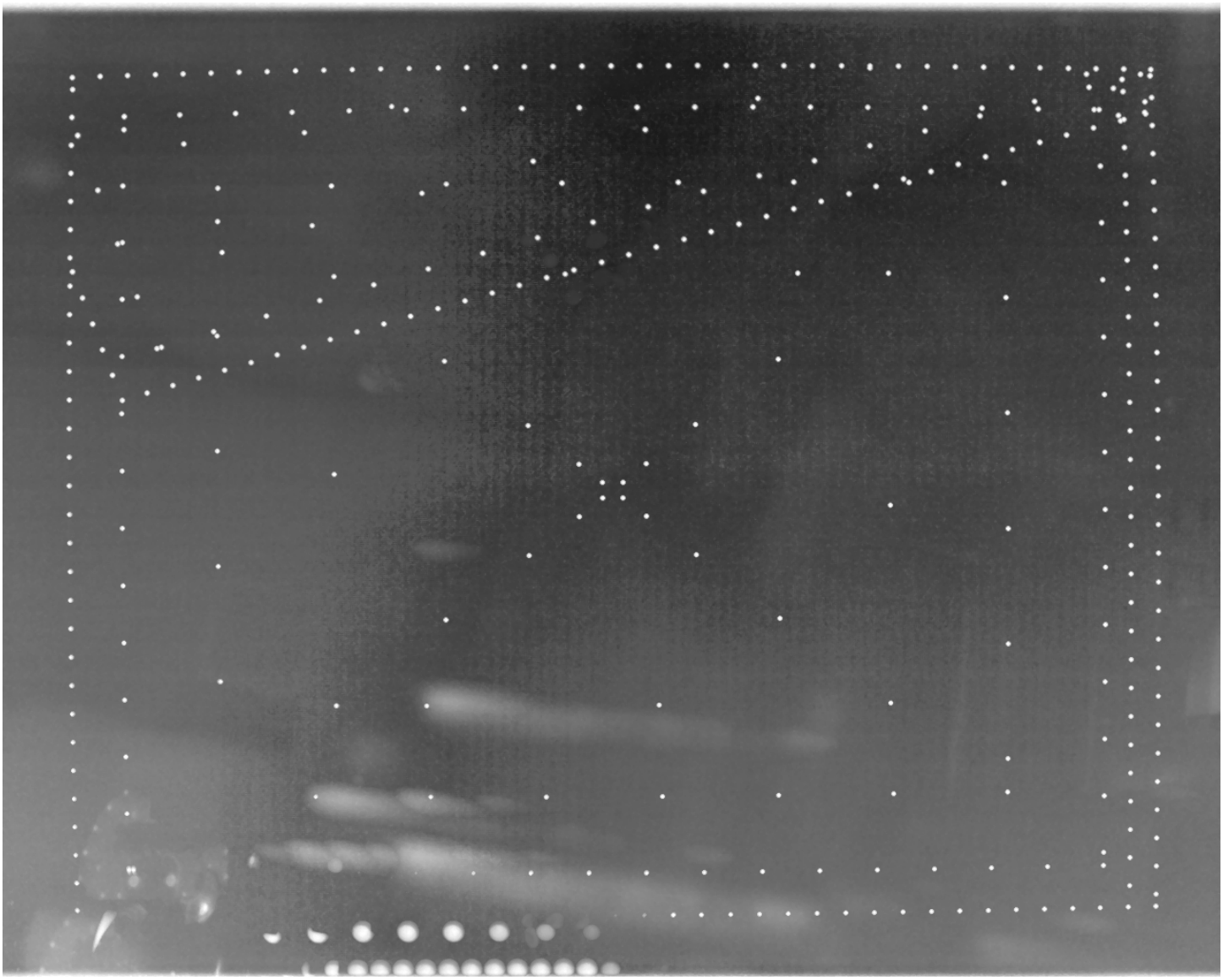


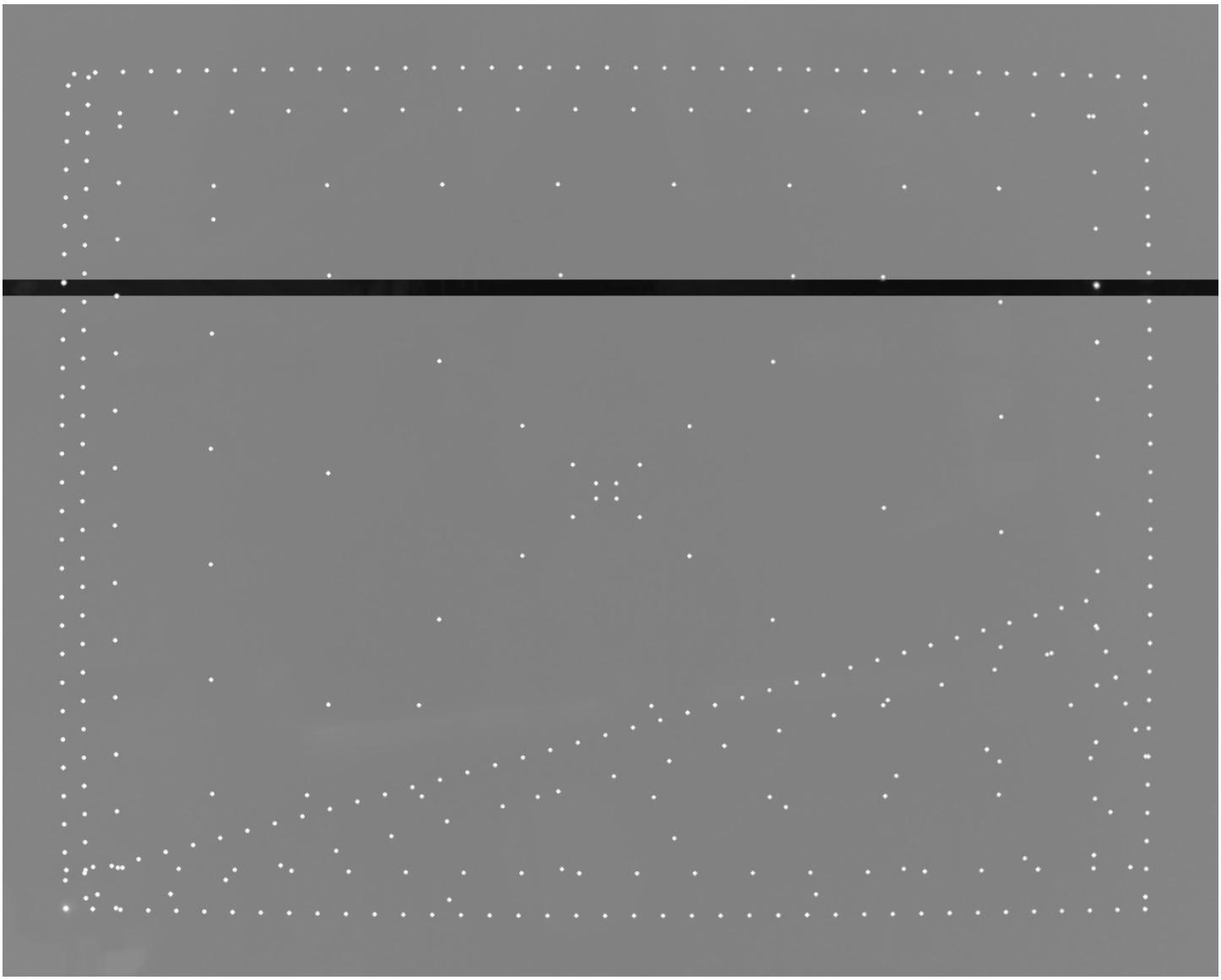
This looks correct, with all points inside the frame.

The entire data set was taken to provide a calibration at infinity.

There were some problems with the collection:

- The following cameras had some of the dots move past the edge of the collimator, resulting in large lens-flare or out-of-focus spots at far offsets from their proper location: 1, 4, 7, 10, 13 (see first image below). The problem is the large dots in incorrect locations. We have some additional margin, so we can pull the calibration values further from the edges to compensate for this, perhaps 100 points from the edges rather than 70.
- Some of the images had strong banding: 3, 11, 15, 17, 18, 21 (see second image below). This may not cause calibration failure.
- Some of the images had regions that were so bright that they would wash out the calibration targets: 7, 11, 13, 15, 16, 17, 18 (see third image below). These will prevent calibration from working on those cameras.



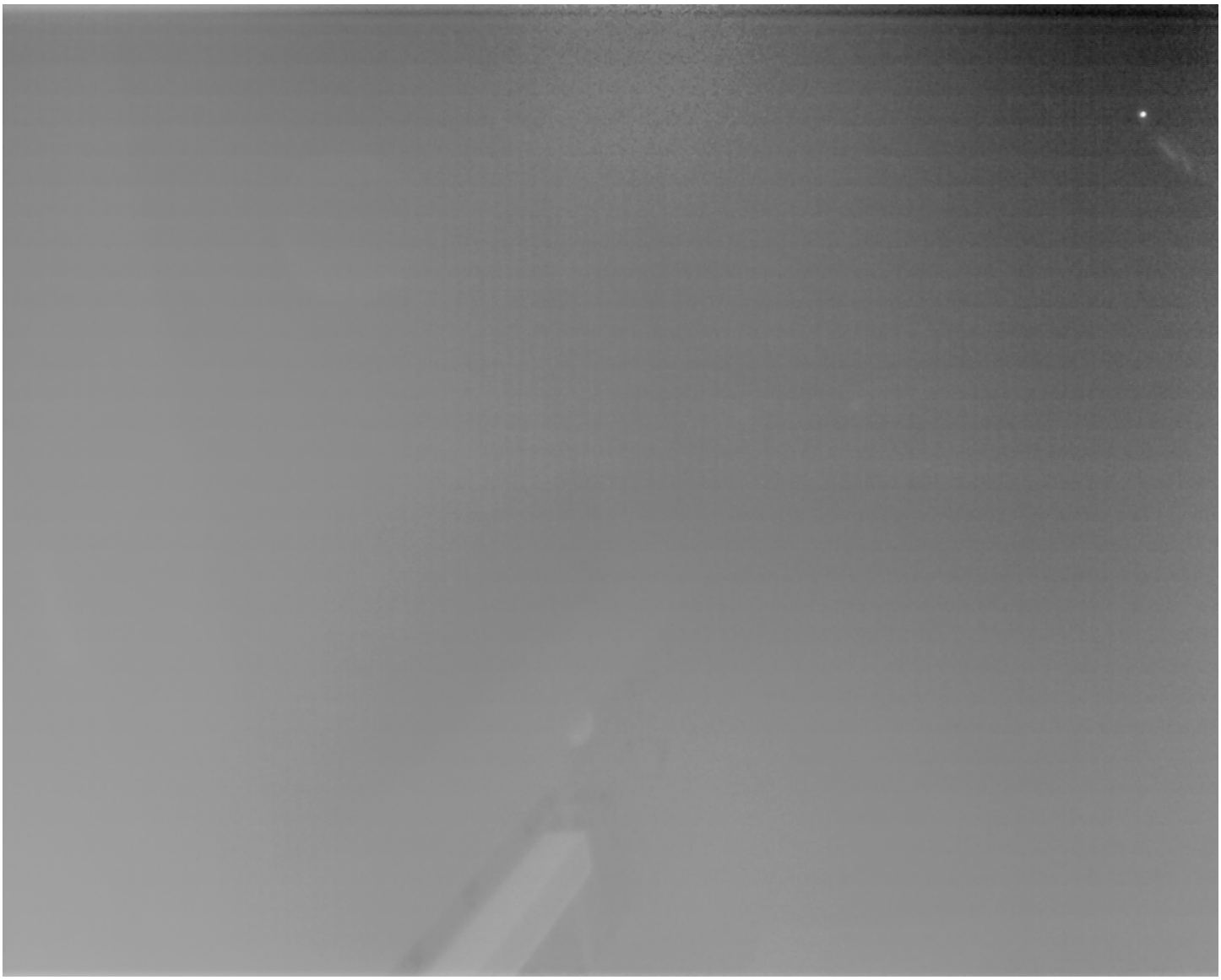


As discussed above, we can compensate for the points outside the image by pulling in 100 pixels from the edges rather than 70 pixels, but the other artifacts will require more careful monitoring of the collection and/or improved autoscaling. It is recommended to do the scans without autoscaling so that we can rerun different autoscaling algorithms without having to redo the scans.

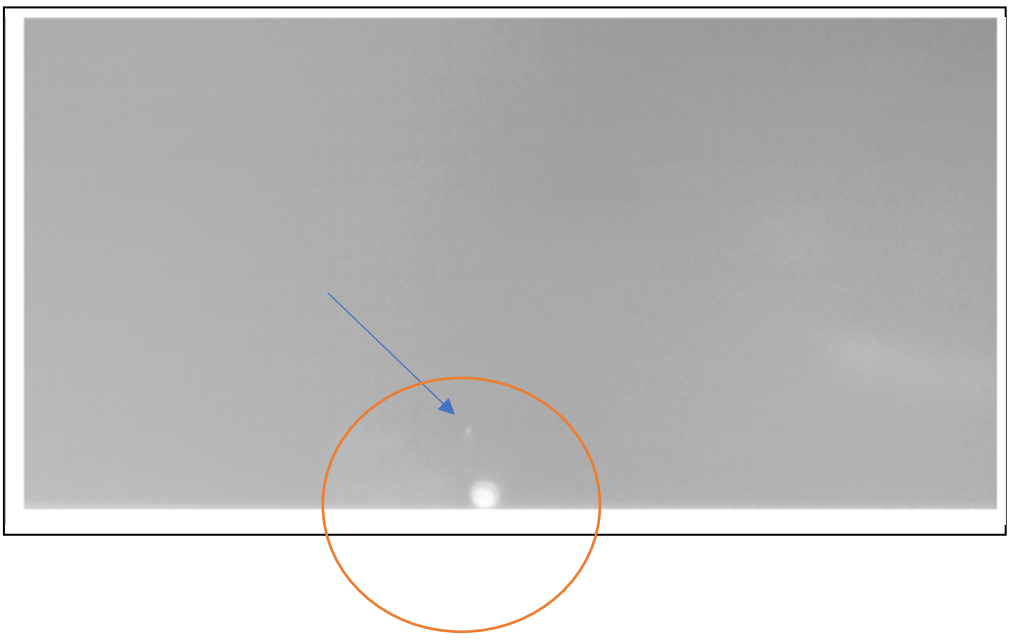
On ***8/27/2025*** we re-ran a scan, keeping the raw (non-auto-ranged) scans and then auto-ranging them in a second pass. This is intended to enable us to adjust the auto-range function and re-use existing scans if necessary.

There were missing dots on cameras 1, 4, 7, 10, and 13. There were brightness/contrast issues on those plus scans 11, 14, 16, 17, 19, and 20.

Camera 1 was the only one that showed NUC-related artifacts, having both a gradient that was darker at the upper-right corner and an overall reduction in contrast. This alone would not have impeded calibration, as seen in the image below:



The calibration is impacted by the fact that the target fades and eventually becomes invisible as the scan progresses around the edges on all three passes in the lower-left corner of this image.



The figure to the right zooms in on the lower-left region. The blue arrow points at the mostly-faded target; there is lens flare or other another large reflection of the target to its lower right. The orange circle shows the approximate image region within the primary mirror for the

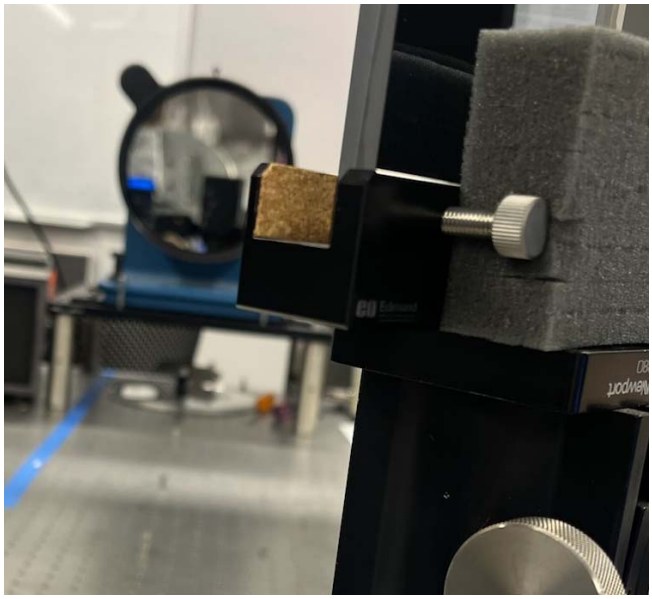
collimator. The target disappears while it is still within the region of the primary mirror.

Other cameras had the expected contrast except when the target disappeared, in which case auto-range code brought the lens flare or other image objects up to the maximum temperature, corrupting the image. Individual cases studied:

- Camera 1: Lower-left region had missing target in all three rectangle scans.
- Camera 4: Upper-left corner had a dimmer target, upper-right region had missing target on all three passes.
- Camera 19: Upper-right corner had reduction (but not missing) target in all three passes.
- Camera 20: Upper-right corner had reduction (but not missing) target in all three passes.

This reveals that ***the problem with the scans is that the target dims and disappears in some images, in at least some cases this happens when it is well within the region covered by the collimator's primary reflector***. The camera NUCs were all sufficient and their stability over the run sufficient (without flashing).

Testing on ***8/28/2025*** by moving to the orientation where the target was not seen on camera 1 (frame 1616, orientation -97.8516 -48.0625) did make the target disappear from camera 1's view. Inspection revealed that the holder for the collimator's fold mirror is blocking the view of a portion of the primary mirror, so the target disappears behind it from some viewing directions (see the brown cork region in the image to the right).

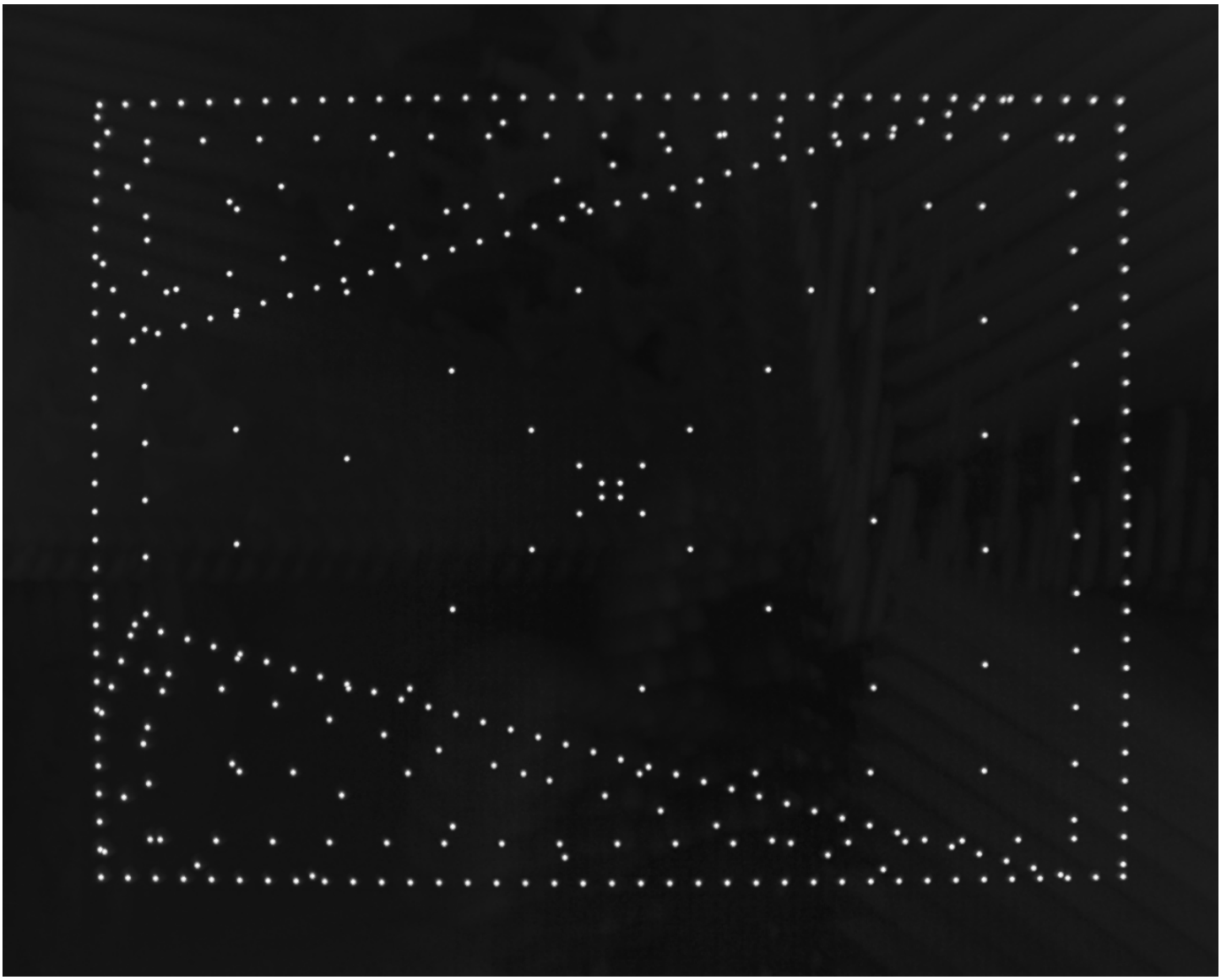


The optical path was adjusted to remove this interference. Scanning along the top and bottom after adjustment by going up and down 53 degrees from horizontal, which is the range that the scans cover.

While looking at the behavior, it was noticed that the overall pier had shifted down on the gimbal assembly and the counterweight was running into the tripod legs. Adjusted the tripod and collimator setup to compensate.

The scan was redone, including target lateral position estimation.

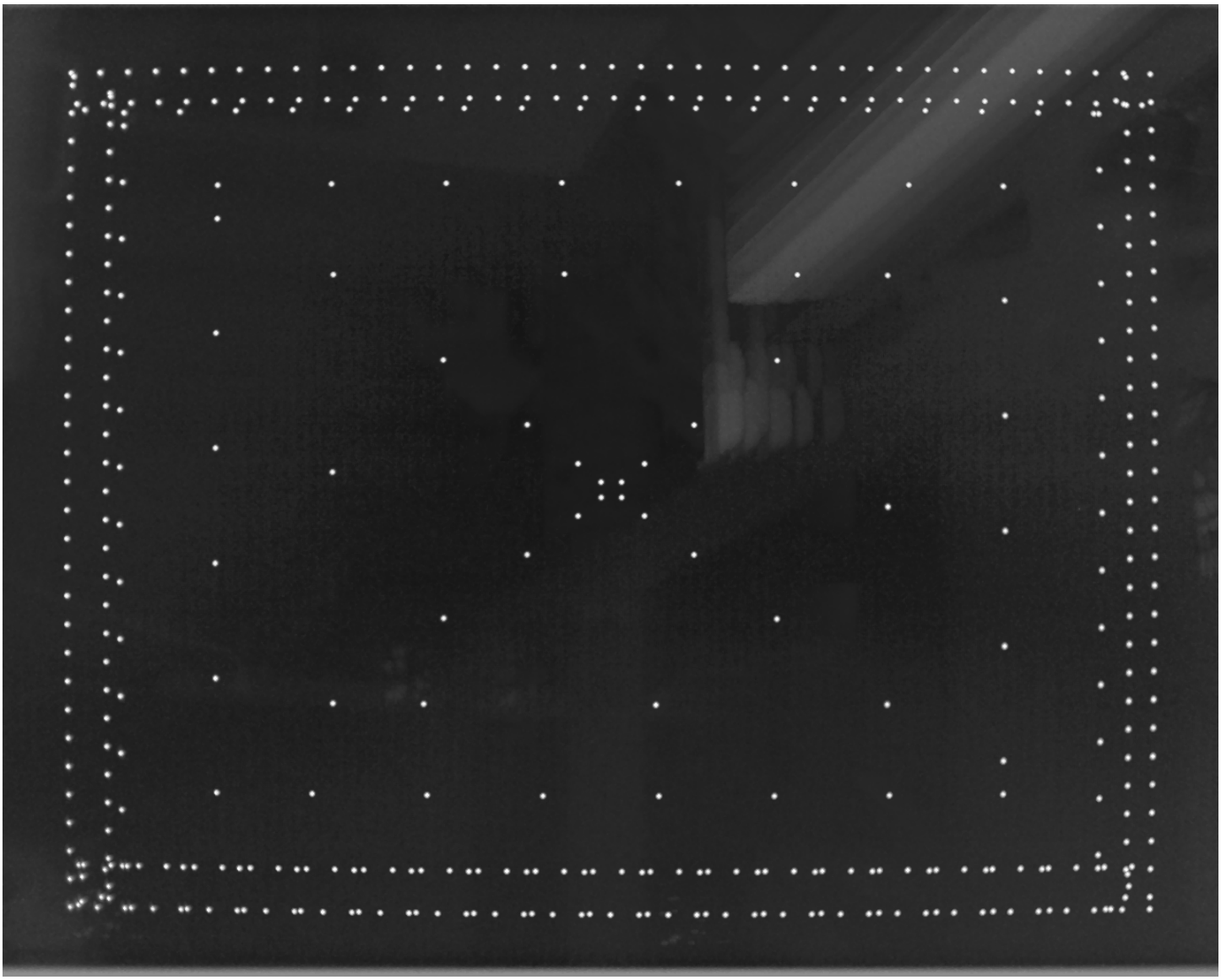
The scans were done with a 100-pixel edge gap around the cameras. This resulted in all points for a camera being within the scan (as hoped). It also resulted in points from neighboring cameras not being visible except on the cameras with large overlaps at top and bottom (no overlap with the camera to the image's right, only those above and below):



This means that the seams do not have the same point visible in both cameras, which should mean that the stitching will not be perfect between cameras.

Testing on ***8/29/2025*** used a micromoter-offset Z stage for the mirror to provide finer control over the range of visibility. Calibration was re-run using a 70-pixel margin rather than a 100-pixel margin. The adjusted lateral images were not as perfectly overlapped as before, but they are plenty close to each other to avoid the edges:

The complete scans seemed to have captured all points, and the reduced margin brought points from neighboring cameras into view:



This calibration was validated against video taken outside with objects far from the camera.

Various scripts used during LWIR calibration

1.sh: /home/rtaylor/build/ASDP_Render_Module/Target_Calibration_Make_Scan --frames 3 3_rup.json targetConfig.json gimbal.json

2.sh: /home/rtaylor/build/ASDP_Render_Module/Collect_Calibration_Data --autoRange --removeSpikes 200 10.10.11.21 3 target_1_poses.csv target_lateral_1_images

2_watch.sh: /home/rtaylor/build/ASDP_Render_Module/Collect_Calibration_Data --autoRange --median 10.10.10.21 3 target_1_poses.csv target_lateral_1_images

3.sh: /home/rtaylor/build/ASDP_Render_Module/Target_Calibration_Estimate_Lateral 3_rup.json targetConfig.json gimbal.json 65500 .

4.sh: /home/rtaylor/build/ASDP_Render_Module/Camera_Calibration_Make_Scan cameras_opt.json targets_lateral_opt.json gimbal.json --frames 3 --topMarginPixels 70 --bottomMarginPixels 70 --leftMarginPixels 70 --rightMarginPixels 70

5_watch.sh: /home/rtaylor/build/ASDP_Render_Module/Collect_Calibration_Data 10.10.10.21 3 poses.csv camera_images_flat

6_autorange.sh: /home/rtaylor/build/ASDP_Render_Module/Collect_Calibration_Data 10.10.10.21 3 poses.csv camera_images_autorange --readFrom camera_images_flat --autoRange --median

5_watch_autorange.sh: /home/rtaylor/build/ASDP_Render_Module/Collect_Calibration_Data --autoRange --median 10.10.10.21 3 poses.csv camera_images_autorange

7.sh: /home/rtaylor/build/ASDP_Render_Module/Camera_Calibration_Estimate_Distortion_Extrinsics cameras_opt.json targets_lateral_opt.json gimbal.json poses.csv camera_images_autorange 65500 --writeMaps maps.csv 3_opt.json

1_recurse.sh: /home/rtaylor/build/ASDP_Render_Module/Target_Calibration_Make_Scan --frames 3 cameras_opt.json targets_lateral_opt.json gimbal.json

3_recurse.sh: /home/rtaylor/build/ASDP_Render_Module/Target_Calibration_Estimate_Lateral cameras_opt.json targets_lateral_opt.json gimbal.json 65500 .

Validation of Phase 2

The optimization was first tested using a simulated rendering environment that produces images from any camera at any selected rotation based on a Rendering Module JSON camera configuration file and a target-describing JSON configuration file. This includes individual camera translation, rotation, FOV, and distortions. This simulation reads in a list of gimbal orientations and sets of cameras and will produce a set of images at each pose including specified random jitter in gimbal orientation and specified per-pixel noise in each image.

The JSON configuration files provided to the optimizer have known error offsets applied to the individual camera FOVs, positions and orientations along with different distortion models (no distortion). They also have known error offsets to the target positions. This tests whether the optimizer can measure and correct these errors.

The optimizer reads the JSON input files along with the camera orientation and image files and produces optimized JSON files for the camera configuration and for the target locations.

The optimized JSON file for the camera is compared with the one fed to the simulator by a *Compare_Configurations* validation program in the Render Module that reports the mean and max 3D spatial distance in meters and projected difference in pixels between pixel locations along the edges of each camera, and then the average mean and overall maximum across all cameras. The projected pixel differences indicate how much misalignment is expected to be visible to the pilot.

Results

The target positions were moved by 0.1 meters in opposite directions in X and 0.02 meters in opposite directions in Y compared to the simulated position before being given to the optimization procedure. The adjusted points were used both to generate the view angles to be simulated and to initialize the optimization.

The initial average RMS reprojection errors across cameras when using incorrect target locations was around 2.5 pixels. After optimization of target locations, the average errors were around 0.25 pixels. The actual offsets vs. estimated offsets were as follows:

- Actual 0.1, 0.02, 0, -0.1, -0.02, 0
- Estimated: 0.11297, 0.0230773, -0.114089, -0.0827659, -0.0126162, -0.0278583

Comparison using *Compare_Configurations* showed an average mean pixel offset of 0.25 (matching the reprojection errors estimated by OpenCV) and maximum of 19.3.

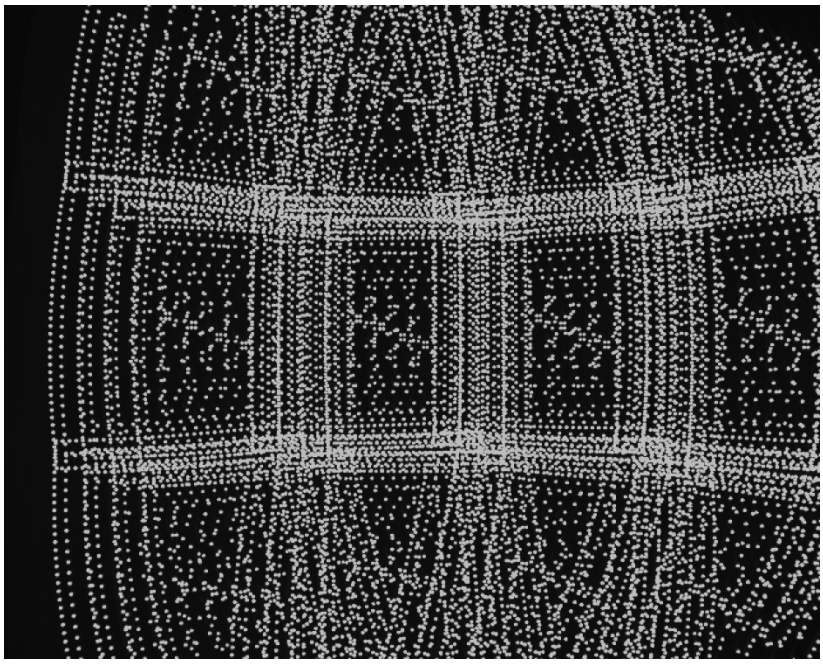
Application to real camera ball

In early March 2026, an initial Phase-2 procedure was applied to camera 5, which is a visible-light camera that includes four wide-FOV cameras, using a pair of targets located around 11 meters and around 5 meters from the gimbal center of rotation (both turned on at once).

Initial testing showed that the projected locations of the targets on the cameras were less than 250 pixels across on the narrow-field cameras, so the threshold for calibration was adjusted to 200 pixels. The distance between target centers on the wide-field cameras is around 74 pixels. The code was modified to scale the distance based on the ratio of the camera hFOV to 40 (the default for the narrow-field cameras) so that it would work for both camera types.

The following command was run to generate the scan for the camera:

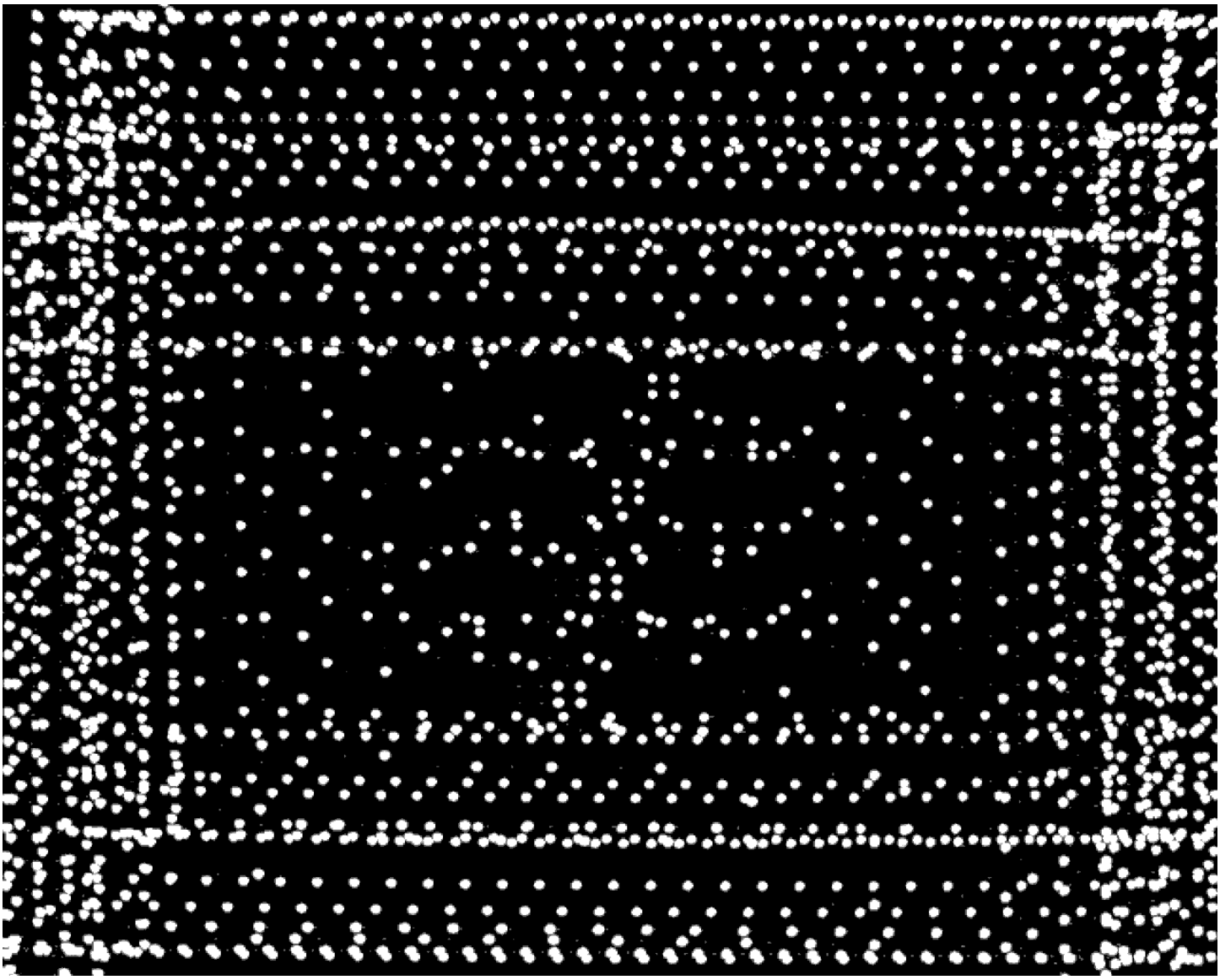
Camera_Calibration_Make_Scan 5.json targets.json gimbal.json --frames 3 --densityScaleFactor 1.3 --noWFOVScan --topMarginPixels 20 --bottomMarginPixels 20 --leftMarginPixels 20 --rightMarginPixels 20



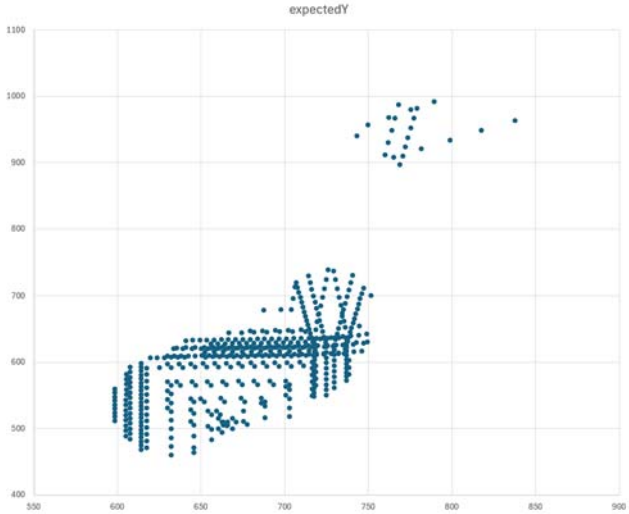
The maximum-value image for camera 22 across all images (seen to the right) shows that for the initial lenses and with the wFOV cameras moved forward, there are no occlusions, but the camera cannot see the entire vertical range of the narrow-field cameras. It can see the entire horizontal range of the hemisphere it is looking at (more than halfway across the center cameras, and beyond the edge cameras).

The other three wFOV cameras behave similarly, overlapping the entire horizontal fields of view but not the entire vertical fields of view.

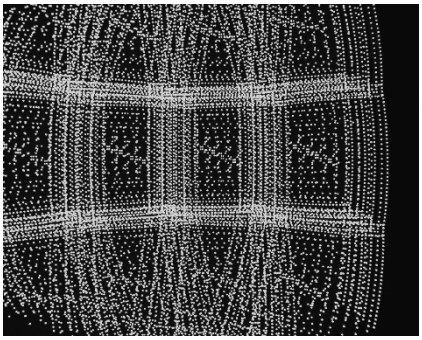
The narrow-FOV camera images had points across the entire frame, but because there are two different targets it is not clear whether a single target can be seen very near to each edge. The center camera's maximum overlay image is seen below. There do not appear to be any points outside of the target locations whose brightness is near the threshold; the small dots away from the targets are less than half of the intensity of the targets.



When the calibration solver was run with an ***intensity threshold of 1020 and a distance threshold of 150 pixels***, camera 24 had only a small subset of the expected points visible (above 590 in X and above 450 in y, and a small grouping. See the plot to the right.



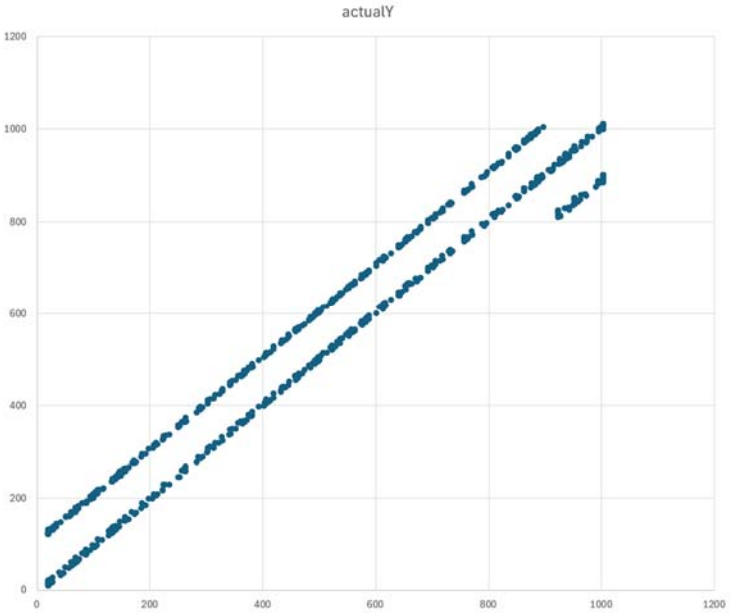
As seen in the overlay image, there were many more visible locations in the scan.

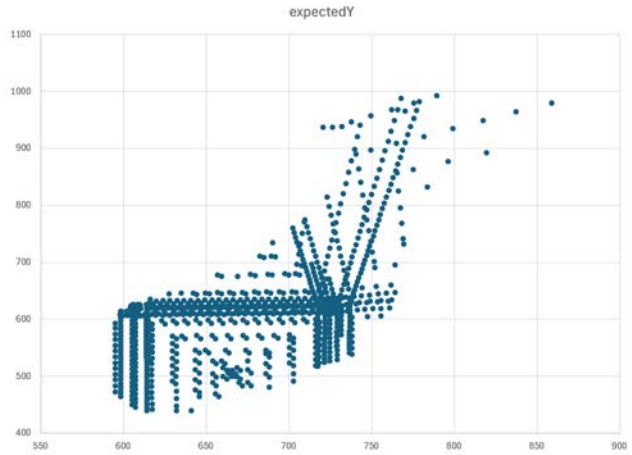


The restrictive thresholds were to try and remove incorrect target matches when the looked-for target was not visible due to misalignment of the cameras compared to the ideal case. Re-running the calibration step with a distance threshold of 200 pixels produced the following map for camera 24. The map

has a little bit more filled in around the original map, including a bridge between the two separate islands, but it is still missing most of the points that are visible in the image.

Camera 14 had the expected range of expected X, Y locations. Its actual vs. expected X values were along several lines rather than one, and its Y values were along lines that were even more separated (see below):

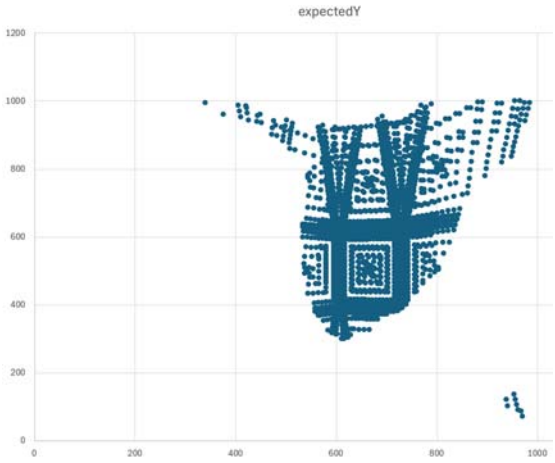




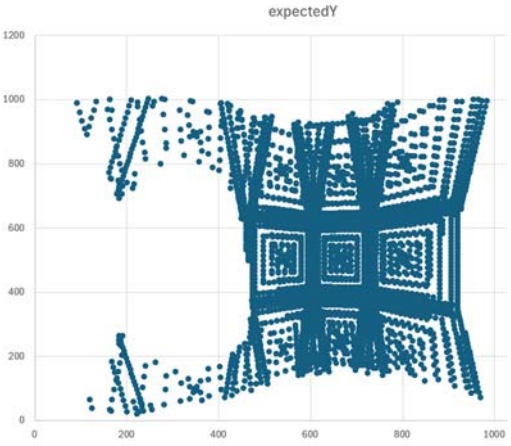
This separation is presumably due to an incorrect target location estimation caused by the poorly-located cameras pushing target estimations to bad locations.

Using an intensity threshold of 1020 and a distance threshold of 200, we found 29889 points, not including many points beyond what was found in camera 24. The same happened (with one more point found) with a brightness threshold of 1000 and a distance threshold of 200.

Using an ***intensity threshold of 1000 and a distance threshold of 400*** we found 40267 points, including a much broader range on camera 24 (see image below right).



Using an ***intensity threshold of 1000 and a distance threshold of 600*** we found 50708 points, including an even broader range for camera 24 (see image to the below left). This indicates that **a distance threshold will not accurately discriminate between the two targets** in the system. The projected distance between targets is much less than 400 pixels and the threshold must be >400 pixels to capture all valid correspondences.

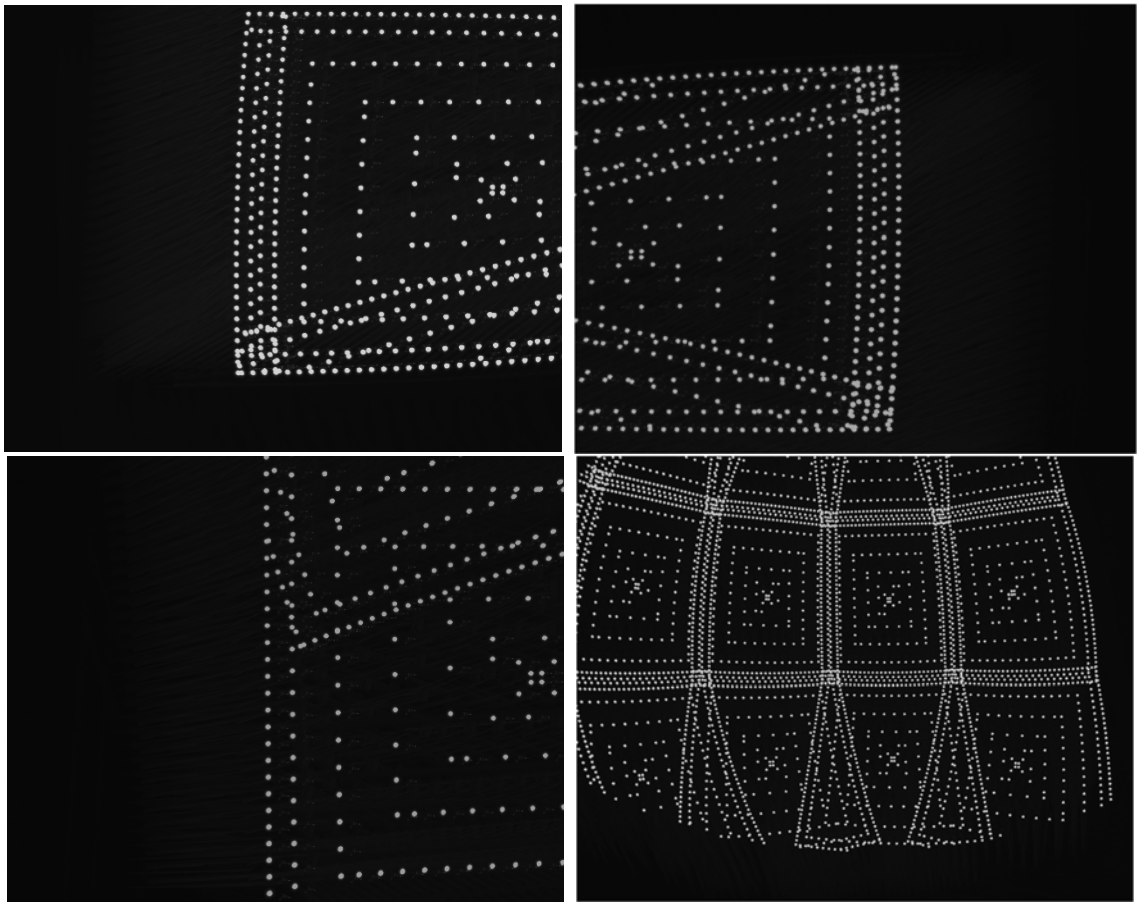


As a result, the procedure was modified to allow the command-line specification of a break in the scan where the system waits for the operator to turn off the first target and turn on the second target such that each frame has only one target present. When this is done, the distance threshold can be set to a very large value because there is only one target to see in each image, so there is no possibility of confusion.

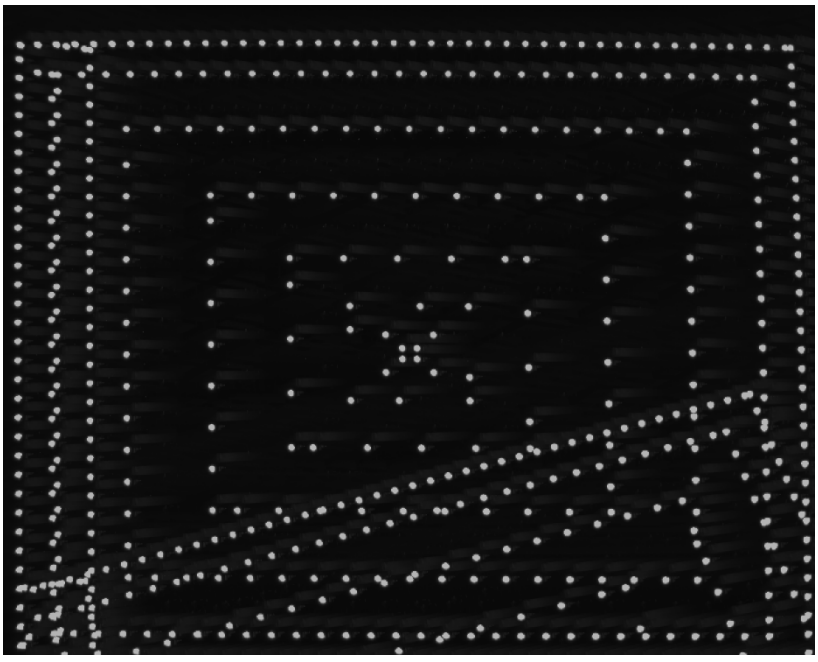
Turning on only one target at a time

The *--waitForTargetChange* command-line option was used with ***Collect_Calibration_Data*** to collect images with each target separately. A new ***max_image_overlay_per_target.py*** script was used to produce the per-target, per-camera

overlay images. These revealed that the first target appears to be well centered in the cameras but the second target is not. The overlay for cameras 1, 4, 21, and 25 for target 2 are shown below.



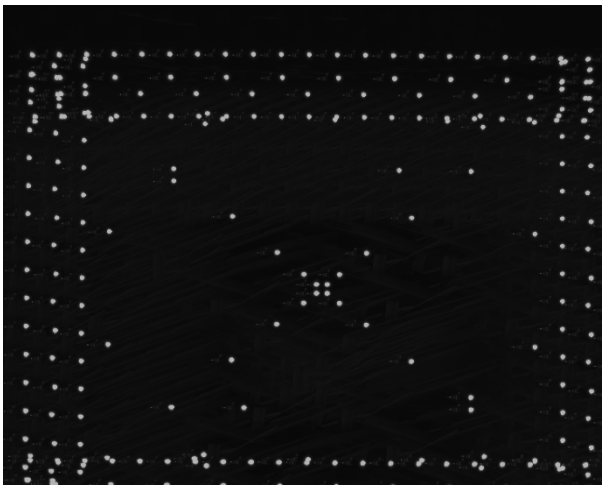
For comparison, camera 1 for target 1 is shown to the right. The target-1 image has the expected scale and is shifted in the (rotated) horizontal direction, consistent with the single-depth calibration results for the first-calibrated visible-light camera ball.



In contrast, target 2 images are smaller than the expected size and shifted in both directions (more in the rotated vertical direction). The fact that the pattern appears smaller than expected indicates that the gimbal is moving further than expected between points, which means that the target is located further from the camera than expected.

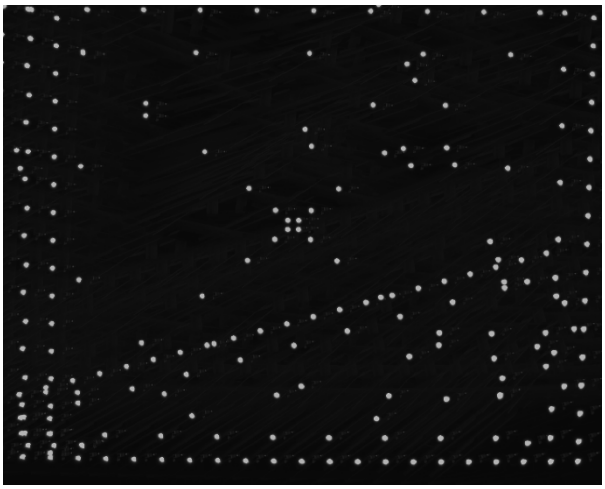
The target's initial estimated location was 5.41 meters in the +Y direction and 0.035 meters in the +X direction in camera space. Re-measuring showed about ***216 inches (5.49 meters) in Y and 14 inches (0.36 meters) in X***. That explains the rotated vertical distance offset but not the large rotated horizontal offset or the scale difference.

One likely cause seems to be a problem with the 0.2m +Z offset in the gimbal.json file. If this were not applied at all (leaving the origin at the camera center of rotation), we would see a vertical shift in the wFOV cameras such that their actual location is higher than their expected location. This would cause visible target locations to be lower than expected, which is the opposite of what we are seeing. This effect would be increased if the modeled location were shifted by -0.2 meters. A new measurement shows that the offset is actually ***0.34 in Z and 0.05m in Y***. This could explain the offset and perhaps some of the scaling.



Re-running the camera calibration for camera 11 only using the new offsets for the trackers.json and the gimbal.json file with a scale factor of 2.0 and a step size of 60 (to reduce the number of measurements) produced a reasonably-sized and mostly centered scan (seen in image to the right).

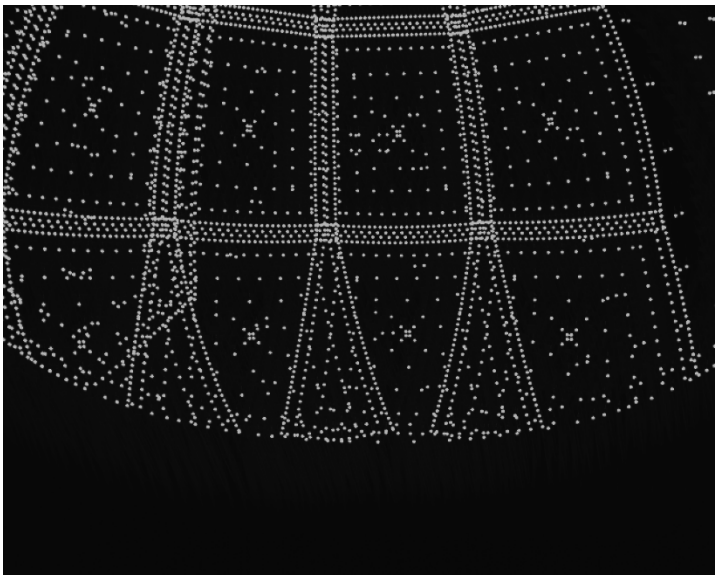
Doing the same for camera 1 also produced better results (see image to right).



Re-ran with the updated configuration files and using a single target at a time with --densityScaleFactor 1.5 and edge margins of 20 pixels.

The resulting RMS errors were between 13 and 145 pixels, which is much too high. Camera 25's overlay plot for the second (closer) target has a lot of vertical offset (image below left), as does camera 1 (below right, which also has a smaller image spread than seen in the test run to the right).

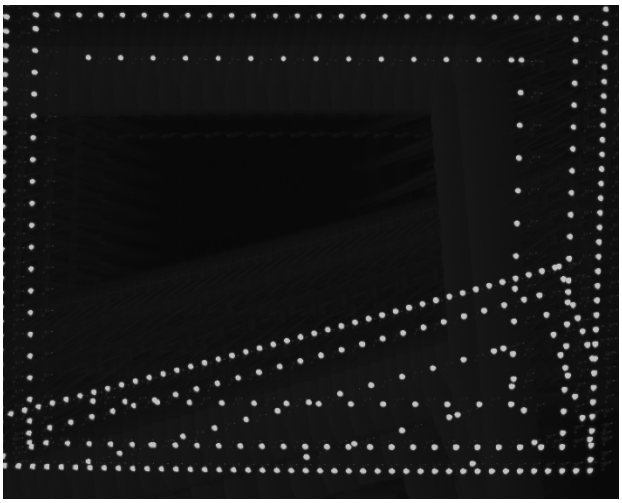
The gimbal.json file being used has the corrected gimbal



configuration file, so should have corrected alignment according to the initial single-camera test runs above.

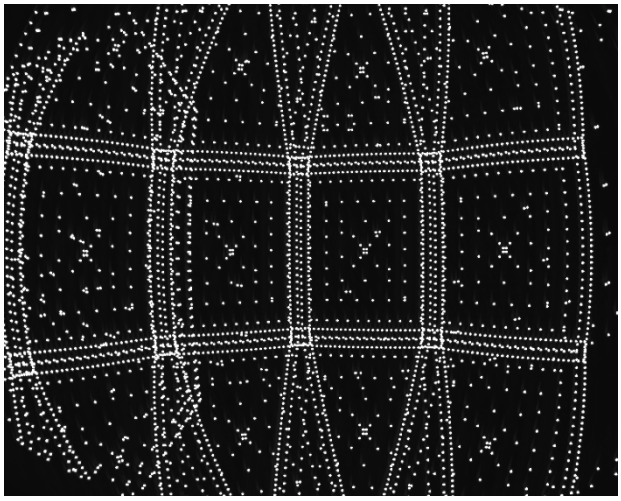
Checking the freedom of motion for the gimbal to ensure that it is not binding during the run seemed to show good clearance. The ball is slightly top-heavy on the mount, but not to the extent that it should impede movement.

Re-running the scan for only target 2 and only camera 1 while watching the first part of the scan showed no collisions; the gimbal had been re-homed before this single-camera run. The first portion of the scan indicates that the covered range is about as expected (see image to right).



This seems to indicate that the gimbal controls are losing track of the correct orientations somewhere during the scan.

Reviewing the images from all per-target, per-camera overlays from the full run reveals that all target-1 scans seem to work as expected (see camera 25 target 1 image below right).

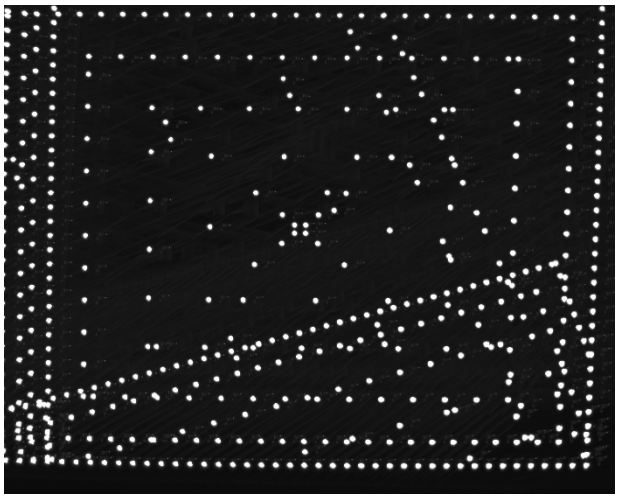


This seems to indicate that there is a discontinuity between the two sets of scans.

The difference between the two scans in the waiting approach is that there is a long delay between when the first scan stops and when the second scan starts. Because the telescope mount that we are using as our gimbal tries to track objects over time, it seems possible that it would be moving while sitting still and would also be moving its internal clock forwards. If this is the case, then the delay could potentially cause an offset. However, the time is reset before every move command and then tracking is disabled after every move, so this should not be happening.

Another possibility is that the code is always using the first target location when there is more than one target, but inspection shows that the Collect_Calibration_Data motion is insensitive to the target ID and simply uses the specified rotation degrees for each pose, which are read from each line.

Rerunning using just the target-2 entries from the poses.csv file (overwriting half the images from the earlier run) ***produced images that were properly aligned***, as seen in the image from camera 1 target 2 to the right. Some of the cameras had RMS errors under 10 pixels (3, 2, 1, 17, 6), most were in the hundreds, and 22 and 25 were over 1000.



Ran another very sparse scan (1680 total frames) and scanning the target-2 values first, then running again to scan the target-1 values without homing the gimbal between the runs.

Camera_Calibration_Make_Scan --frames 3 --topMarginPixels 40 --bottomMarginPixels 40 --leftMarginPixels 40 --rightMarginPixels 40 --stepPixels 500 5.json targets.json gimbal.json –noWFOVScan

Collect_Calibration_Data 10.10.11.12 5 poses.csv camera_images_t2 --target 2

Collect_Calibration_Data 10.10.11.12 5 poses.csv camera_images_t1 --target 1

The maximum-projection images from the first scan (target 2) scan had quite a bit of pattern shift between cameras. Camera 19 (the first camera) seemed well-centered. The next cameras to its left across the bottom row: 16 (large shift

down), 13 (centered), 10 (centered), 7 (shifted up), 4 (centered), and 1 (shifted up). Stereo-pair cameras 22&23 had largely the same centering with distortion of the points consistent with stereo), but stereo-pair cameras 24&25 had a large shift. The stereo-pair cameras *did not have internal pattern disruptions consistent with mis-aiming of the ball* due to some sort of fixed offset during the run. This leads to the conclusion that ***individual camera orientations rotate around X and Y quite a bit compared to the as-designed geometry***. The shift between cameras 19 and 16 seems to be about 508 – 616 in Y and 660 – 673 in X. The Y shift of 108 pixels out of 1024 is 0.1 of the image, whose estimated vertical field of view is about 34 degrees, so a shift of about ***3.4 degrees***.

The maximum-projection images from the second scan (target 1) have the same stereo-pair behaviors as for the first scan, indicating that they are almost certainly caused by camera aiming differences rather than target-location errors. Visual inspection seems to show agreement between the pointing directions and spacing of each of the wFOV cameras, so this ***must be due to shift in the sensor location w.r.t. the lens center of projection***. The pointing differences along the bottom row of cameras: 19 (low),16 (centered), 13 (low), 10 (high), 7 (centered/low), 4 (high), 1 (low) do not seem consistent between runs, so are presumably caused by a ***combination of camera orientation errors and target location errors***.

Even though there were lots of missing border readings, the resulting RMS reprojection errors were around 1.5 for the narrow-FOV cameras and around 3-5 for the wFOV cameras. Displaying the resulting stitched images at a distance of 12 meters (approximately the distance to the far end of the basement) saw out-of-bounds errors on some cameras and slight edge misalignment on the cameras. This data is stored in calibrate_camera_5r3.

Ran a two-step sparse calibration using lateral target estimations step: This time, we used the two-step process to first estimate the camera orientations and target lateral positions and then using that new set of configuration files to generate the entire scan. The hope is to remove out-of-bounds points.

Gimbal_Test –home

Target_Calibration_Make_Scan 5.json targets.json gimbal.json --frames 3

mkdir target_lateral_1_images

Collect_Calibration_Data 10.10.11.12 5 target_1_poses.csv target_lateral_1_images --shift 6

mkdir target_lateral_2_images

Collect_Calibration_Data 10.10.11.12 5 target_2_poses.csv target_lateral_2_images --shift 6

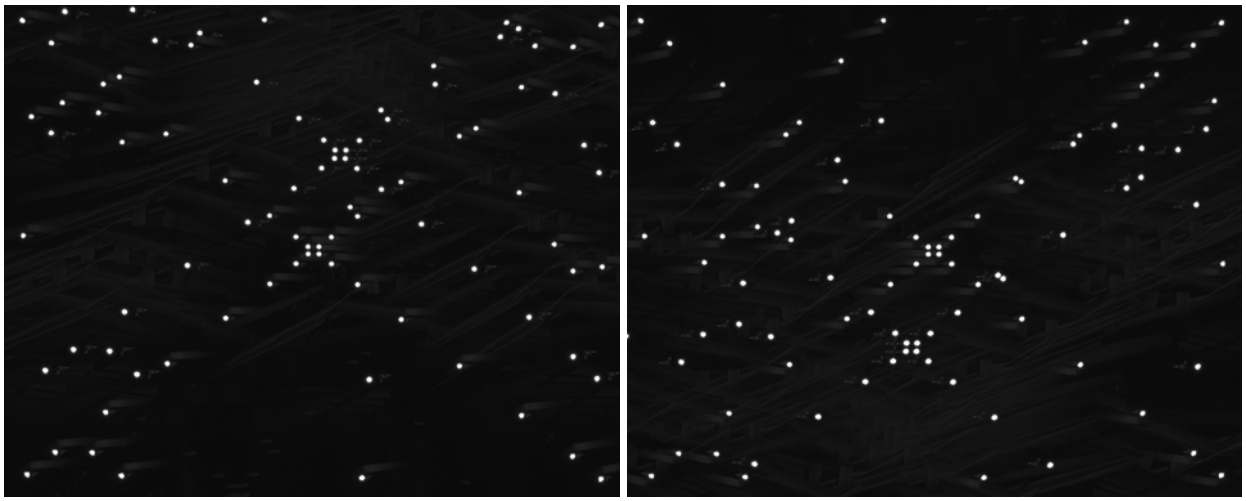
Target_Calibration_Estimate_Lateral 5.json targets.json gimbal.json 60000 .

Camera_Calibration_Make_Scan --frames 3 --topMarginPixels 40 --bottomMarginPixels 40 --leftMarginPixels 40 --rightMarginPixels 40 --stepPixels 500 cameras_opt.json targets_lateral_opt.json gimbal.json –noWFOVScan

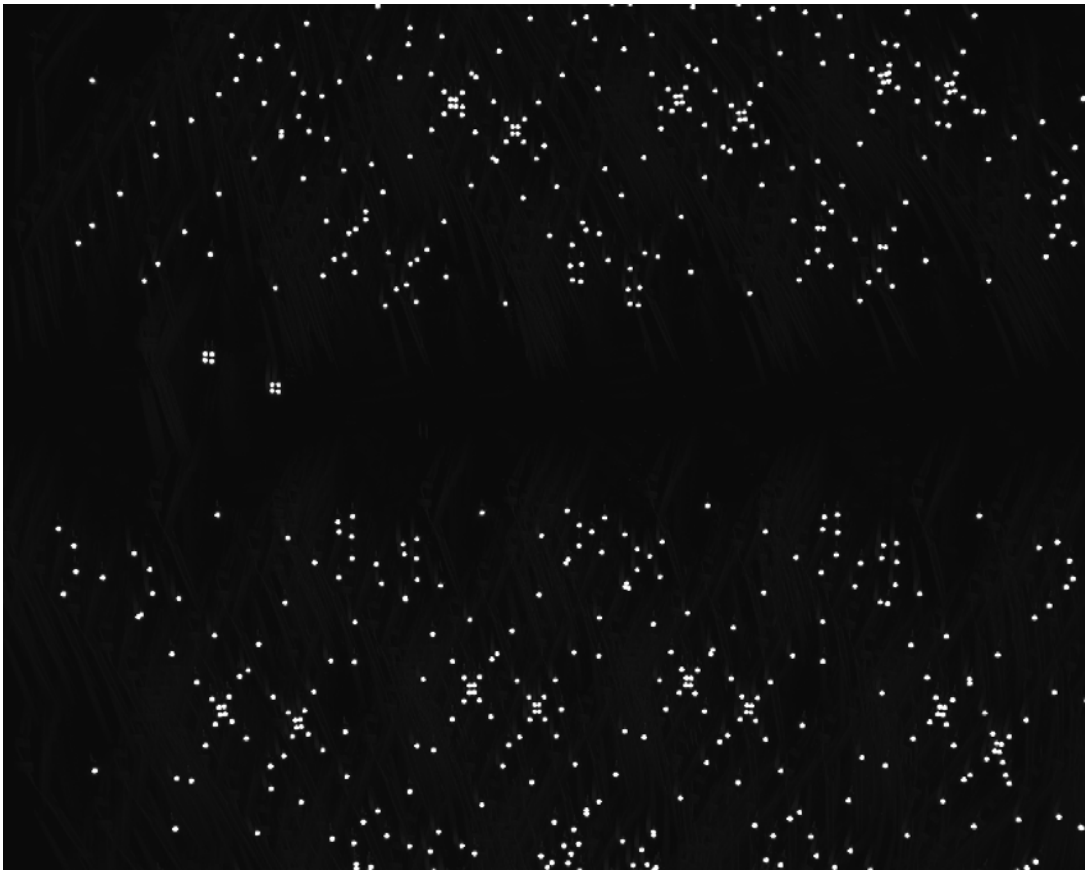
mkdir camera_images

Collect_Calibration_Data 10.10.11.12 5 poses.csv camera_images --shift 6

The point locations found by Target_Calibration_Estimate_Lateral were all within a few pixels (looks like less than 10 off in either direction) for all images. Both targets were left on during the entire scan and the data was collected for both targets in a single run; it is safer in practice for both lateral estimation and camera estimation to turn on and scan one at a time. Cameras 19 and 16 (the first two scanned) had what looked like centered patterns (with a second shadow pattern due to the other target):



The rest of that row looked good, except for camera 1 which seemed to be missing its center four locations for target 1. Those center four locations appeared to have shifted out of the image when looking at the camera-22 image; note the pair of clusters of 4 points in the center of the image below, which have shifted down from where they should have been in the top row:



Even more disturbing is the ***lack of samples in the center of the camera-22 image, for the center row of narrow-field cameras***. ***The cameras in the center row each have a large gap across the middle and those at the top have points only along one edge at the end of the target-1 scan***.

Re-running the target-1 scans for camera 11 showed a large vertical offset compared to the expected location, which matched the images taken by the camera:

Moving the gimbal to location (0,0) showed a tilt consistent with this offset, so we're seeing a ***gimbal pointing offset that seems to be happening during the scan and that persists***. Re-homing and then running the locations for camera 4 followed by the locations for camera 1 does not cause anything to go wrong, but running the whole range from the first camera-4 to the last camera-1 point moves the ball into collision with the power supply mounted on a post behind it. Whittled this down to ***the following set of three moves*** (no two of which cause this):

- 274,83.1432,35.2833,1,3,1
- 274,83.1432,35.2833,22,3,1
- 274,83.1432,35.2833,23,3,1
- 275,80.8549,35.299,1,3,1
- 275,80.8549,35.299,22,3,1
- 275,80.8549,35.299,23,3,1
- 276,80.8564,32.9106,1,3,1
- 276,80.8564,32.9106,22,3,1
- 276,80.8564,32.9106,23,3,1

Checking for firmware updates on the iOptron mount reveal that the last one was in 2023, so there is not a newer release since long before we purchased our mount.

The document for GEP command states that the 18th and 19th digits specify the side of the pier and pointing state (which way the counterweight is); do these also have significance in the command to set this? No; there are separate commands to set these things.

We seem to be sending fewer characters than available for declination, 7 after the sign rather than 8. Switching to 8 digits had the same behavior… but it was still sending only 7. Modifying it to really send 8 also did not make a difference in the behavior.

Negating both axes causes the same failure in the other direction. Zeroing one of the axis does not fail. Negating one or the other axis does not fail.

Copilot thinks: Your coordinate (Dec ≈ +80.9°) places you firmly in the **zenith/pole singularity region**, where iOptron's MS1 shortest-path solver becomes unreliable. This can make it take the longer path around instead of the shorter path. Its proposed solution is to check whether abs(Dec) > 80 degrees; if so, first command a move to well below 80 degrees and then a second move to the correct location. Adding this behavior did not help the cases that we're seeing and did not stop the behavior.

Once the angles are converted to the commanded angles (6.8568 and 324.717), it thinks that the problem is that the system is flipping when crossing the meridian. The behavior is set using the :SMT command and the Pier Side can be queried using the :GEP command, it is the next to last digit; the Pier Side should change when we have the long move. By stopping at some distance beyond the meridian, the slew will complete but will not be at the requested orientation. There is no other indication that it failed. However, the counterweight does not seem to be switching pier sides and the head is not rotating 180 degrees, so this seems unlikely to be the problem. When asked about the fact that the counterweight is moving up, it recommended either allowing flips (avoiding its choice of the *inverted solution*) or increasing degrees past meridian to 10-15 degrees so that it doesn't get forced into this solution. It is also possible to do a similar forcing slew to an intermediate point. **Setting the meridian treatment using *:SMT115#* resulted in the problem disappearing for both cases**.

Verifying during motion: To avoid future such instabilities causing crashes, the *waitForSlewStop* method is augmented to watch the motion to ensure that the system does not move beyond 90 degrees tilt on either side of the pillar. Watching altitude and azimuth using *:GAC#* did not always update during motions. The longitude and latitude from the status command don't change. Watching *:GEP#* found ranges of 0-88 and 272-360 to be valid for motions in all cases. Testing this range setting causes (1) --home to fail, and (2) sometimes failures near (0,0) like (10,10) where the angle is around 190 degrees. Added the range within 88 degrees of 180, which seems to cover all cases.

Re-ran two-step sparse calibration using lateral target estimations step: Again used the two-step process to first estimate the camera orientations and target lateral positions and then using that new set of configuration files to generate the entire scan. The hope is to remove out-of-bounds points.

Gimbal_Test –home

Target_Calibration_Make_Scan 5.json targets.json gimbal.json --frames 3

mkdir target_lateral_1_images

Collect_Calibration_Data 10.10.11.12 5 target_1_poses.csv target_lateral_1_images --shift 6

mkdir target_lateral_2_images

Collect_Calibration_Data 10.10.11.12 5 target_2_poses.csv target_lateral_2_images --shift 6

Target_Calibration_Estimate_Lateral 5.json targets.json gimbal.json 60000 .

Camera_Calibration_Make_Scan --frames 3 --topMarginPixels 40 --bottomMarginPixels 40 --leftMarginPixels 40 --rightMarginPixels 40 --stepPixels 500 cameras_opt.json targets_lateral_opt.json gimbal.json --noWFOVScan

mkdir camera_images

Collect_Calibration_Data 10.10.11.12 5 poses.csv camera_images --shift 6

We had **the same failure** as in the previous run, at the following line (which was the same part of the scan that failed last time, the three small moves near the center of the last camera to be run on the first row): *Writing 3 images for frame 275 camera 1 (frame index 275 of 1680)*; here are the lines leading up to that and the one after (which causes the problem):

- 272,79.2254,31.0788,1,3,1
- 272,79.2254,31.0788,22,3,1
- 272,79.2254,31.0788,23,3,1
- 273,83.0477,32.8397,1,3,1
- 273,83.0477,32.8397,22,3,1

- 273,83.0477,32.8397,23,3,1
- 274,83.1115,35.228,1,3,1
- 274,83.1115,35.228,22,3,1
- 274,83.1115,35.228,23,3,1
- 275,80.8232,35.2435,1,3,1
- 275,80.8232,35.2435,22,3,1
- 275,80.8232,35.2435,23,3,1
- 276,80.8255,32.855,1,3,1
- 276,80.8255,32.855,22,3,1
- 276,80.8255,32.855,23,3,1

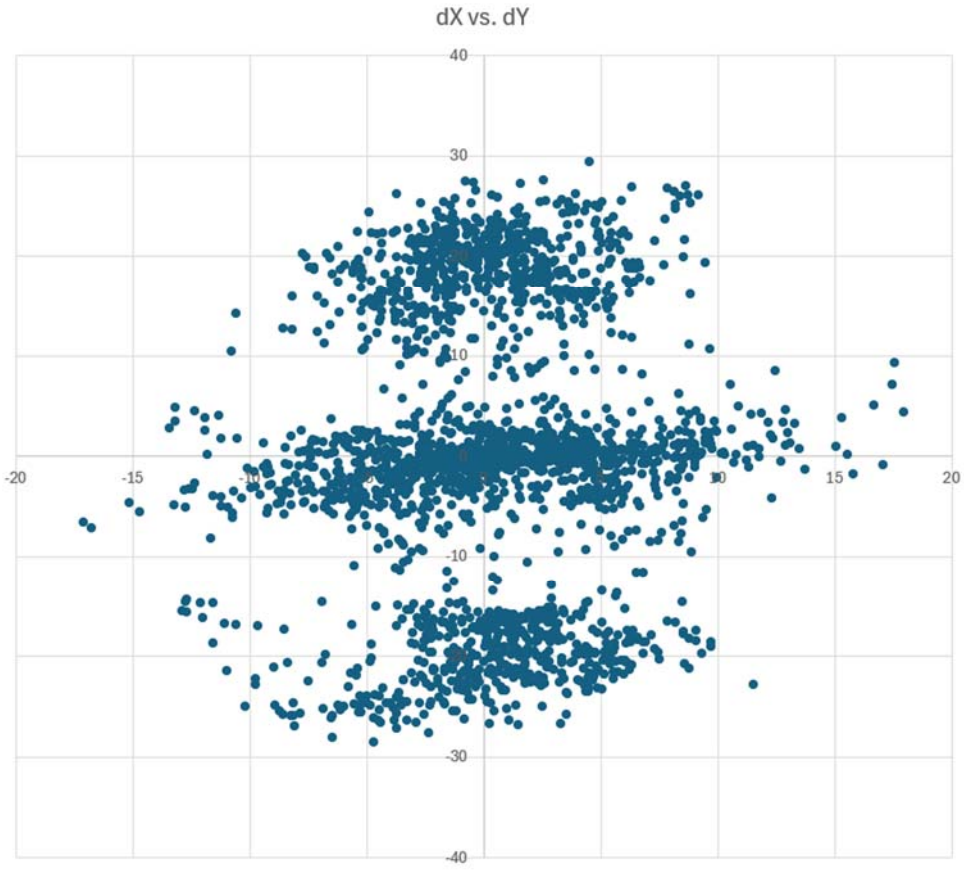
The test code for 88 degrees failed because **the readings skipped from 272+ to 268- while the motion was heading out of bounds**. Adjusting the dead zone to be 80 degrees rather than 88 to avoid crashes in the future.

There is some likelihood per run of each command (both original ones and new ones) causing movement to the edge. The code has been modified to recover from that by moving back towards vertical and then retrying the move after going back to (0,0). This seems to keep the scan running.

The run seemed to suffer from **missing-target confusion**, so produced a result with large reprojection errors.

Re-ran using lateral target estimations step, using a single target at a time: calibrate_camera_5r5. The scans all completed. The per-target camera scans for the narrow-FOV cameras with target 1 were all well centered. The wFOV cameras had similar distributions to earlier scans. For target 2, the narrow-FOV cameras had a slight offset (vertical when viewed landscape) but still appeared to contain all images of the target scanned within a particular image (though half of one edge were not further than 40 pixels from the edge). The wFOV cameras looked like in earlier scans.

The RMS errors for the narrow-FOV cameras were between around 1 and 2 pixels (most much less than 2), with the wFOV errors in the tens of pixels. Adding a *--noWFOV* parameter to the estimation routine dropped the errors for the narrow-field cameras slightly, to between around 1-1.7 with an average of 1.28. The cameras are not well matched at the edges, having several-pixel offsets. At least some of the cameras have points that are far from the image. All located points are more than 10 pixels from the edge of the image. Over all cameras, the distance from expected to actual location is bounded by +/- 60 in X and around +/-80 in Y. Over the narrow-FOV cameras, the distribution was multimodal (see plot to right). This matches the



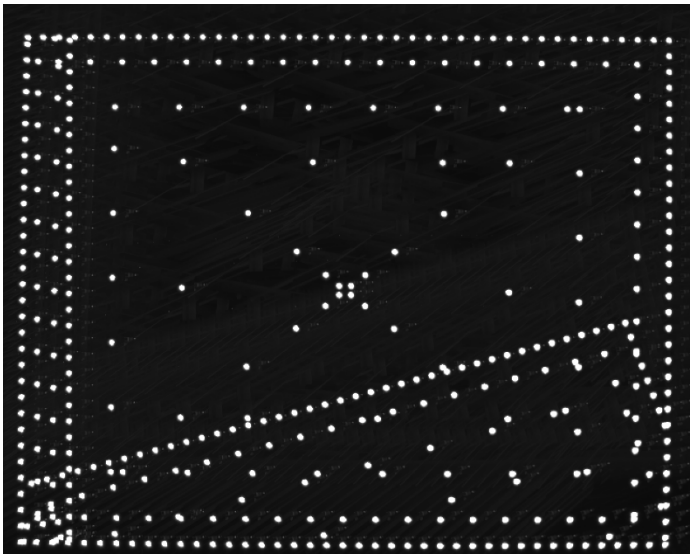
observed Y offsets in the captured patterns across cameras.

The stitching was poor at the edges between cameras, offset by many pixels.

The ***optimized fields of view for all cameras were all the same, as if they had not been adjusted***. The aspect ratio and principal point are fixed during optimization, but it should allow the overall zoom to adjust. Allowing the principal points and aspect ratios to change dropped the RMS error on the narrow-FOV cameras to 0.52 on average; the resulting edge alignments were much worse, the mesh loaded more slowly than expected but does not seem to have extreme outliers from looking at the diffs file (and each mesh loaded about the same speed for the small meshes – perhaps the total volume of points is set based on the wFOV cameras, causing the narrow-FOVs to bunch together?). The slowdown was due to the large number of map points constructed in the 2-target case; reducing this sped up mesh construction.

Re-ran two-step sparse calibration with a single target at a time, using standard sampling to improve matching at the edges. calibrate_camera_5r6. This was the same protocol as before, with the following line changed to remove the --stepPixels option (using 30-pixel rather than 500-pixel steps along the outer edge):

> *Camera_Calibration_Make_Scan --frames 3 --topMarginPixels 40 --bottomMarginPixels 40 --leftMarginPixels 40 --rightMarginPixels 40 cameras_opt.json targets_lateral_opt.json gimbal.json --noWFOVScan*



For the narrow-FOV cameras, target 1 was well centered. Target 2 had a slight shift (see image to the right), but all points were still within the boundaries.

For the wide-FOV cameras, both targets had similar centers and points went all the way to three of the edges; camera 25 had regions of visible points that were not imaged in cameras 22-24 even though they would have been visible. Ran for all cameras. At least one target was listed as being out of bounds (presumably at the edges of the wFOV cameras).

The resulting solution had RMS errors of between 1 and 1.6 for the narrow-FOV cameras and 40-50 for the wides.

The stitching results when using all 25 cameras looked like those for the coarser calibration in calibrate_camera_5r5, with many-pixel shifts along the edges between cameras.

Running the ***single-target solution*** with an edited targets_1_only.json and poses_target_1.csv and writing to 5_opt_target_1.json produced a much cleaner stitch at some of the boundaries. Although it was quite accurate along the edge between two cameras, it seemed to be ***misaligned in the direction across the boundary*** between cameras, as if the scale was off for the cameras.

The ***FOV of all cameras remain exactly the same after optimization***. The *targetCameraMatrix* values are not the same from camera to camera, nor are the *cri.m_fovDegrees* parameters… the discrepancy is because they are being filled into *camerasToOptimize*, but *cameraRenderInfos* is what is used to generate the JSON. Changed these into pointers so that they would update the values in place.

Once the FOVs were updated in the optimized model, and once we used --*staticDepth* to set the render depth to match part of the scene, the ***visible seams matched those expected from the 1-1.6-pixel RMS values reported by the optimizer***.

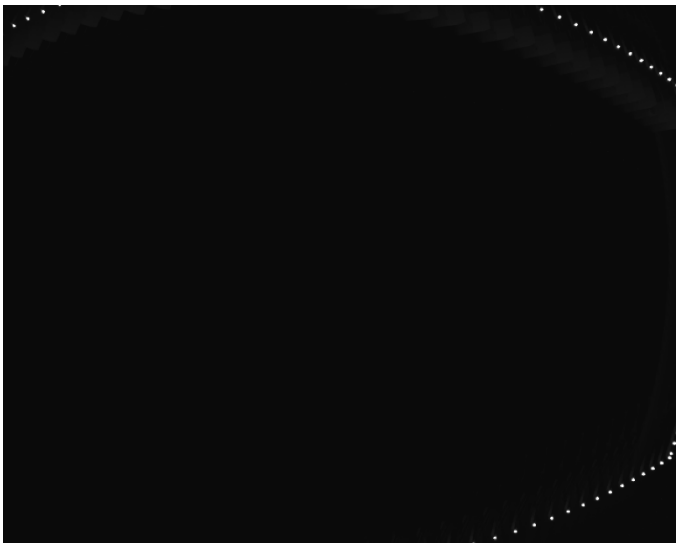
Improving RMS errors: Trying to improve the RMS errors on the narrow-field cameras (1-2) and wFOV cameras (40-50):

- Enable principal point adjustment: 1-2, 32-35
- Enable aspect ratio adjustment: 0.4-1.2, 16-18
- Enable adjustment of both: 0.3-0.7, 16-18

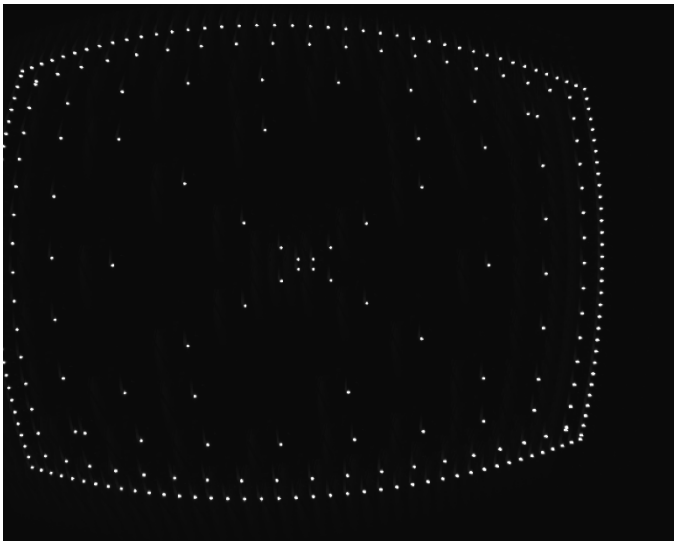
The solution with both enabled seems to work well for the narrow-FOV cameras when rendered with appropriate --*staticDepth* arguments.

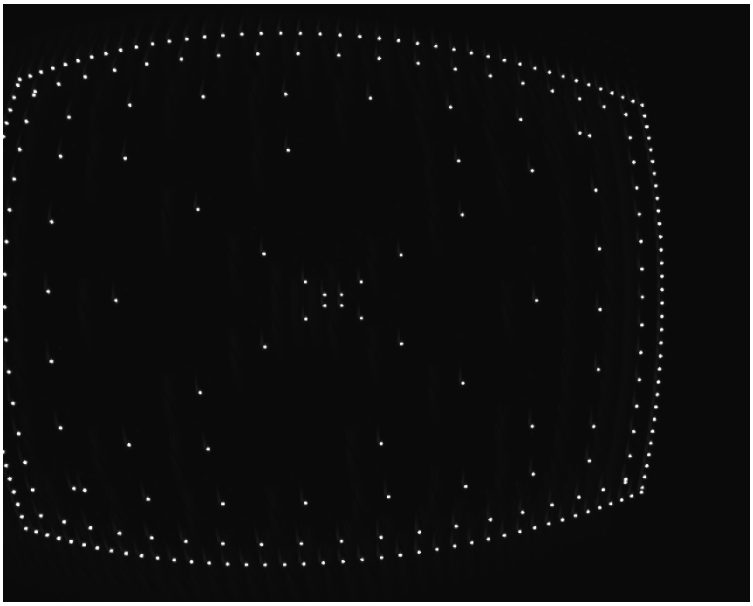
To improve the wFOV lenses, we tried using ***cv::CALIB_RATIONAL_MODEL***: 0.3-0.65, 16-18 (no improvement on wFOV even with new terms).

calibrate_camera_5r7: To improve the wFOV lenses, we tried scanning them deliberately, setting the FOV so that we would collect points across the entire camera. With the initial FOVs of [137.5, 110.0], the points scanned past the top, bottom and sides of the cameras (in addition to being quite distorted). This can be seen in the overlaid image to the right, which was taken before the scan had completed the first full loop around the outside.



Dropping the FOV to [110, 88] produced a scan for target 1 that just skimmed one edge on camera 22 due to the mis-estimation of its original pose (even after doing lateral estimation). Its second ring of points lies far enough from the edge at all locations. Camera 23 was shifted even more but still had its second-line points away from the edge (see image to the right below). Camera 24 was like a mirror image of camera 22. Camera 25 was like a mirror of 23, but slightly more shifted out of frame so that even its second row of pixels came and touched the edge of the image; see image below.





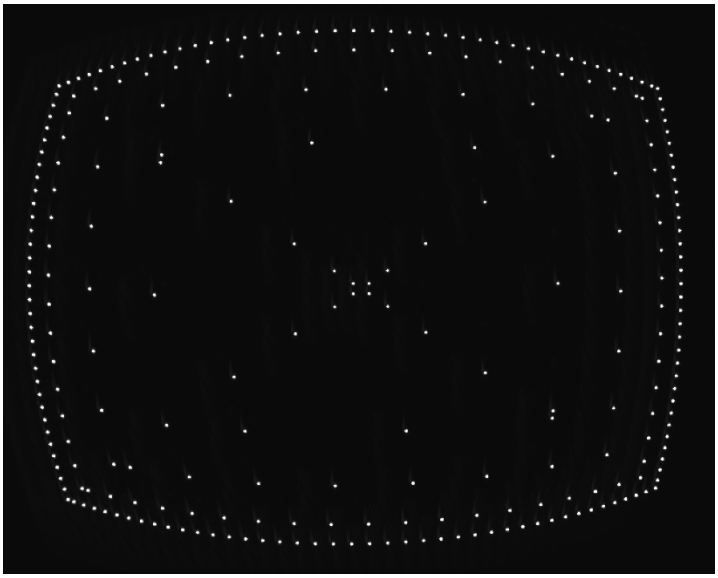
Note: The extra shift in the images is expected due to the incorrect estimation of their fields of view when the lateral calibration was performed. Because we only ran these four cameras for this scan, the lateral estimation was not re-done. For a full scan, these should be centered on the first (farther) target.

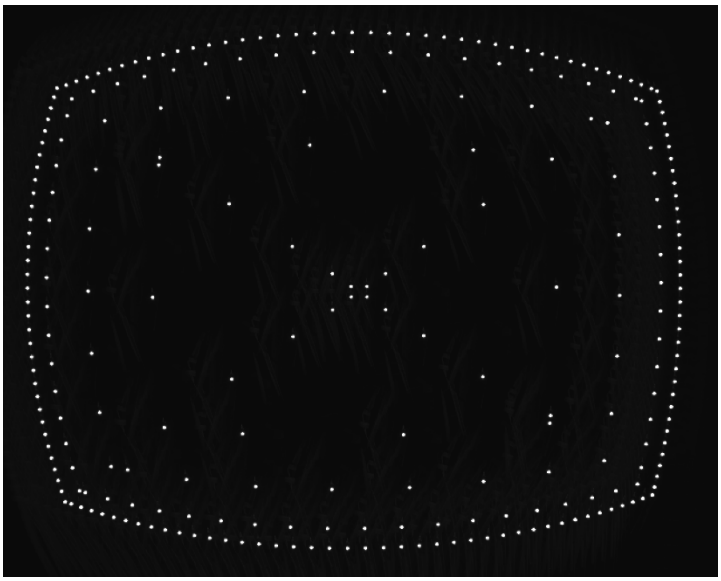
Scans for target 2 were better behaved, with all points in bounds on some cameras and the second line in bounds on all of them.

The solution had RMS errors between 16 and 24, which was not better than the original solutions. The distortion coefficients for cameras 22-24 were small, all magnitude <2. Those for camera 25 were 30+ in some cases.

calibrate_camera_5r8: To avoid missing points and to slightly enlarge the scan size near the corners, the lateral optimization was performed again on the four cameras, using their initial laterally optimized orientations as a starting point. The fields of view were set to [121,96.8]. We used margins of 20 around all sides rather than 40. The images were much better centered. This had points past the edges due to the larger size.

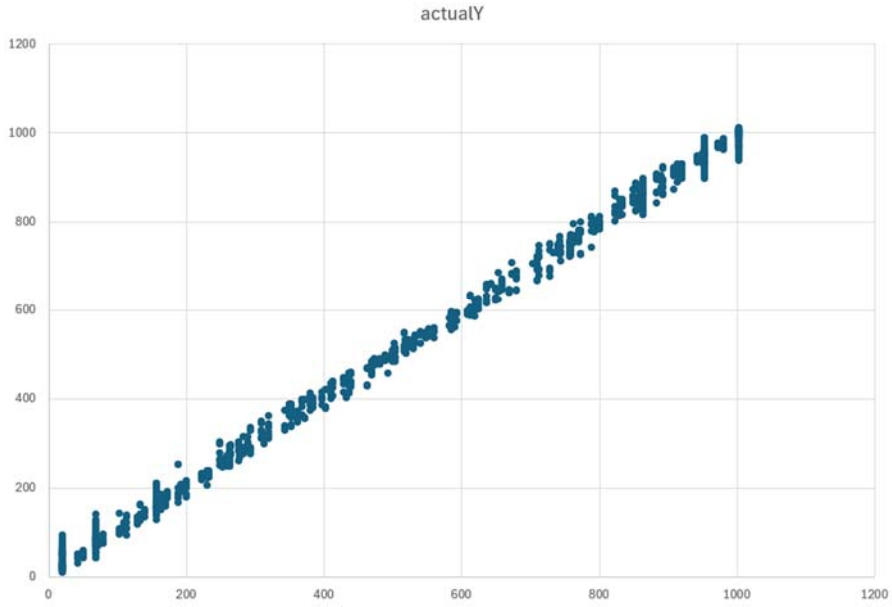
calibrate_camera_5r9: Switched back to FOVs of [110,88], still using margin of 20 on all sides. Re-ran lateral estimation and camera pointing estimation with the different FOVs. All scans of target 2 were well-centered with good margins from the edges. The image below on the right shows the scan for camera 22. The image below on the left shows camera 25 for target 1, which also fits within bounds for all images.



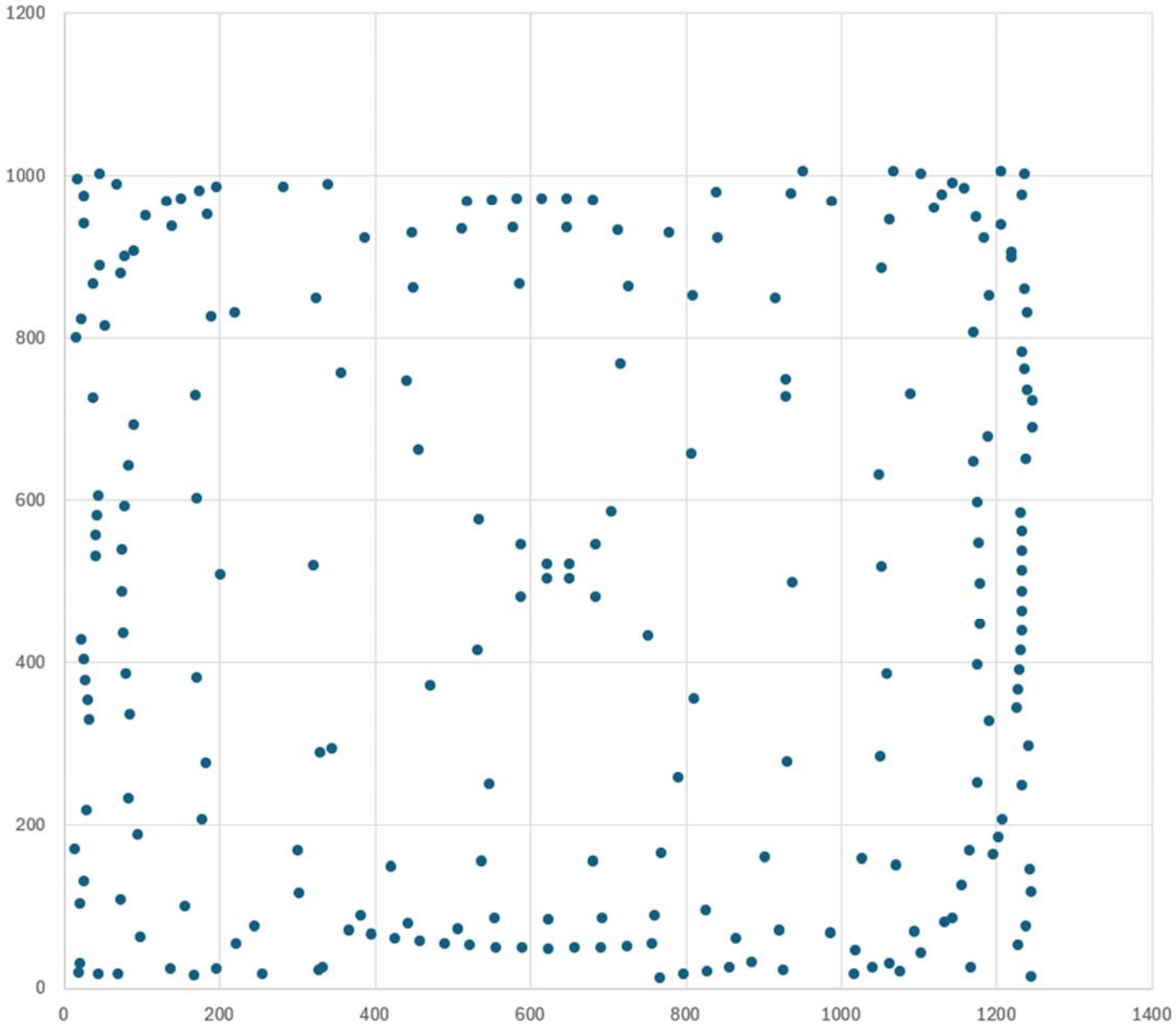


The ***RMS errors for each camera are around 20***, so there is a problem with the procedure or parameters. The laterally-aligned estimated orientations are upside down, as expected – which matches the original estimated orientations and the fact that the cameras are in fact mounted upside down.

The mapping from expected to actual locations in X and Y look as expected, with no large outliers. The Y vs. Y plot is seen to the right.

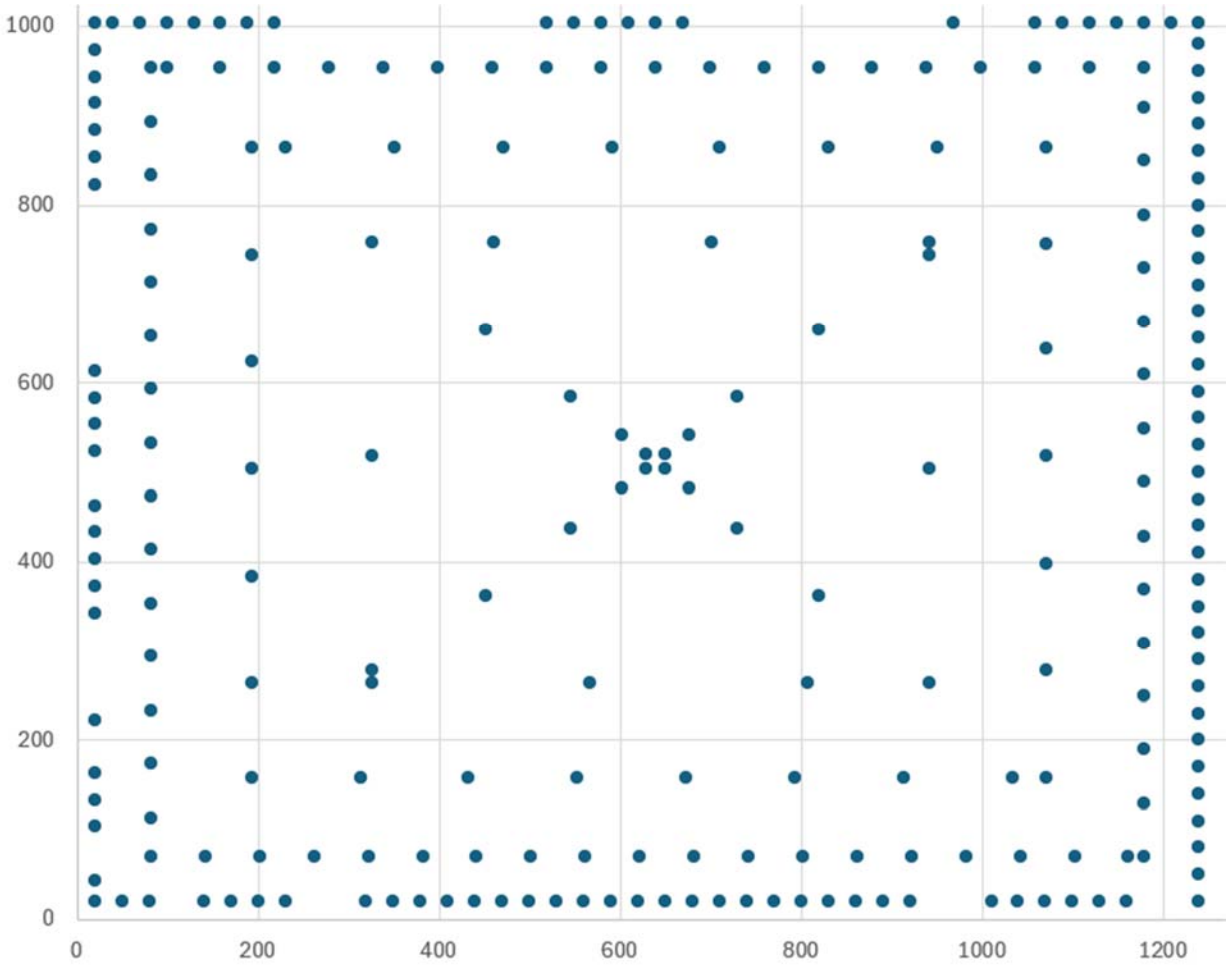


However, the (X,Y) actual-location plot for camera 22 shows that the locations being discovered are not correct (see image below).

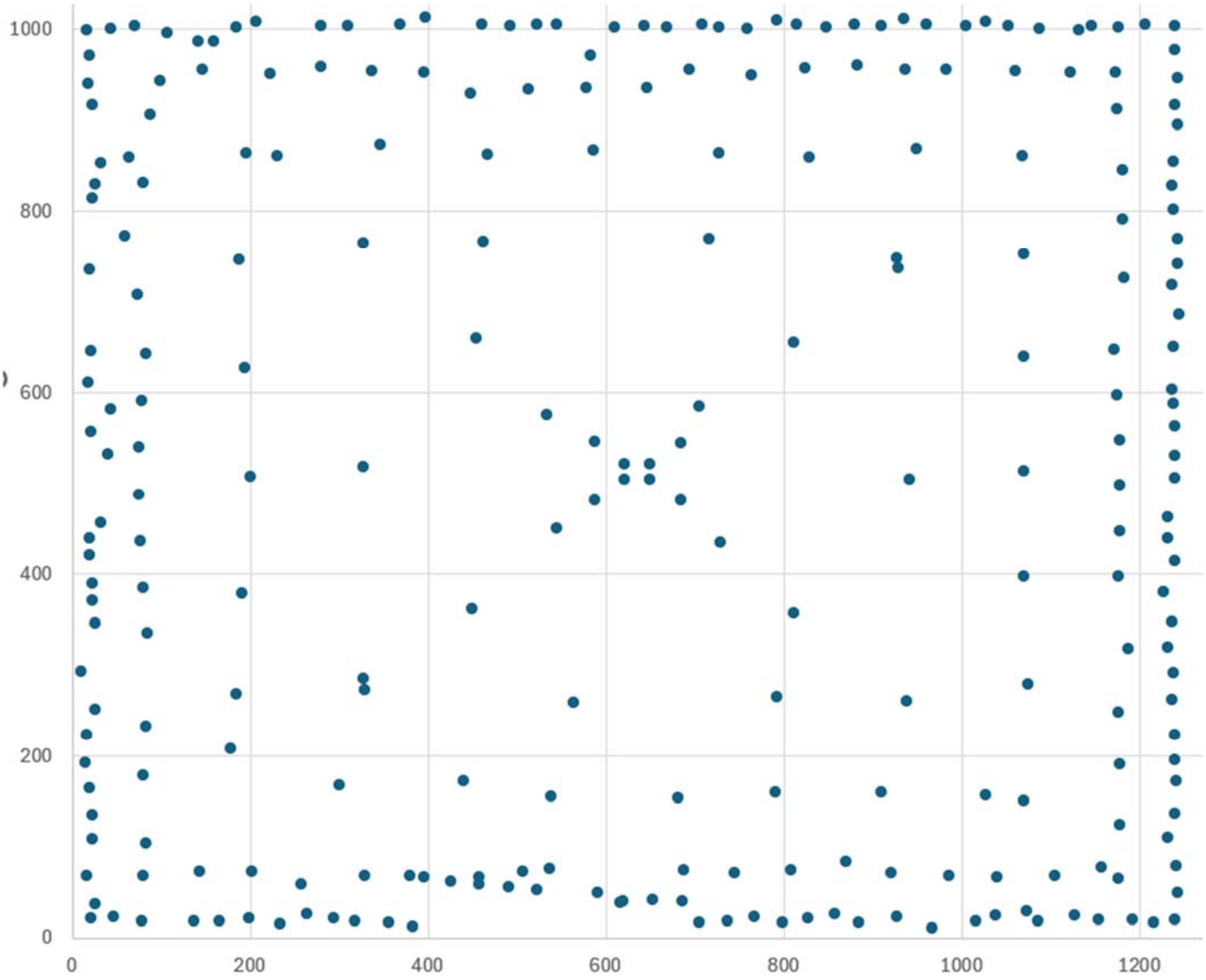


Dropping the brightness threshold from 20000 to 10000 produced similar results. The actual targets are very bright (254/255) on the image. Raising the threshold from 20000 to 60000 also produced similar errors, and similar maps.

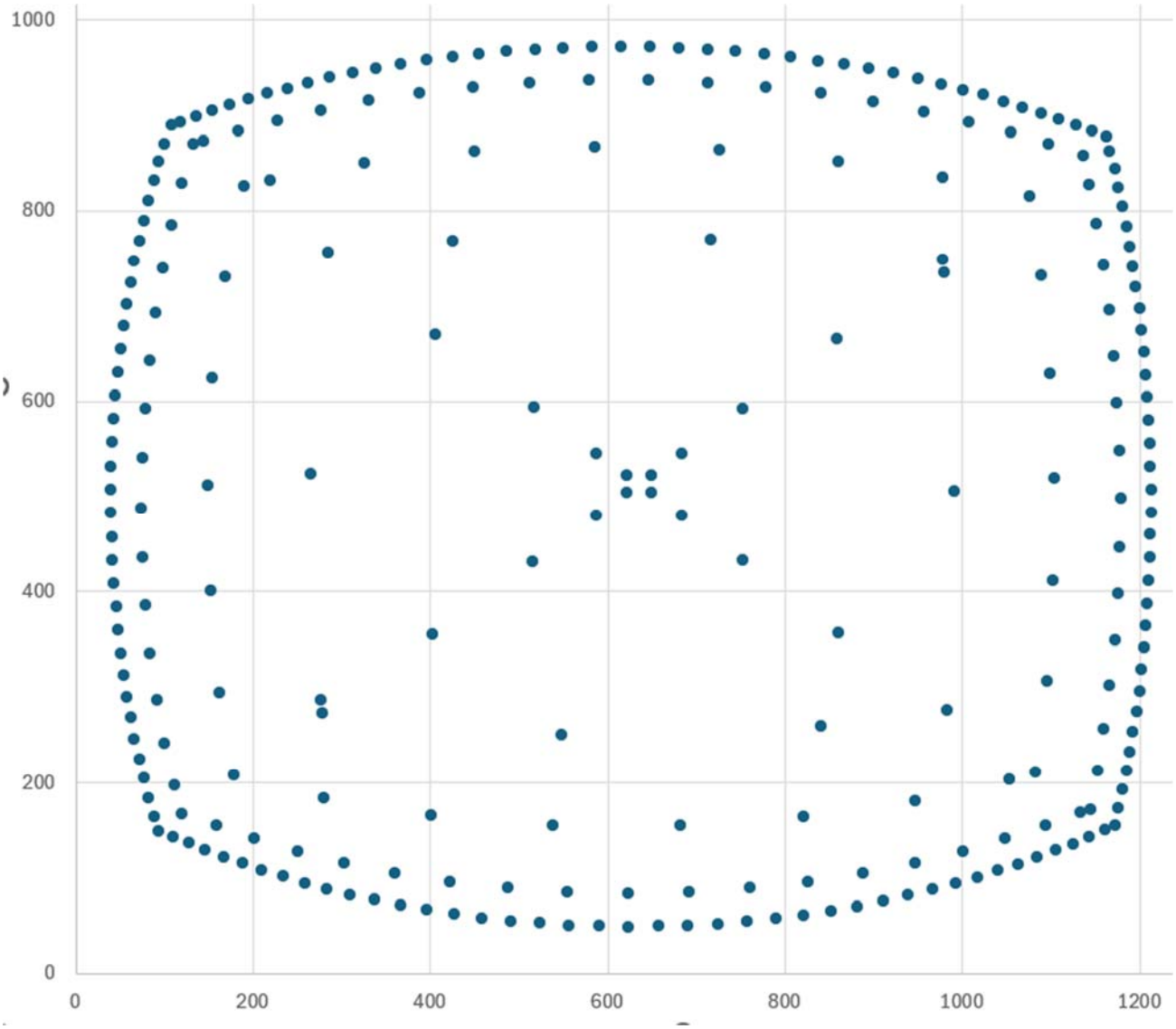
Oddly, some of the points along the outer edge in the expected (X,Y) plot for camera 22 target 1 are missing, perhaps corresponding to messages about out-of-bounds points printed during map construction (presumably because the tracked points wandered to or beyond the image edge).



Changing the cone tracker radius to 3 pixels and the symmetric tracker radius to 5 pixels reduced the RMS errors to around 10 pixels. However, the resulting located positions appear to be more rectangular than expected (we expect them to be curved to match the locations of the bright points):



Note: The problem was that we were adjusting the offset pixels threshold, but the correct one to change was the brightness threshold, which was 1000. Making it 60000 rather than 1000 (because we're multiplying the image values when shifting them) gets the RMS errors down to around 0.2-0.3 and made the located points match the expected pattern:



Switching the tracker radii back to 10 and 10 still maintained RMS errors of between 0.2-0.3 (0.317, 0.267, 0.284, 0.276), with the cone-3 radial-5 radius producing (0.317, 0.284, 0.267, 0.276). It looks like the original values are just as good so long as the threshold is set appropriately.

calibrate_camera_5r10: Re-ran the calibration on all cameras, taking images only for the camera being scanned and using a single target at a time. The resulting average RMS error over all cameras was 0.36 pixels with the largest adjustment of target location being 0.12 meter in X in the far target. The ***k4-k6 coefficients for the narrow-FOV cameras were quite large*** in magnitude (may 10's to 100's) but they were small for the wide-FOV cameras. The target points all fell within the images, though some were close to the edges for some of the narrow-FOV cameras.

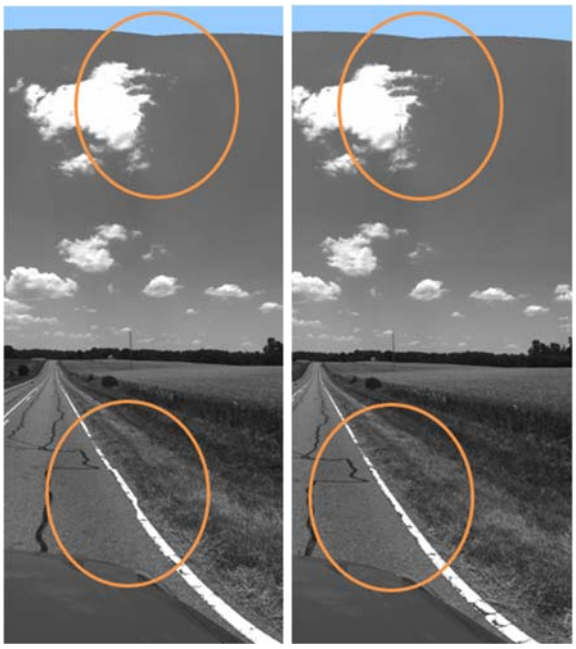
The ***corners of the wFOV cameras have rendering artifacts***, causing them to wrap around to the other side of the images. ***Avoiding the use of k4-k6 on all cameras had an average RMS error of 0.39 pixels. The corner distortions were much more pronounced, covering the whole image with overlapping triangles***.

As of 4/24/2026, the procedure is working using the develop branch of ASDP_Render_Module. Fixes included:

- Using an alpha of 0 rather than 1 when computing optimal camera parameters, clipping to the valid regions to avoid bogus distortion mappings when doing extrapolations.

- Performing depth-estimation rotations in the proper order, which handles the more general case of multiple-axis rotations correctly.

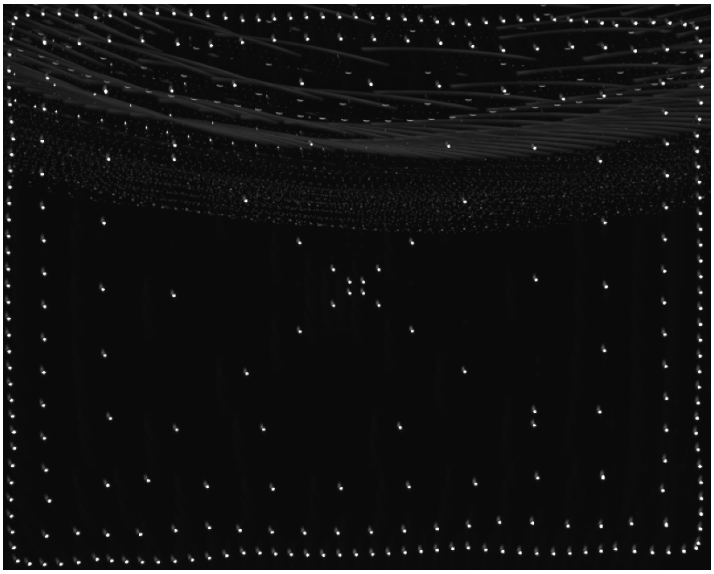
With these changes, we had <1 pixel RMS errors for all cameras. Applying the resulting calibration to a video with a large range of depths showed agreement at both near and far locations: The image to the left shows focus at 2000 meters, which has good alignment in the clouds and poor alignment on the nearby road. The image in the right shows focus at 5m, which shows good alignment on the road and poor alignment in the clouds.



The final command-line argument list for the Windows-based estimation follows (an earlier --writeMaps run had been done):

--offsetThresholdPixels 3000 --readMaps
D:\data\Apache\calibrate_camera_5r10\maps.csv
D:\data\Apache\calibrate_camera_5r10\cameras_opt.json
D:\data\Apache\calibrate_camera_5r10\targets_lateral_opt.json
D:\data\Apache\calibrate_camera_5r10\gimbal.json
D:\data\Apache\calibrate_camera_5r10\poses.csv
D:\data\Apache\calibrate_camera_5r10\camera_images 60000
D:\data\Apache\calibrate_camera_5r10\5_opt.json

calibrate_camera_5r11: On 9/2/2026, an adjusted camera calibration program was used that respects the distortion correction in each camera, and Make_Camera_Config_File.py had a ***--wFOV_Distortion*** command-line option added that constructs a configuration file that includes empirically determined changes in field of view and radial distortion to make it sample points near the edges of the lenses being used for calibration.



These new programs were used to construct new configuration file entries for the wFOV cameras on the ball. The image to the right shows the maximum-projection overlay for all images from one camera, which samples points near the edges while keeping all points within the image.

Camera 24 had points that came right to the edges of the image, which may impair the calibration to some extent; they should be ignored by the existing algorithm and the fact that we do a radial distortion fit for 3D calibration should make us robust to these missing points.

The final command-line argument list for the Windows-based estimation follows:

--offsetThresholdPixels 200000 D:\data\Apache\calibrate_camera_5r11\wide\cameras_opt.json
D:\data\Apache\calibrate_camera_5r11\wide\targets_lateral_opt.json
D:\data\Apache\calibrate_camera_5r11\wide\gimbal.json D:\data\Apache\calibrate_camera_5r11\wide\poses.csv
D:\data\Apache\calibrate_camera_5r11\wide\camera_images 60000
D:\data\Apache\calibrate_camera_5r11\wide\wide_opt.json

The average RMS error among the four cameras was 0.272 pixels, with the maximum being 0.286.

One of the cameras had **wrap-around distortion** at its edge, which pulled back across the image. Another corner of that pair had a lot of distortion at a neighboring corner. ***Removing the CALIB_RATIONAL_MODEL flag from OpenCV*** made the RMS errors slightly higher (0.278) but solved this problem.

The resulting depth map was **upside down** w.r.t. the camera imagery. Looking at the camera poses, they were flipped in both position and rotation compared to the original solution – it is probably the case that our initial model was not upside down even though the camera is mounted upside down on the calibration jig. Flipping the optimized camera model and running estimation against that model produced pixels behind the cameras, so failed to solve. However, ***flipping the final optimized result worked*** to align the images and depth calculations.

Validation on composite LWIR + wFOV ball

Camera 6 has visible-light wFOV cameras and LWIR narrow-field cameras. Calibration was performed in a long hallway on 7/16/2026 with two bright targets for the wFOV and two blackbody targets for the narrow-field cameras.

Narrow-field cameras: The narrow-field cameras had some flickering behavior during collection that caused some frames to be missed: 16 no-targets errors across cameras 20, 9, 4, 13, 2, 14; 17 too-far errors across cameras 4, 5, 7, 20, 17, 3.

The command used to calibrate the narrow-field cameras was as follows:
Camera_Calibration_Estimate_Distortion_Extrinsics --invert --offsetThresholdPixels 50 --writeMaps
D:\data\Apache\calibrate_camera_6_3D\narrow\maps.csv
D:\data\Apache\calibrate_camera_6_3D\narrow\cameras_opt.json
D:\data\Apache\calibrate_camera_6_3D\narrow\targets_lateral_opt.json
D:\data\Apache\calibrate_camera_6_3D\narrow\gimbal.json
D:\data\Apache\calibrate_camera_6_3D\narrow\orig_poses.csv
D:\data\Apache\calibrate_camera_6_3D\narrow\camera_images 50000
D:\data\Apache\calibrate_camera_6_3D\narrow\6_narrow_opt.json

The resulting RMS errors were less than 1 for most cameras and as large as ~2.2:

- Camera ID 13 RMS error: 0.350479
- Camera ID 17 RMS error: 0.288154
- Camera ID 14 RMS error: 0.22333
- Camera ID 11 RMS error: 0.349837
- Camera ID 2 RMS error: 0.280521
- Camera ID 10 RMS error: 0.410605
- Camera ID 16 RMS error: 0.567571
- Camera ID 5 RMS error: 0.383886
- Camera ID 15 RMS error: 0.966249
- Camera ID 9 RMS error: 1.01253
- Camera ID 7 RMS error: 0.305557
- Camera ID 21 RMS error: 1.01537
- Camera ID 3 RMS error: 1.03116
- Camera ID 8 RMS error: 0.236444
- Camera ID 1 RMS error: 0.302099
- Camera ID 20 RMS error: 0.624702
- Camera ID 19 RMS error: 2.16903

- Camera ID 6 RMS error: 1.11685
- Camera ID 12 RMS error: 1.16749
- Camera ID 18 RMS error: 0.818929
- Camera ID 4 RMS error: 0.34726

Wide-field cameras: The cameras seemed to have only the bright spot be at the maximum intensity, enabling us to use a very high brightness threshold. The distance threshold scales by the FOV ratio, so we chose one to try and avoid removing points that were very distorted. The command used was:

Camera_Calibration_Estimate_Distortion_Extrinsics --offsetThresholdPixels 1000 --writeMaps
D:\data\Apache\calibrate_camera_6_3D\wide\maps.csv
D:\data\Apache\calibrate_camera_6_3D\wide\cameras_opt.json
D:\data\Apache\calibrate_camera_6_3D\wide\targets_lateral_opt.json
D:\data\Apache\calibrate_camera_6_3D\wide\gimbal.json D:\data\Apache\calibrate_camera_6_3D\wide\poses.csv
D:\data\Apache\calibrate_camera_6_3D\wide\camera_images 65500
D:\data\Apache\calibrate_camera_6_3D\wide\6_wide_opt.json

This produced **very large RMS errors** across all cameras:

- Camera ID 22 RMS error: 1.30885
- Camera ID 23 RMS error: 6.38434
- Camera ID 24 RMS error: 5.98727
- Camera ID 25 RMS error: 5.41622

and **what appear to be invalid distortion mappings (initial term negative) for cameras 23-25**:

- distCoeffs for camera 22: 10.2894, -1.44859, 0.000274168, -0.000110279, -0.188789, 10.4154, 0.785475, -0.68468, 0, 0, 0, 0, 0, 0
- distCoeffs for camera 23: -1.64437, 0.60915, 0.000120236, -0.000694679, 0.0557051, -1.42917, 0.239027, 0.214716, 0, 0, 0, 0, 0, 0
- distCoeffs for camera 24: -1.77569, 0.709071, 0.000478581, -0.000402493, 0.0714856, -1.55793, 0.304378, 0.259397, 0, 0, 0, 0, 0, 0
- distCoeffs for camera 25: -0.563773, 0.0691811, 0.000604216, -0.000569733, 0.00753111, -0.355126, -0.0622445, 0.0307642, 0, 0, 0, 0, 0, 0

The camera rotation adjustments made during the initial lateral target estimation procedure moved the rotations from around 48 degrees to around 40 degrees (same sign) for all four cameras, with one at 39 and one at 41 on each side. The pose and orientation selected for each was on the same half of the camera and in the same relative direction.

The problem was that **offset in Z for camera 22 was set at 0.4 meters rather than 0.04**, which caused both camera and target pose estimation errors that presumably rippled through the calculations.

Re-running the calibration with all cameras starting with a Z of 0.04 produced a result whose RMS errors are ~0.37:

- Camera ID 25 RMS error: 0.379864
- Camera ID 23 RMS error: 0.346516
- Camera ID 22 RMS error: 0.373839
- Camera ID 24 RMS error: 0.367231

The 9_join.sh script was used to combine the narrow and wide results into a common file describing all cameras.

The image above shows two of the wide-field cameras stitched together at a static depth of 1.5 meters. The fact that the ceiling line is straight across the image indicates that the overall distortion is corrected. The alignment of the two cameras on the door at the center of the image indicates that they are jointly calibrated. When stitched at further or nearer depths, the images diverge as expected due to the parallax effect.

The narrow-field cameras also stitched well, here at a depth of 3 meters.